# C 语言考试大纲与复习

## 考试内容与要求

### 一、C 语言基础

1. 了解程序、程序设计语言和算法的基本概念；理解算法的基本特性

> **解析：**
>
> - **程序** = 数据结构 + 算法。**算法**是解决某一问题的有限步骤序列。
> - 算法五大特性：**有穷性**（有限步内结束）、**确定性**（每步含义唯一、无二义）、**可行性**（每步都可由基本运算实现）、**输入**（0 个或多个）、**输出**（1 个或多个，必须至少有一个）。
> - 三种基本结构：**顺序、选择、循环**。它们可以组合出任何结构化算法。
> - 算法描述方式：自然语言、流程图、伪代码、程序代码。

2. 掌握 C 程序的基本组成、`main` 函数、头文件、注释和语句的书写规则；了解编辑器、编译器、链接器和调试器各自的基本作用。

> **解析：**
>
> - 一个 C 程序**有且仅有一个 `main` 函数**，程序从 `main` 的第一条可执行语句开始，到 `main` 的 `}` 结束。
> - 基本组成：预处理命令（`#include`）→ 全局声明 → 函数定义。
> - 头文件：`<stdio.h>`（输入输出）、`<stdlib.h>`（`malloc`/`free`/`abs`）、`<string.h>`（字符串函数）、`<ctype.h>`（字符判断/转换）、`<math.h>`（数学函数）。
> - 注释：`/* ... */` 块注释（可跨行），`// ...` 行注释（C99 起）。注释在预处理阶段被替换为一个空格，**注释中不能嵌套块注释**。
> - 每条语句以 `;` 结尾；**预处理命令和函数头不以 `;` 结尾**。
> - 工具链分工（一条流水线，前一个的输出是后一个的输入）：
>   - **编辑器（Editor）**：面向程序**写**源代码，负责文本编辑、语法高亮、缩进与查找替换。产物：`.c` / `.h` 纯文本文件。**只产生文本，不检查对错**。
>   - **编译器（Compiler）**：把**人能读**的源代码翻译成**机器能执行**的目标代码；同时做**语法检查、类型检查**，把错误报出来。产物：`.o` / `.obj` 目标文件。
>   - **链接器（Linker）**：把多个目标文件和库**拼装**成一个完整可执行程序，负责解析“函数在哪里”的引用、合并同名段、分配最终地址。产物：`.exe` / `.out`。
>   - **调试器（Debugger）**：把可执行程序**停下来慢放**，可设断点、单步执行、查看变量、观察调用栈。它不改代码，只在编译时加上 `-g` 信息后才能用。
> - **一句话区分**：编辑器管“写”，编译器管“翻译并查错”，链接器管“拼装”，调试器管“排错”。
> - **必须知道的先决条件**：一个完整的可执行程序是“**一个编辑器 → 一个或多个编译器 → 一个链接器**”的产物。主调与自定义函数分放在多个 `.c` 里时，各自编译成 `.o`，最后由链接器合并。

3. 理解 C 程序通常经过预处理、编译、汇编和链接生成可执行文件的过程，了解各阶段的主要任务及源文件、目标文件、库文件和可执行文件的区别。

> **解析：**

- **一眼看懂完整流水线**（以 `main.c` 和 `util.c` 为例）：

```
flowchart LR
    A["main.c
util.c
源码"] -->|预处理 cpp| B["main.i
util.i
预处理后"]
    B -->|编译 cc1| C["main.s
util.s
汇编代码"]
    C -->|汇编 as| D["main.o
util.o
目标文件"]
    D -->|链接 ld| E["a.exe
可执行文件"]
    F["库文件
libc.a / libc.so"] -->|链接 ld| E
```

- **四个阶段对比表**（重点）

| 阶段 | 工具（GCC 下） | 输入 → 输出 | 主要任务 | 错误特点 |
|---|---|---|---|---|
| 预处理 | `cpp` | `.c` → `.i` | 展开 `#include`、宏替换、条件编译、去注释 | 找不到头文件 |
| 编译 | `cc1` | `.i` → `.s` | 词法/语法/语义分析、优化、生成汇编 | **语法错、类型错** |
| 汇编 | `as` | `.s` → `.o` | 汇编指令 → 机器指令，生成符号表 | 极少出错 |
| 链接 | `ld` | `.o` + 库 → `.exe` | 符号解析、重定位、合并段 | **未定义/重复定义** |

- **GCC 常用命令对照**（了解即可，帮助理解各阶段）：

```
gcc -E main.c -o main.i    # 只做预处理
gcc -S main.c -o main.s    # 只到编译（生成汇编）
gcc -c main.c -o main.o    # 只到汇编（生成目标文件）
gcc main.o util.o -o app   # 链接（也可 gcc main.c util.c -o app 一步到位）
```

- **常见文件扩展名与作用**：

| 文件 | 典型扩展名 | 内容形式 | 能否直接运行 | 作用 |
|---|---|---|---|---|
| 源文件 | `.c`（实现）、`.h`（声明） | 文本，人可读 | 否 | 存放源代码 |

| 文件 | 典型扩展名 | 内容形式 | 能否直接运行 | 作用 |
| --- | --- | --- | --- | --- |
| 预处理后文件 | `.i` | 文本，已展开宏 | 否 | 预处理中间产物 |
| 汇编文件 | `.s` / `.asm` | 文本，汇编指令 | 否 | 编译中间产物 |
| 目标文件 | `.o` / `.obj` | 二进制，含机器码 + 符号表 | 否 | 可重定位，有未解析引用 |
| 库文件 | `.a` / `.so` / `.lib` / `.dll` | 二进制 | 否 | 预编译好的函数集合 |
| 可执行文件 | `.exe` / `.out` | 二进制，已重定位 | **是** | 可交给操作系统装入运行 |

> - **库文件分两种**（易考）：
>   - **静态库**（`.a` / `.lib`）：链接时把用到的代码**复制**进可执行文件，产物大、但可独立运行。
>   - **动态库**（`.so` / `.dll`）：只记录引用，运行时才加载，产物小、可共享。
> - **目标文件为何不能直接运行**：① 代码还没固定到最终地址（可重定位）；② 调用的库函数地址还没填上（未解析符号）；③ 缺少启动代码（`_start` 与 `main` 的衔接）。
> - **链接器要做的三件事**：① **符号解析**（把每个外部引用对应到某处定义）；② **重定位**（把符号的相对地址换成最终地址）；③ **合并段**（把多个 `.o` 的代码段、数据段合到一起）。
> - **分块编译的好处**：改了 `util.c` 只需重编 `util.o`，再重新链接，不必重编 `main.c`。这也解释了为何要区分**声明（`.h`）与定义（`.c`）**。

4. 了解编译器进行词法分析、语法分析、语义分析和代码生成的基本思路，知道编译优化的基本用途；能结合简单示例理解标识符识别、语法检查和类型检查，不要求实现编译器。

> **解析**：
>
> - **词法分析**：把字符流切成**记号（token）**，如关键字、标识符、常量、运算符。例如 `int a=1;` → `int` / `a` / `=` / `1` / `;`。
> - **语法分析**：按文法把记号组织成语法树，检查结构是否合法。例如缺 `;`、括号不配对报错。
> - **语义分析**：类型检查（如把浮点赋给指针、参数个数不符）、作用域检查、是否已声明。
> - **代码生成与优化**：生成目标代码；优化手段有常量折叠（`2*3` 直接算成 `6`）、公共子表达式消除、循环不变式外提。**优化不改变程序的语义**。
> - 记忆要点：**词法错** ≈ 出现了不合法的字符/记号；**语法错** ≈ 结构不合法；**语义错** ≈ 结构合法但类型/含义不合法。

5. 了解操作系统在程序装入、运行和结束过程中的基本作用，理解程序与进程的区别；了解进程地址空间、内存访问保护和文件管理的基本概念。

> **解析**：
>
> - **程序**是磁盘上的**静态**文件；**进程**是程序的一次**动态**执行实例，具有独立的地址空间和运行状态。一个程序可对应多个进程。
> - 进程地址空间自低到高大致为：**代码段（.text）→ 已初始化数据段（.data）→ 未初始化数据段（.bss）→ 堆（heap，向上增长）→ 空闲区 → 栈（stack，向下增长）→ 命令行参数/环境变量**。

- 变量落在哪一段：全局/静态已初始化 → `.data`；全局/静态未初始化（或初值为 0）→ `.bss`；`malloc` 得到的内存 → 堆；局部自动变量 → 栈；字符串常量 → 只读数据段。
- 内存访问保护：操作系统以页为单位做权限控制（只读/读写/可执行），越界访问触发**段错误（Segmentation Fault）**。
- 文件管理：操作系统通过**文件描述符/文件表**统一管理打开的文件，C 运行库的 `fopen` 最终要请求操作系统的系统调用。

6. 了解 C 程序通过运行库与操作系统获取内存、访问文件和进行输入输出的基本关系；理解标准输入、标准输出和标准错误的基本用途，能区分文件路径中的相对路径与绝对路径。

**解析：**

- **核心思路：把“运行库”看成 C 程序与操作系统之间的“翻译官/传达室”。**
  - 操作系统只认识**系统调用**（一整套低级、原始、与具体系统绑定的接口），不认识 `printf`、`scanf`、`fopen` 这些名字。
  - C 程序也不愿意直接写系统调用（不同系统名字、参数都不同，写出来的代码不能跨平台）。
  - 于是中间插了一层：**C 标准库（运行库，libc）**。它把“人用的函数”与“系统用的调用”一一对应起来，并用标准化的名字对外提供接口。
  - 类比：你（程序）只会说“标准汉语”（`printf`），操作系统只会听“方言”（`write` 系统调用），运行库就是中间的**翻译**。换到另一个操作系统，你只需换一套运行库，程序源码不用改。
- **三层调用关系（自顶向下）：**

```
flowchart TD
    A["应用程序 user code
printf / scanf / fopen / malloc"] --> B["C 标准库 libc
缓冲、格式化、错误码转换"]
    B --> C["操作系统内核 kernel
系统调用 write / read / open / brk"]
    C --> D["硬件
屏幕 / 键盘 / 磁盘 / 内存"]
```

- **四条典型调用链（看“用户函数”与“系统调用”的对应关系）：**

| C 库函数（用户层） | 运行库做的事 | 底层系统调用 | 涉及硬件 |
| --- | --- | --- | --- |
| `printf` / `fputs` | 格式化字符串、写入用户缓冲区 | `write` | 屏幕 |
| `scanf` / `fgets` | 从用户缓冲区取数据、解析格式 | `read` | 键盘 |
| `fopen` / `fclose` | 维护 `FILE` 结构与文件表项 | `open` / `close` | 磁盘 |
| `malloc` / `free` | 从内存池切块、记录空闲块 | `brk` / `mmap` | 内存 |

- **关键差别：库函数 ≠ 系统调用：**
  - `printf` 是**库函数**，`write` 才是**系统调用**。库函数是“外套”，里面可以包一层甚至多层的缓冲与处理。
  - **缓冲是运行库加的**，操作系统不知道有缓冲区这回事。这也是为何 `printf` 不加 `\n` 可能不立即显示，而 `write` 是直接落盘/落屏。
- **为什么要这层中间层（好处）：**

| 角度 | 说明 |
| --- | --- |
| **可移植** | 源码只依赖标准库，换系统只需换库；`printf` 在 Windows 和 Linux 上写法一样 |
| **效率** | 运行库做**缓冲**：攒一批再交系统调用，减少昂贵的模式切换（用户态↔内核态） |
| **好用** | 提供格式化（`printf`）、错误码、`FILE` 抽象等，不用自己拼字符串、自己管内存块 |
| **安全** | 运行库可帮你做参数检查、缓冲区边界、分配失败的统一报告 |

- **三个标准流**（`<stdio.h>` 中预定义，无需打开，它们的类型是 `FILE *`）：
  - `stdin` 标准输入（键盘，文件描述符 **fd 0**）。
  - `stdout` 标准输出（屏幕，文件描述符 **fd 1**）——**有缓冲**，所以可能滞后显示。
  - `stderr` 标准错误（屏幕，文件描述符 **fd 2**）——**无缓冲**，所以报错信息总能第一时间看到（这是故意的）。
  - 程序启动时操作系统自动打开这三个流，退出时自动关闭。
- **它们到底是什么？** 三个**全局的 `FILE *` 变量**（宏），类型和 `FILE *fp = fopen(...)` 里的 `fp` **完全一样**：

```
extern FILE *stdin;    /* 在 <stdio.h> 中声明，程序启动时已指向键盘 */
extern FILE *stdout;   /* 已指向屏幕 */
extern FILE *stderr;   /* 已指向屏幕 */
```

  正因为类型相同，所以 `scanf`/`printf` 能做的事，用 `fscanf(stdin, ...)`/`fprintf(stdout, ...)` 一样能做。
- **为什么平时写代码几乎用不到？** 因为一批函数已经把三个标准流**隐式绑定好了**，帮你省掉了写名字：

| 你平时写的（隐式用标准流） | 完全等价的显式写法 |
| --- | --- |
| `printf(...)` | `fprintf(stdout, ...)` |
| `scanf(...)` | `fscanf(stdin, ...)` |
| `getchar()` | `fgetc(stdin)` |
| `putchar(c)` | `fputc(c, stdout)` |
| `gets` / `puts(s)` | `fgets(s, ..., stdin)` / `fputs(s, stdout)` |

- **什么时候才真的会写 `stdin`/`stdout`/`stderr`？** 这四种场景（很常见，建议掌握第一个和第三个）：
  1. **读一整行含空格的字符串**——只能用 `fgets`，必须传流：

```
char line[100];
fgets(line, sizeof(line), stdin);    /* 不能用 scanf("%s") 代替 */
```

  2. **把格式化内容输出到文件**（复用 `printf` 的格式串）——用 `fprintf`，这时要写 `stderr` 或 `fp`：

```
fprintf(fp, "%d %s\n", n, name);   /* 写文件 */
```

3. **输出错误信息用 `stderr`**（而不是 `printf`）——因为 `stderr` 无缓冲且不会被重定向到结果文件：

```
fprintf(stderr, "错误：文件打开失败\n");
```

4. **重新把标准流当普通文件用**——如 `rewind(stdin)` 清空输入缓冲。

- **与文件描述符的区别**（易混）：`fd 0/1/2` 是**操作系统层**的整数编号，`stdin`/`stdout`/`stderr` 是**C 运行库层**的 `FILE *`。平时写代码只用后者，前者只在系统调用（`read(0, ...)`）里出现。
- **标准流能重定向**（这正是"它们只是流"的体现）：
  - `a.exe > out.txt` 把 `stdout` 改写到文件；`a.exe < in.txt` 把 `stdin` 改为从文件读。
  - 因为 `stdin`/`stdout` 只是"**流**"，不一定非是键盘/屏幕，所以可以换成文件或管道。
  - **注意**：重定向只影响 `stdin`/`stdout`，**不影响 `stderr`**——所以错误信息仍然会显示到屏幕，这正好是第 3 种场景的用意。
- **相对路径 vs 绝对路径**：
  - **相对路径**以**当前工作目录（CWD）** 为基准，如 `data.txt`、`../a/b.txt`；同一份代码在不同目录下运行会打开**不同文件**，这是常见的坑。
  - **绝对路径**从根目录写起，如 Windows 的 `E:\C_learning\a.txt`、类 Unix 的 `/home/user/a.txt`；不随当前目录变化。
  - 注意路径分隔符差异：**Windows 用反斜杠（`\`），类 Unix 用正斜杠（`/`）**。在 C 字符串里反斜杠是转义前导，所以写一个 Windows 路径需要**每处都写两个**，如 `"E:" + "\\" + "C_learning" + "\\" + "a.txt"`；也可以直接用正斜杠 `"E:/C_learning/a.txt"`，多数环境都认。
- **一句话记忆**：C 程序不直接"碰"硬件，它只调用运行库；运行库再去请求操作系统的系统调用。因此换平台只需换运行库，程序源码基本不动。

7. 能用自然语言、流程图或伪代码描述简单算法；能区分编译错误、链接错误、运行时错误和逻辑错误，根据基本诊断信息及题意检查输入、处理过程和输出。

**解析**：

- 四类错误对照表：

| 错误类型 | 出现阶段 | 典型现象 | 例子 |
|---|---|---|---|
| 编译错误 | 编译期 | 无法生成目标文件 | 缺 `;`、括号不配对、类型不匹配 |
| 链接错误 | 链接期 | 有未解析的外部符号 | 函数名拼写错、忘记链接库、只声明未定义 |
| 运行时错误 | 运行时 | 程序崩溃或异常中止 | 除数为 0、数组越界、空指针解引用 |
| 逻辑错误 | 运行时 | 程序能运行但结果错 | `=` 误写成 `==`、循环边界写错、算法写错 |

- 排查思路：**先看输入是否符合预期 → 再追踪处理过程 → 最后核对输出**。逻辑错误编译器不报错，只能靠对照手算结果或插桩打印定位。

> **提示**：本部分将结合 C 程序的编译和运行过程，考查与程序设计有关的计算机基础知识。所以不管你们看什么视频，用什么教材，学习时应认真理解引言、导论及运行环境等内容，了解程序从源代码到实际运行所经历的过程，以及各类开发工具和操作系统在其中的作用。

## 二、数据类型与表达式

1. 掌握标识符、常量和变量的使用，整型、浮点型和字符型数据的表示与初始化；理解有符号与无符号整数的基本区别。

> **解析**：
>
> - **标识符命名**：只能由字母、数字、下划线组成；**不能以数字开头**；区分大小写；不能是关键字；长度上编译器通常只前 31 个字符有意义。
> - 常见类型与典型宽度（32/64 位常见实现）：

| 类型 | 典型字节数 | 取值范围（有符号） |
|---|---|---|
| `char` | 1 | −128 ~ 127 |
| `short` | 2 | −32768 ~ 32767 |
| `int` | 4 | −2147483648 ~ 2147483647 |
| `long long` | 8 | ≈ ±9.22×10$^{18}$ |
| `float` | 4 | 约 6~7 位有效数字 |
| `double` | 8 | 约 15~16 位有效数字 |

> - **有符号与无符号**：`unsigned` 把全部位用于表示数值，最大值为 $2^n - 1$。
> - **补码（重点，必会手算）**：现代计算机的**有符号整数一律用补码**表示，最高位是符号位（0 正 1 负），但它同时也是**数值位**。

> **① 三种编码的对比（以 8 位为例）**

| 真值 | 原码 | 反码 | 补码 |
|---|---|---|---|
| +5 | `0000 0101` | `0000 0101` | `0000 0101` |
| −5 | `1000 0101` | `1111 1010` | `1111 1011` |
| +0 | `0000 0000` | `0000 0000` | `0000 0000` |
| −0 | `1000 0000` | `1111 1111` | **不存在**（与 +0 都是 `0000 0000`） |
| 范围 | −127 ~ +127 | −127 ~ +127 | **−128 ~ +127** |

> - **原码**：最高位符号位，其余位是真值的绝对值。缺点：**0 有两种表示**，且加减法要分情况。
> - **反码**：正数同原码；负数 = 原码符号位不变、其余按位取反。缺点：**0 仍有两种表示**，且会有「+0/−0」的歧义。
> - **补码**：正数同原码；负数 = 反码 + 1。优点：**0 的表示唯一**，且**减法可以当加法算**（这是它被采用的根本原因）。
>
> **② 求负数补码的两条路径（结果一样，任选）**
>
> **路径 A：符号位不变，其余取反，末位加 1**（教材常写"取反加一"）

```
求 -5 的补码：
  -5 的原码          1000 0101
  符号位不变，取反   1111 1010      ← 这一步得到反码
  末位加 1            1111 1011      ← 得到补码
```

**路径 B：先求绝对值的二进制，再"从右往左，第一个 1 及其右边不变，左边全部取反"**（更快）

```
求 -5 的补码：
  5 = 0000 0101
  从右往左找第一个 1 → 是末尾的 1，它和它右边（没有）保持不变
  它左边的位全部取反 → 1111 1011
```

**记忆口诀：正数照抄；负数 = 取反加一**（注意是**整个数的反码再加 1**，加 1 产生的进位要正常往前进）。

**③ 补码转真值（反过来算）**

- **方法 1：看符号位**。若最高位是 0，直接当无符号数读；若是 1，把它当作「取反加一再加负号」或直接用公式：

$$X = -(\text{符号位}) \times 2^{n-1} + \sum_{i=0}^{n-2} b_i \cdot 2^i$$

- **方法 2（推荐，最快）：补码的补码就是原码**。所以对 `1111 1011` 再求一次补码：

```
1111 1011 → 取反得 0000 0100 → 加 1 得 0000 0101 = 5 → 加负号 = -5
✓
```

- **小技巧**：也可以「最高位当 −128，其余位当正权相加」：`1111 1011` = −128 + 64 + 32 + 16 + 8 + 0 + 2 + 1 = **−5** ✓

**④ 为什么补码能「减法当加法」？**

以 8 位算 $5 - 5$ 为例，把它变成 $5 + (-5)$：

```
5 的补码     0000 0101
-5 的补码 + 1111 1011
──────────────────────
结果         10000 0000
丢弃最高位的溢出进位后 → 0000 0000 = 0 ✓
```

- **关键的「模」**：$n$ 位补码本质是在**模 $2^n$** 下做算术。超出 $n$ 位的进位**自然丢弃**，正好对应「减去 $2^n$」。

- 这正是「−5 用 `1111 1011` 表示」的理由：`1111 1011` 当无符号读是 251，而 $251 = 256 - 5$，即 $251 \equiv -5 \pmod{256}$。
- 类比时钟：3 点「往回拨 5 小时」和「往前拨 7 小时」结果一样（模 12）。补码就是把「减」换成了「加 $2^n - x$」。

**⑤ 补码的范围为什么是 −128 ~ +127（不对称）**

- 8 位一共 $2^8 = 256$ 个编码。因为 **+0 和 −0 合并成 1 个**，省出来的那个编码 `1000 0000` 被分配给了 **−128**（它没有对应的正数）。
- 所以负数比正数**多一个**。一般公式（$n$ 位）：

$$\text{范围} = -2^{n-1} \sim 2^{n-1} - 1$$

- 常用值要记住：8 位 → **−128 ~ 127**；16 位 → **−32768 ~ 32767**；32 位 → **−2147483648 ~ 2147483647**。
- **注意**：在 C 里直接写 `-2147483648` 有坑——字面量 `2147483648` 会先被当作正数，超出 `int` 范围；保险写法是 `-2147483647 - 1` 或直接用 `<limits.h>` 里的 `INT_MIN`。

**⑥ 两个必考的陷阱**

- **陷阱 1：最小值取反是它自己**。因为 `-(-128)` 超出 8 位有符号的范围（+128 存不下），所以：

```
signed char c = -128;
printf("%d", -c);       /* 仍是 -128（结果是未定义行为） */
```

- **陷阱 2：有符号与无符号混算**。有符号会被「重新解释」为无符号，例如：

```
int a = -1;
printf("%u", (unsigned)a);    /* 输出 4294967295（即 2^32 - 1） */
if (a > 0u) printf("真");     /* 会输出“真”！因为 a 被当成很大的无符号数 */
```

  验证方法：把该负数按补码写出，再当作无符号数读。**同一个二进制串，类型不同，真值就不同**。

**⑦ 字面量后缀 `u`：`0u` 是什么？**

- `0u` = **值为 0 的 `unsigned int` 常量**。它由两部分组成：`0` **是数值，`u` 是类型后缀**。
- 后缀的作用是**指定常量的类型**，不加后缀时编译器按默认规则推断（整数默认 `int`，不够大才升级）。

| 后缀 | 含义 | 类型 | 例子 |
|---|---|---|---|
| 无 | 有符号整数（默认） | `int` | `0`、`100`、`-5` |
| `u` / `U` | **无符号整数** | `unsigned int` | `0u`、`10U`、`0xFFu` |

| 后缀 | 含义 | 类型 | 例子 |
|---|---|---|---|
| l / L | 长整数 | long | 0L、100000L |
| ul / UL | 无符号长整数 | unsigned long | 0ul、100UL |
| ll / LL | 长长整数 | long long | 0ll、10000000000LL |
| f / F | 单精度浮点 | float | 3.14f |
| l / L（浮点） | 长双精度 | long double | 3.14L |

- `0` 和 `0u` 的数值完全相同（都是 0），区别只在类型。类型不同会导致后续运算走「无符号」的规则，这正是陷阱 2 的关键：

```
int a = -1;
if (a > 0)   printf("A");   /* 假：-1 > 0 不成立 —— 有符号比较 */
if (a > 0u)  printf("B");   /* 真：a 被转成无符号的 4294967295 > 0
—— 输出 B */
```

  - 原因是 C 的寻常算术转换：`int` 与 `unsigned int` 混算时，有符号的那个会被转成无符号，于是 `-1` 变成 `4294967295`。
  - 所以 `0u` 常被用来强制触发无符号比较。写代码时若不注意，很容易写出「永远为真」的条件。
- 常见写法与用途：
  - `unsigned int x = 0u;`——显式表明「这是无符号的 0」，比 `unsigned int x = 0;` 更清楚地表达意图。
  - `for (unsigned i = 0u; i < n; i++)`——避免「有符号/无符号比较」的编译警告。
  - `x & 0u`——按位与，用途上等价于清零（写 `0` 也对，加 `u` 只是标明类型）。
- 易错点：`0u - 1` 不是 −1，而是 `4294967295`！因为无符号运算减不下就会「回绕」：

```
printf("%u", 0u - 1);     /* 输出 4294967295，不是 -1 */
for (unsigned i = n - 1; i >= 0; i--)   /* 死循环！i >= 0 永远成立
*/
```

  千万不要用无符号变量做「递减到 0 结束」的循环。
- 记忆要点：`0u` = 无符号的零；后缀 `u` 只改类型不改值；一旦出现无符号，整个表达式就按无符号规则算。

⑧ 一句话总结

同一个 $n$ 位二进制串，按无符号读是 $0 \sim 2^n - 1$；按有符号补码读是 $-2^{n-1} \sim 2^{n-1} - 1$。最高位既是符号位、又参与计算；正数照抄，负数取反加一。

- 整型常量前缀：`0` 开头是八进制，`0x`/`0X` 开头是十六进制，无前缀是十进制；后缀 `u`（无符号）、`l`/`ll`（长）。

- **浮点常量**默认是 `double`，加 `f` 才是 `float`。默认 `printf("%f", 3.14)` 正确；若 `float` 用 `%f` 则在变参传递中被提升为 `double`。
- **字符常量**用单引号，如 `'A'`；**字符串常量**用双引号，如 `"A"`。`'A'` 是 65（1 字节），`"A"` 是 `{'A','\0'}`（2 字节）。
- 初始化与赋值不同：`int a = 3.7;` 是**截断**（得 3），不是四舍五入。
- `const` 定义只读变量，必须在定义时初始化，之后不可修改。

2. 理解位、字节、存储空间和地址的基本含义；掌握常用整数进制的表示与简单转换、字符与字符编码的关系、常用转义字符和 `sizeof` 运算。

**解析：**

- **位（bit）** 是最小单位，只能取 0/1；**字节（byte）= 8 位**；**地址**是内存单元的编号，按字节编址。
- 存储空间换算：$1\,B = 8\,b$，$1\,KB = 1024\,B$，$1\,MB = 1024\,KB$，$1\,GB = 1024\,MB$。
- 进制转换：
  - 二 → 十：按权展开，如 $1011_2 = 8 + 0 + 2 + 1 = 11$。
  - 十 → 二：除 2 取余、倒序排列。
  - 二 ↔ 八：每 3 位一组；二 ↔ 十六：每 4 位一组。
- **ASCII 编码**（要记）：`'0'` = 48，`'A'` = 65，`'a'` = 97；大写与小写相差 32；数字字符连续。
- **ASCII 表（完整 0~127，十进制 / 十六进制 / 字符）**

**① 控制字符 0~31 与 127（不可打印）**

| 名称 | 缩写 | 十进制 | 十六进制 | 转义写法 | 含义 |
|---|---|---|---|---|---|
| NUL | `\0` | 0 | 0x00 | `\0` | **空字符（字符串结束标志）** |
| SOH | | 1 | 0x01 | | 标题开始 |
| STX | | 2 | 0x02 | | 正文开始 |
| ETX | | 3 | 0x03 | | 正文结束 |
| EOT | | 4 | 0x04 | | 传输结束 |
| ENQ | | 5 | 0x05 | | 询问 |
| ACK | | 6 | 0x06 | | 确认 |
| BEL | `\a` | 7 | 0x07 | `\a` | **响铃** |
| BS | `\b` | 8 | 0x08 | `\b` | **退格** |
| HT | `\t` | 9 | 0x09 | `\t` | **水平制表（Tab）** |
| LF | `\n` | 10 | 0x0A | `\n` | **换行（Line Feed）** |
| VT | `\v` | 11 | 0x0B | `\v` | 垂直制表 |
| FF | `\f` | 12 | 0x0C | `\f` | 换页 |
| CR | `\r` | 13 | 0x0D | `\r` | **回车（Carriage Return）** |

| 名称 | 缩写 | 十进制 | 十六进制 | 转义写法 | 含义 |
|---|---|---|---|---|---|
| SO | | 14 | 0x0E | | 移出 |
| SI | | 15 | 0x0F | | 移入 |
| DLE | | 16 | 0x10 | | 数据链路转义 |
| DC1 | | 17 | 0x11 | | 设备控制 1 |
| DC2 | | 18 | 0x12 | | 设备控制 2 |
| DC3 | | 19 | 0x13 | | 设备控制 3 |
| DC4 | | 20 | 0x14 | | 设备控制 4 |
| NAK | | 21 | 0x15 | | 否定确认 |
| SYN | | 22 | 0x16 | | 同步空闲 |
| ETB | | 23 | 0x17 | | 传输块结束 |
| CAN | | 24 | 0x18 | | 取消 |
| EM | | 25 | 0x19 | | 介质结束 |
| SUB | | 26 | 0x1A | | 替换 |
| ESC | \e | 27 | 0x1B | | 转义 |
| FS | | 28 | 0x1C | | 文件分隔符 |
| GS | | 29 | 0x1D | | 组分隔符 |
| RS | | 30 | 0x1E | | 记录分隔符 |
| US | | 31 | 0x1F | | 单元分隔符 |
| DEL | | 127 | 0x7F | | 删除 |

**② 可打印字符 32~126（考试重点在这段）**

| 字符 | 十进制 | 十六进制 | 字符 | 十进制 | 十六进制 | 字符 | 十进制 | 十六进制 |
|---|---|---|---|---|---|---|---|---|
| （空格） | 32 | 0x20 | ! | 33 | 0x21 | " | 34 | 0x22 |
| # | 35 | 0x23 | $ | 36 | 0x24 | % | 37 | 0x25 |
| & | 38 | 0x26 | ' | 39 | 0x27 | ( | 40 | 0x28 |
| ) | 41 | 0x29 | * | 42 | 0x2A | + | 43 | 0x2B |
| , | 44 | 0x2C | - | 45 | 0x2D | . | 46 | 0x2E |
| / | 47 | 0x2F | **0** | **48** | 0x30 | **1** | **49** | 0x31 |
| 2 | 50 | 0x32 | 3 | 51 | 0x33 | 4 | 52 | 0x34 |
| 5 | 53 | 0x35 | 6 | 54 | 0x36 | 7 | 55 | 0x37 |

| 字符 | 十进制 | 十六进制 | 字符 | 十进制 | 十六进制 | 字符 | 十进制 | 十六进制 |
|---|---|---|---|---|---|---|---|---|
| 8 | 56 | 0x38 | 9 | 57 | 0x39 | : | 58 | 0x3A |
| ; | 59 | 0x3B | < | 60 | 0x3C | = | 61 | 0x3D |
| > | 62 | 0x3E | ? | 63 | 0x3F | @ | 64 | 0x40 |
| A | **65** | 0x41 | B | 66 | 0x42 | C | 67 | 0x43 |
| D | 68 | 0x44 | E | 69 | 0x45 | F | 70 | 0x46 |
| G | 71 | 0x47 | H | 72 | 0x48 | I | 73 | 0x49 |
| J | 74 | 0x4A | K | 75 | 0x4B | L | 76 | 0x4C |
| M | 77 | 0x4D | N | 78 | 0x4E | O | 79 | 0x4F |
| P | 80 | 0x50 | Q | 81 | 0x51 | R | 82 | 0x52 |
| S | 83 | 0x53 | T | 84 | 0x54 | U | 85 | 0x55 |
| V | 86 | 0x56 | W | 87 | 0x57 | X | 88 | 0x58 |
| Y | 89 | 0x59 | Z | 90 | 0x5A | [ | 91 | 0x5B |
| \ | 92 | 0x5C | ] | 93 | 0x5D | ^ | 94 | 0x5E |
| _ | 95 | 0x5F | ` | 96 | 0x60 | a | **97** | 0x61 |
| b | 98 | 0x62 | c | 99 | 0x63 | d | 100 | 0x64 |
| e | 101 | 0x65 | f | 102 | 0x66 | g | 103 | 0x67 |
| h | 104 | 0x68 | i | 105 | 0x69 | j | 106 | 0x6A |
| k | 107 | 0x6B | l | 108 | 0x6C | m | 109 | 0x6D |
| n | 110 | 0x6E | o | 111 | 0x6F | p | 112 | 0x70 |
| q | 113 | 0x71 | r | 114 | 0x72 | s | 115 | 0x73 |
| t | 116 | 0x74 | u | 117 | 0x75 | v | 118 | 0x76 |
| w | 119 | 0x77 | x | 120 | 0x78 | y | 121 | 0x79 |
| z | 122 | 0x7A | { | 123 | 0x7B | ` | ` | 124 |
| } | 125 | 0x7D | ~ | 126 | 0x7E | | | |

**③ 必背的 6 个关键值 + 规律**

| 字符 | 十进制 | 十六进制 |
|---|---|---|
| '\0' | 0 | 0x00 |
| '\n' | 10 | 0x0A |
| ' '（空格） | 32 | 0x20 |

| 字符 | 十进制 | 十六进制 |
| --- | --- | --- |
| `'0'` | **48** | 0x30 |
| `'A'` | **65** | 0x41 |
| `'a'` | **97** | 0x61 |

- **规律 1**：数字字符 `'0'`~`'9'` **连续**，从 48 开始。想求数字字符的值：`ch - '0'`；想由数值得到字符：`n + '0'`。
- **规律 2**：大写字母 `'A'`~`'Z'` **连续**（65~90），小写 `'a'`~`'z'` **连续**（97~122），**大小写相差 32**。
  - 大写转小写：`ch + 32` 或 `ch + ('a' - 'A')`；小写转大写：`ch - 32`。
  - 也可用位运算：`ch | 0x20` 变小写，`ch & ~0x20` 变大写（因为 32 恰好是第 5 位）。
- **规律 3**：`'A'`(65) 与 `'a'`(97) 之间的 91~96 是非字母字符 `[ \ ] ^ _ \``，所以**不能**用 ch >= 'A' && ch <= 'a'` 判字母。
- **常用判断写法**（推荐用标准库 `<ctype.h>`）：

```
/* 手写 */
if (ch >= '0' && ch <= '9')          /* 数字 */
if (ch >= 'A' && ch <= 'Z')          /* 大写 */
if (ch >= 'a' && ch <= 'z')          /* 小写 */
if ((ch >= 'A' && ch <= 'Z') || (ch >= 'a' && ch <= 'z'))   /* 字母 */
/* 库函数（更省事） */
isdigit(ch);  isupper(ch);  islower(ch);  isalpha(ch);
isalnum(ch);
tolower(ch);  toupper(ch);   /* 返回转换后的字符，不修改原值 */
```

- **一句话记忆：空格 32，数字从 48 起连续，大写从 65 起，小写从 97 起，大小写差 32，`'\0'` 是 0。**
- **注意**：ASCII 只有 0~127；128~255 是「扩展 ASCII」，不同系统含义不同，**不要依赖**。汉字等多字节字符不属于 ASCII。
- **常用转义字符**：`\n` 换行、`\t` 水平制表、`\r` 回车、`\0` 空字符（字符串结束标志）、`\\` 反斜杠、`\'` 单引号、`\"` 双引号、`\b` 退格、`\a` 响铃。
- `sizeof` **详解（重点）**
  - **语义**：`sizeof` 求的是一个**类型或表达式所占的字节数**，结果是 `size_t`（无符号整数类型）。
  - **它是运算符（operator），不是函数**，所以**不需要 `#include`，也不能写 `sizeof(int)` 之外的函数式调用**（`sizeof` 后的括号仅在操作数是类型名时才必需）。

```
sizeof(int)      /* 必需括号：操作数是类型名 */
sizeof x         /* 可以不加括号：操作数是表达式 */
sizeof(x)        /* 也可以加，习惯写法 */
```

- **两种写法**：`sizeof(类型名)` 和 `sizeof 表达式`。
- **关键特性：在编译期求值**（除变长数组外），**不求值操作数**——这点极易考：

```
int i = 5;
size_t n = sizeof(i++);   /* i++ 不会执行！i 仍然是 5 */
```

- **常见结果表**（典型 32/64 位环境）：

| 写法 | 结果 | 说明 |
| --- | --- | --- |
| `sizeof(char)` | 1 | **永远是 1**（标准保证） |
| `sizeof(int)` | 4 | 常见，但标准不保证 |
| `sizeof(float)` / `sizeof(double)` | 4 / 8 | — |
| `sizeof(指针)` | 8（64 位）/ 4（32 位） | **与所指类型无关** |
| `sizeof('A')` | **4**（C） | 字符常量在 C 中是 `int`！C++ 里是 1 |
| `sizeof("abc")` | 4 | 数组含 `'\0'` |
| `sizeof(int[10])` | 40 | 整个数组 |
| `sizeof(a)`（`a` 是 `int a[10]`） | 40 | 数组名不退化 |
| `sizeof(p)`（`p` 是 `int *` 指向该数组） | 8 或 4 | **指针退化后的结果** |

- **数组名不退化**：`sizeof(数组名)` 得到整个数组大小；但**数组作函数参数后就是指针**，所以函数内 `sizeof(a)` 得到指针大小。这是经典陷阱：

```
void f(int a[]) { printf("%zu", sizeof(a)); }   /* 输出 8 或 4，不是 40！ */
```

- **最重要的用途**：算 `malloc` 的字节数，使代码与平台无关：

```
int *a = malloc(n * sizeof(int));      /* 推荐 */
int *a = malloc(n * sizeof(*a));       /* 更好：改类型时不用改这里 */
```

    - **输出格式**：`sizeof` 的结果类型是 `size_t`，标准写法用 `%zu`；用 `%d` 在 64 位下可能出错。

  - 易错点汇总：`sizeof('A')` 在 C 中等于 4（字符常量被当作 `int`）；`sizeof("abc")` = 4（含 `'\0'`）；`strlen("abc")` = 3；`sizeof` 不改变、不计算操作数。

3. 掌握算术、赋值、关系、逻辑、条件和逗号表达式，以及自增、自减和复合赋值运算；理解常用运算符的优先级、结合性和逻辑运算的短路规则。

**解析：**

- **优先级从高到低**（同级按"结合方向"结合）：

| 级别 | 运算符 | 名称 | 结合方向 |
|---|---|---|---|
| 1（最高） | `() [] -> .` | 括号、下标、成员 | 左 → 右 |
| 2 | `! ~ ++ -- + - * & (type) sizeof` | 单目（含强转、取址、解引用） | **右 → 左** |
| 3 | `* / %` | 乘、除、求余 | 左 → 右 |
| 4 | `+ -` | 加、减 | 左 → 右 |
| 5 | `<< >>` | 移位 | 左 → 右 |
| 6 | `< <= > >=` | 关系（大小比较） | 左 → 右 |
| 7 | `== !=` | 相等（判等） | 左 → 右 |
| 8 | `&` | 按位与 | 左 → 右 |
| 9 | `^` | 按位异或 | 左 → 右 |
| 10 | `\|` | 按位或 | 左 → 右 |
| 11 | `&&` | 逻辑与 | 左 → 右 |
| 12 | `\|\|` | 逻辑或 | 左 → 右 |
| 13 | `?:` | 条件（三目） | **右 → 左** |
| 14 | `= += -= *= /= %= &= ^= \|= <<= >>=` | 赋值、复合赋值 | **右 → 左** |
| 15（最低） | `,` | 逗号 | 左 → 右 |

  - **速记要点**：
    - 只有 **3 个级别是右结合**：单目（2）、条件（13）、赋值（14）。其余都是左结合。
    - **算术 > 移位 > 关系 > 相等 > 按位 > 逻辑 > 条件 > 赋值 > 逗号**（按位运算比关系运算**低**，这点最反直觉）。
    - **单目高于双目**：`-a * b` = `(-a) * b`；`*p++` = `*(p++)`（先取值再让指针后移）。
    - **赋值最靠后**：`a = b + c` 先算右边整体再赋。
    - **逗号最低**：`a = 1, 2` 是 `(a = 1), 2`，`a` 得到 1。
  - **经典例题**：

```
int a = 1, b = 2, c = 3;
a + b * c        /* 7 : 先 * 后 + */
a<b == c>b       /* 1 : 关系高于相等，等价于 (a<b) == (c>b)，即 1 == 1 */
a & b == 1       /* 0 : 相等高于按位与！等价于 a & (b==1)。因 b==2 所以 b==1 为 0，结果 0 */
a = b = c        /* a 和 b 都变成 3：赋值右结合 */
```

    - **重点体会第二个和第三个**：很多人会误以为 `a & b == 1` 是 `(a & b) == 1`（得 1），实际是 `a & (b == 1)`（得 0）。**这就是按位运算优先级低于相等运算的坑。**
- 记不住时**直接加括号**，最保险。
- **注意点：`*p++` 的语义（极易写错）**
    - **结论**：`*p++` 等价于 `*(p++)`，即**先取 `*p` 的值，再让 `p` 自增**（不是 `(*p)++`）。
    - **为什么**：后缀 `++` 与一元 `*` **优先级相同且都是右结合**，右结合意味着"从右往左结合"，所以 `*p++` 从右边开始读：先看到 `p++`（后缀自增的语义是"先用旧值、后自增"），再对**旧值**解引用。
    - **两个易混写法对比**：

| 表达式 | 等价于 | 含义 | `p` 是否变化 | `*p` 指向的值是否变化 |
|---|---|---|---|---|
| `*p++` | `*(p++)` | 取出 `*p`，**然后指针前移** | **是**（指向下一元素） | 否 |
| `(*p)++` | — | 把所指的**值**加 1 | 否 | **是** |
| `*++p` | `*(++p)` | 指针**先前移**，再取新位置的值 | **是** | 否 |
| `++*p` | `++(*p)` | 把所指的**值**加 1 | 否 | **是** |

    - **最常见的用途——遍历数组/字符串**：

```
char *p = s;
while (*p) putchar(*p++);   /* 逐个输出字符，p 每次前移一格 */
/* 等价于 */
while (*p) { putchar(*p); p++; }
```

      **注意**：这里必须像上面那样**先 `char *p = s;` 复制一份游标**，只移动 `p`。若直接写 `while (*s) putchar(*s++);` 就会**把指向串头的 `s` 推到底**，之后 `strlen(s)`、`strcpy(dst, s)` 等库函数全部失效（详见第四章「遍历字符串」）。
    - **记忆口诀**："**先看 `++` 在后还是在前——后置先用值、前置先动手；`*` 只是对最终结果解引用**"。写不清楚时**拆成两行**最安全。

- **短路规则**：`A && B` 中若 A 为假，**B 不求值**，整个式为假；`A || B` 中若 A 为真，**B 不求值**，整个式为真。
- 短路有副作用：`if (p != NULL && *p > 0)` 是安全的；若写成 `*p > 0 && p != NULL` 则可能崩溃。
- **自增自减**：前缀 `++i` 先加后用，后缀 `i++` 先用后加。作为独立语句时二者等价。
- 易错：`i = 1; printf("%d %d", i++, i++);` 属于未定义行为（参数求值顺序未规定），**考试不考**。
- **复合赋值**：`a += b` 等价于 `a = a + b`，但左值只求一次，且自带类型转换。
- **条件表达式**：`cond ? x : y`，是唯一的三目运算符，要求 `cond` 为数值即可（非 0 为真）。
- **逗号表达式**：从左到右依次求值，整个表达式的值是**最后一个**的值，优先级最低。

4. 掌握整数除法、求余及常见类型转换；能判断表达式的类型与结果。理解按位与、按位或、按位异或、按位取反和移位运算，能按给定整数类型及位宽分析简单示例。

**解析**：

- **整数除法向零截断**：`7/2 = 3`，`-7/2 = -3`。只要有一个操作数是浮点，就做浮点除法：`7.0/2 = 3.5`。
- **求余 `%`**：结果符号**与被除数相同**：`7%3 = 1`，`-7%3 = -1`，`7%-3 = 1`。`%` **只能用于整型**（浮点用 `fmod`）。
- **负数取余行为（易错，必掌握）**
  - **核心规则**：在 C 中，`a % b` 的结果**符号与被除数 `a` 相同**（不是与被除数或除数中绝对值大的那个）。因为 C99 规定整数除法**向零截断**，而取余必须满足下面的恒等式，所以符号只能跟着被除数。
  - **唯一依据的恒等式**：

$$(a/b) \times b + (a\,\%\,b) = a$$

    也就是说：**先算商（向零截断），再用除法倒推出余数**。只要记住“除法向零截断”，余数符号就自然确定了。
  - **四种符号组合的完整对照表**（每种都列出商的截断过程和验证）：

| 表达式 | 商 `a/b` | 推导（商×除数） | 余数 `a%b` | 与 `%` 符号相同的是 | 验证 |
|---|---|---|---|---|---|
| `7 % 3` | `2` | `2 × 3 = 6` | **1** | 被除数 +、除数 + → **正** | `6 + 1 = 7` ✓ |
| `-7 % 3` | `-2`（向零） | `-2 × 3 = -6` | **-1** | 被除数 − → **负** | `-6 + (-1) = -7` ✓ |
| `7 % -3` | `-2`（向零） | `-2 × -3 = 6` | **1** | 被除数 + → **正** | `6 + 1 = 7` ✓ |
| `-7 % -3` | `2` | `2 × -3 = -6` | **-1** | 被除数 − → **负** | `-6 + (-1) = -7` ✓ |

- **一句话记忆：余数的符号只看被除数，与除数无关。**
  - `-7 % 3` → **−1**（不是 2！）
  - `7 % -3` → **1**（不是 −1！）
  - 这两个是考试最容易写错的。
- **为什么不是"数学上的余数"**：数学中常把余数规定为非负（如 Python 里 `-7 % 3` 得 **2**）。但 **C 的余数可以是非负数也可以是负数**，与 Python、数学定理都不同，不能混用。
- **常见错误及其后果**：

| 错误想法 | 例子 | 实际结果 | 危害 |
| --- | --- | --- | --- |
| 认为余数总非负 | `-7 % 3` 当成 `2` | **−1** | 数组下标出现负数，越界 |
| 认为符号跟除数 | `7 % -3` 当成 `-1` | **1** | 结果完全反了 |
| 认为 `a%b` 能判奇偶（任意 a） | `if (n % 2 == 1)` | 负数时为假！ | 负奇数被误判为偶数 |

- **两个实用技巧**：
  1. **让余数总非负**（把负数结果"扳正"）：

```c
int r = a % b;
if (r < 0) r += b;          /* 余数变为 0 ~ |b|-1（b ≠ 0，且 b 为正时） */
```

     这个写法在做**循环数组**（环形缓冲、哈希）时非常常用。
  2. **判负数奇偶要小心**：因为 `-5 % 2 == -1`，所以**不能**写 `n % 2 == 1`，应写成：

```c
if (n % 2 != 0)  /* 对：正负都正确 */
if (n % 2 == 1 || n % 2 == -1)  /* 也行，但啰嗦 */
if (n & 1)       /* 对位运算更直接 */
```

- **其他相关注意点**：
  - `b` **不能为 0**（除零在 C 中是未定义行为，通常直接崩溃）。
  - `INT_MIN % -1` 是特例：数学上是 0，但在 32 位补码下 `-(-2147483648)` 溢出，属**未定义行为**，考试不考。
  - `-7 / 2` **与** `-7 >> 1` **不同**：除法向零得 `-3`，算术右移向下取整得 `-4`。余数同理，只能用 `/`、`%` 这一对，不要用移位和掩码代替。
  - **浮点求余**：`%` 不能用于 `double`/`float`，要用 `<math.h>` 的 `fmod`；但 `fmod(-7.0, 3.0)` 同样是 **−1.0**，规则与整数一致。

- 整除与取余的关系：`(a/b)*b + a%b == a`（b ≠ 0）。
- **类型转换**：自动转换按"**由窄到宽**"提升：`char/short` → `int` → `unsigned int` → `long` → `unsigned long` → `float` → `double`。
- 有符号与无符号混算时，**有符号会转成无符号**：`-1 > 0u` 的结果是**真**（因为 −1 被当成一个很大的无符号数）。这是经典陷阱。
- **强制转换** `(int)3.9` 是**截断**得 3，不是四舍五入；四舍五入用 `(int)(x + 0.5)`。
- **按位运算（位操作行为详解）**
  - **共同前提**：按位运算的操作数**必须是整型**（`char`/`short`/`int`/`long` 及其无符号版），**不能是浮点**。运算时先做**整型提升**（短于 `int` 的先转成 `int`），再**逐位对齐**运算。
  - **四种按位运算的真值表**（以 8 位为例，`a = 0b1100`，`b = 0b1010`）：

| 运算 | 规则（逐位） | `a op b` | 结果 | 记忆 |
|---|---|---|---|---|
| `a & b` | **两个都是 1 才得 1** | `1100 & 1010` | `1000` | 有 0 则 0 |
| `a \| b` | **有一个是 1 就得 1** | `1100 \| 1010` | `1110` | 有 1 则 1 |
| `a ^ b` | **两个不同才得 1** | `1100 ^ 1010` | `0110` | 相同则 0 |
| `~a` | **逐位取反**（0↔1） | `~1100` | `0011` | 全翻过来 |

  - **三种典型用途（必记）**：
    - **取位 / 清零**：`x & 1` 判奇偶（末位为 1 是奇数）；`x & 0x0F` 取低 4 位；`x & ~(1<<k)` 把第 k 位清 0。
    - **置位**：`x | (1<<k)` 把第 k 位置 1（其余位不变）。
    - **翻转位**：`x ^ (1<<k)` 把第 k 位取反；`x ^ 1` 在奇偶之间切换。
  - **异或的三条性质**：`x ^ x = 0`；`x ^ 0 = x`；**满足交换律与结合律**。因此 `x ^ y ^ y == x`，这正是"**异或交换两数**"的原理：

```
x ^= y;  y ^= x;  x ^= y;     /* 无需临时变量即可交换 x 与 y */
```

**具体例子（以 `x = 7`、`y = 9` 为例，4 位演示）**：

| 步骤 | 语句 | `x` 的值 | `y` 的值 | 说明 |
|---|---|---|---|---|
| 初始 | — | `7` = `0111` | `9` = `1001` | 待交换 |
| ① | `x ^= y` | `14` = `1110` | `9` = `1001` | `0111 ^ 1001 = 1110`，x 变成两者的"差异" |
| ② | `y ^= x` | `14` = `1110` | `7` = `0111` | `1001 ^ 1110 = 0111`，y 已得到**原来的 x** |

| 步骤 | 语句 | x 的值 | y 的值 | 说明 |
|---|---|---|---|---|
| ③ | x ^= y | 9 = 1001 | 7 = 0111 | 1110 ^ 0111 = 1001，x 得到**原来的 y** |

逐位看第 ① 步：

```
  0 1 1 1     (x = 7)
^ 1 0 0 1     (y = 9)
─────────
  1 1 1 0     (= 14，这是 x 与 y 所有"不同的位")
```

第 ② 步：用"差异"去异或原来的 y，不同的位被翻回去，就还原出了 x：

```
  1 0 0 1     (y = 9)
^ 1 1 1 0     (差异)
─────────
  0 1 1 1     (= 7，即原来的 x) ✓
```

第 ③ 步：再用"差异"去异或新的 y（= 原来的 x），就得到原来的 y：

```
  1 1 1 0     (差异)
^ 0 1 1 1     (y = 7)
─────────
  1 0 0 1     (= 9，即原来的 y) ✓
```

- **一句话理解**：第 ① 步把 x 变成"x 与 y 的差异表"，之后两次异或就是**查表还原**。 注意：若 x 与 y 是**同一变量**（swap(&a, &a)），会把值变成 0，这是该技巧的坑。

- **取反 ~ 与补码的关系**：因为 $-x$ 的补码 = ~x + 1，所以

$$\sim x \;=\; -x - 1$$

验证：~0 = -1，~5 = -6，~(-1) = 0。

- **移位运算的行为（最易考）**
  - **语义**：a << n 把各位**左移** n 位；a >> n **右移** n 位。n 必须是**非负整数**。
  - **左移 <<：低位空出的位置一律补 0**。数学上等价于乘以 $2^n$（在不溢出的前提下）：3 << 2 = 12，即 $3 \times 4$。
  - **右移 >>：分两种情况**（这是必考点）：

| 左操作数类型 | 右移规则 | 名称 | 例子 |
|---|---|---|---|
| **无符号** | 高位一律**补 0** | 逻辑右移 | 248 >> 1 = 124 |

| 左操作数类型 | 右移规则 | 名称 | 例子 |
| --- | --- | --- | --- |
| **有符号、非负** | 高位补 0（补符号位也是 0） | 算术右移 | `8 >> 1 = 4` |
| **有符号、负数** | 高位**补符号位（补 1）** | 算术右移 | `(-8) >> 1 = -4` |

- **负数右移 ≈ 除以 $2^n$ 并向下取整**，这与 C 的**除法向零截断**不同，二者要区分：

```
-7 >> 1   /* -4  —向下取整 */
-7 / 2    /* -3  —向零截断 */
```

所以**不要用右移代替负数除法**，两者在不能整除时结果不同。

- **溢出与越界移位（易错）**
  - **左移的溢出**：被移出的高位直接**丢弃**，不是报错：`1 << 31` 结果为 `-2147483648`（`int` 为 32 位时，符号位被置 1）。
  - **移位位数越界是未定义行为**：移位数大于等于操作数的位宽时（如 32 位 `int` 写 `x << 32`），结果是**未定义**，考试不考。
  - **`1 << 31` 是未定义行为**：字面量 `1` 是 `int`，$2^{31}$ 超出 `int` 范围。要得到 `0x80000000` 应写 `1u << 31`（无符号）或 `(int)(1u << 31)`。
  - **有符号左移溢出**同样是未定义行为：`INT_MAX + 1` 这类溢出不要依赖具体结果。
- **易错点汇总**：
  - `&& ||` 是**逻辑**运算（结果为 0/1，有短路）；`& | ^ ~` 是**按位**运算（逐位处理，**无短路**）。
  - **优先级**：按位运算**低于相等运算**，所以 `x & 1 == 0` 实际是 `x & (1 == 0)`，**必须写 `(x & 1) == 0`**。
  - 移位是**按位**操作，不是"乘除的语法糖"：溢出、符号、取整方向都可能不同。
- **自测示例**（可在机器上验证）：

```
unsigned char u = 0xF8;     /* 248 = 1111 1000 */
printf("%u", u >> 1);       /* 124 —逻辑右移，补 0 */
signed char  c = -8;        /*      1111 1000 */
printf("%d", c >> 1);       /* -4  —算术右移，补 1 */
printf("%d %d", ~0, ~5);    /* -1 -6（即 -x-1） */
printf("%d", 1 << 31);      /* -2147483648 —高位溢出被丢弃 */
```

> **提示**：这部分注重学习和实际编程中容易忽略的基本规则与使用条件，不以生僻用法或刻意构造的复杂表达式设置难点。不要求推测含有未定义行为的程序会输出什么结果。

## 三、数据输入与输出

1. 掌握 printf、scanf、getchar 和 putchar 的基本用法；能使格式说明与参数类型正确对应，按要求完成数值、字符和字符串的输入输出。

> **解析：**
>
> - **printf 格式串**：%[标志][宽度][.精度][长度]转换字符。

| 转换字符 | 含义 | 例子 | 输出 |
|---|---|---|---|
| %d / %i | 有符号十进制整数 | printf("%d", -5) | -5 |
| %u | 无符号十进制 | printf("%u", -1) | 4294967295 |
| %o / %x / %X | 八 / 十六进制 | printf("%x", 255) | ff |
| %f | double（默认 6 位小数） | printf("%f", 3.14159) | 3.141590 |
| %.2f | 保留 2 位小数 | printf("%.2f", 3.14159) | 3.14 |
| %e | 科学计数法 | printf("%e", 314.0) | 3.140000e+02 |
| %c | 单个字符 | printf("%c", 65) | A |
| %s | 字符串 | printf("%s", "ab") | ab |
| %p | 指针地址 | — | 00000000 |
| %% | 输出一个 % | — | % |

> - **宽度和精度**：printf("%5d", 42) 得 42（右对齐，默认）；%-5d 左对齐；%05d 补前导零得 00042。
> - **scanf 格式串**：%d 读 int*，%f 读 float*，%lf 读 double*，%c 读 char*，%s 读 char*（遇空白停，**无法读入含空格的字符串**）。
> - **%f 与 %lf：为什么输出不用写 lf？（必考区别）**
>   - **根本原因：变参函数的默认实参提升**。printf 是变参函数（... 部分），C 规定传入变参的 float 会**自动提升为 double**——也就是说 printf 那边**根本收不到 float**，所有浮点都是 double。
>   - 因此 printf 只需一个 %f 就够（它天然就是收 double）；而 scanf 是**靠指针类型反推数据大小**的，float* 和 double* 必须区分，所以**必须写 %lf 才能读 double**。

| 场景 | 用 %f | 用 %lf | 结论 |
|---|---|---|---|
| printf 输出 double | ✓ 推荐 | ✓ 也可以（C99 起） | 二者等价 |
| printf 输出 float | ✓ 推荐（会自动提升） | ✓ 也可以 | 二者等价 |
| **scanf 读 double** | **✗ 错！** | **✓ 必须用** | **只能用 %lf** |
| **scanf 读 float** | **✓ 必须用** | **✗ 错！** | **只能用 %f** |

> - **为什么会有「没人用 %lf」的感觉**：在 C99 之前，printf 用 %lf 是**未定义行为**，所以那时确实不能写；C99 之后才明确允许。但既然 %f 一样能工作，大家就习惯只写 %f。
> - **一句话结论：输出（printf）用 %f 就行，写 %lf 也不算错；输入（scanf）必须分清——double 用 %lf，float 用 %f。**

```
double d = 3.14;
float  f = 3.14f;
```

```
scanf("%lf", &d);      /* 对：读 double */
scanf("%f",  &f);      /* 对：读 float  */
scanf("%f",  &d);      /* 错：类型不匹配，结果错 */
printf("%f", d);       /* 对：输出 double（推荐写法） */
printf("%lf", d);      /* 也对（C99 起），但不必要 */
```

- **其他长度修饰符（scanf / printf 都适用）：**

| 修饰符 | 配合 | 含义 |
|---|---|---|
| h | %hd / %hu | short |
| hh | %hhd / %hhu | signed char / unsigned char |
| l | %ld / %lu | long |
| ll | %lld / %llu | long long |
| l（浮点） | %lf / %le | double（**仅 scanf 必须**） |
| L | %Lf | long double |

- **易错清单**：printf 输出 long long 必须写 %lld（写 %d 会出错）；输出 long 必须写 %ld。这些长度修饰符**在 printf 里也不能省**。
- **scanf 特别注意**：%s 读入时**自动在末尾补 '\0'**，所以数组要留一位；%c **会读入空白字符**（空格、回车也算），若想跳过空白需写成 " %c"（前面加空格）。
- scanf 返回值是**成功读入的项数**，可用于判题：if (scanf("%d", &n) != 1) { /* 处理错误 */ }。
- getchar() 从 stdin 读**一个字符**并返回其 int 值（EOF 为 −1，所以必须用 int 接收，不能用 char）；putchar(c) 向 stdout 输出一个字符。
- getchar / putchar 是**宏**（在 <stdio.h> 中定义），比 scanf/printf 快且不会跳过空白。

2. 理解 scanf 中地址参数的含义，以及空白字符对常用输入格式的影响；能分析基本的输出宽度、精度和换行格式，识别常见输入输出错误。

**解析：**

- **为什么要取地址 &**：scanf 需要把结果写回调用者的变量，C 又是按值传递，所以只能传变量地址。忘了 & 通常不报错但运行时崩溃。
- **例外**：数组名本身就是地址（scanf("%s", str) 不加 &）；用 %s 读字符数组时已经是指针。
- **常见错误对照：**

| 错误写法 | 后果 | 正确写法 |
|---|---|---|
| scanf("%d", n) | 把 n 当地址，崩溃 | scanf("%d", &n) |
| scanf("%d", n) 但 n 是 char | 类型不匹配 | 用 int n 或 %hhd |
| scanf("%f", &d)（d 是 double） | 结果错 | scanf("%lf", &d) |
| scanf("%d,%d", &a, &b) | 必须输入逗号 | 输入 1,2 |
| printf("%d", 3.14) | 输出垃圾值 | 用 %f |

- **空白字符的处理规则**：`scanf` 格式串中的空格、`\t`、`\n` 意为"**跳过任意多个空白字符**"；而其他字符（如逗号）必须原样输入。
- 大部分转换（`%d %f %s`）会**自动跳过前导空白**；例外是 `%c`（读任何字符包括空白）和 `%[]`。
- **典型坑：混用 `scanf` 和 `getchar`**。`scanf("%d", &n)` 后输入的回车会留在缓冲区，紧跟的 `getchar()` 会读到这个 `\n`。解决：在中间加一句 `getchar();` 或写成 `scanf("%d%*c", &n)`。
- **输入缓冲区**：所有输入先进入缓冲区，`scanf` 从中取。**换行符 `\n` 不会被 `%d` 吃掉**，会留给下一个读字符的函数——这是绝大多数"神秘"行为的根源。
- **输出缓冲区**：`printf` 的输出先进入缓冲区，遇到 `\n` 或缓冲区满或程序正常结束时才真正刷新。调试时可用 `fflush(stdout)`；`stderr` 无缓冲。

**(1) 怎么读取含空格的字符串**

`scanf("%s")` **读不到空格**（遇空白就停），要读含空格的一行只有两条路：

```
/* 推荐：fgets 读整行，再手动去掉末尾 '\n' */
char line[100];
if (fgets(line, sizeof(line), stdin) != NULL) {
    size_t len = strlen(line);
    if (len > 0 && line[len - 1] == '\n') line[len - 1] = '\0';
}
```

或 `scanf(" %99[^\n]", s)`——**必须写宽度**，`%[...]` 不限宽会冲掉内存。

**(2) `getchar()` 的使用技巧与标准代码**

**基本语义**：从 `stdin` 读**一个字符**并返回其 `int` 值（`EOF` 为 −1）。

| 技巧 | 写法 | 作用 |
|---|---|---|
| **必须用 `int` 接** | `int c; c = getchar();` | 用 `char` 接会与 `EOF` 混淆 |
| 吃掉一个字符 | `getchar();` | 丢弃（常用于吃掉回车） |
| **清空本行剩余** | `while ((c = getchar()) != '\n' && c != EOF) { }` | `scanf` 后的标准清场 |
| 逐字符读完全部输入 | `while ((c = getchar()) != EOF) { }` | 通用逐字符处理 |
| 最多读 n−1 个字符 | `while (i < n - 1 && (c = getchar()) != '\n') s[i++] = c; s[i] = '\0';` | 手写限长读行 |
| 读字符并回显 | `while ((c = getchar()) != EOF) putchar(c);` | 逐字符复制输入到输出 |

**标准代码 ①：吃掉 `scanf` 后残留的回车（最常用）**

```c
int n;
scanf("%d", &n);      /* 输入 "5\n" 后，'\n' 仍在缓冲区 */
getchar();            /* 吃掉那个 '\n'，否则下一句 getchar 会读到它 */
char ch = getchar();  /* 现在才真的读到用户想要的字符 */
```

等效写法：`scanf("%d%*c", &n);`（`%*c` 读入一个字符并丢弃）。

**标准代码 ②：清空整行剩余（更稳，不怕残留多个字符）**

```c
int c;
while ((c = getchar()) != '\n' && c != EOF) { }   /* 一直丢到行尾 */
```

> 比单个 `getchar()` 可靠：若输入 `"5 \n"`，一个 `getchar()` 只能吃掉一个空格。

**标准代码 ③：逐字符读完整份输入（不依赖行结构）**

```c
int c;
while ((c = getchar()) != EOF) {
    /* 处理字符 c：统计、分类、累加…… */
}
```

**要点**：`getchar` **不跳过任何字符**（空格、回车、Tab 都会返回给程序）——这既是它的优点（能拿到全部字符），也是坑源（常需配合上面两段清缓冲区代码）。

**易错点**：

- **用 `char c = getchar()` 判断 `EOF` 会出错**：`char` 装不下 −1 时可能变成 255（= `0xFF`），与 `EOF` 比较恒不相等，导致**死循环**。必须用 `int`。
- **`getchar` 是宏，不能传表达式或取地址**：不能写 `getchar(&c)` 或 `x = getchar() + 1`（可能被求值两次）。

**(3) 易错清单（考试高频）**

| 错误 | 后果 | 正确做法 |
|---|---|---|
| `char c = getchar()` 判 `EOF` | 死循环 | 用 `int c` |
| `scanf("%s", s)` 想读含空格的串 | 只能读到第一个词 | 用 `fgets(s, sizeof(s), stdin)` |
| `fgets` 后字符串末尾带着 `\n` | `strcmp` 判等时总是失败 | 手动去掉末尾 `\n` |
| `scanf("%c", &c)` 想读字母 | 读到了上一次留的 `\n` | 写成 `" %c"` |
| `scanf("%d", &n)` 后直接 `getchar()` | 读到 `\n` 而非目标字符 | 先清缓冲区 |

| 错误 | 后果 | 正确做法 |
| --- | --- | --- |
| `scanf("%[^\n]", s)` 不写宽度 | 输入过长时**内存被冲** | 写 `%99[^\n]` |

> **提示**：输入输出不会在没有给出材料的情况下考察缓冲区、读写错误与流状态、部分读写、阻塞、原子性与持久化、终端输入机制与重定向等问题，试题如涉及上述内容，将提供必要的背景知识和规则说明，应能理解题目所给材料，结合已学知识，完成简单的程序分析、补全或编写任务。

## 四、数组与字符串

1. 掌握一维、二维数组的定义、初始化和元素访问；理解下标范围及二维数组按行存放的次序，能完成数组遍历、统计和简单矩阵处理。

> **解析**：
>
> - 声明：`int a[10];` 下标范围是 **0 ~ 9**，**没有** `a[10]`；越界访问编译不报错但行为未定义。
> - **初始化**：`int a[5] = {1,2};` 后三个自动为 0；`int a[] = {1,2,3};` 长度由初始化项数推出（为 3）；`int a[5] = {0};` 全部置 0。
> - **C 语言不支持用变量开数组**（除非变长数组 VLA 或 `malloc`）：`int n = 5; int a[n];` 在 C99+ 合法但不推荐。
> - **二维数组**：`int a[3][4]` 是 3 行 4 列，**按行优先（row-major）存放**：先存 `a[0][0]` 到 `a[0][3]`，再存 `a[1][*]` …。
> - **初始化例子：`int a[2][3] = {1,2,3,4,5,6};`（重点辨析）**
>
>   - **结果**：按行优先**依次填满**，等价于 `{{1,2,3},{4,5,6}}`。
>
>     | 下标 | a[0][0] | a[0][1] | a[0][2] | a[1][0] | a[1][1] | a[1][2] |
>     | --- | --- | --- | --- | --- | --- | --- |
>     | 值 | 1 | 2 | 3 | 4 | 5 | 6 |
>
>   - **它合法吗？合法，结果也对**，但 **GCC 会报 `-Wmissing-braces` 警告**（`missing braces around initializer`）——因为外层括号里本该嵌套两层 `{}`。想要**零警告**就写成 `{{1,2,3},{4,5,6}}`。
>
>   - **为什么能这样写**：C 规定初始化时如果内层花括号被省略，就按**存储顺序**（行优先）连续填充，所以 `1` 进 `a[0][0]`、`2` 进 `a[0][1]` …… `6` 进 `a[1][2]`。
>
>   - **不写满会怎样？会自动补 0**（这点极易考）：
>
>     ```
>     int b[2][3] = {{1,2},{4}};   /* 其余全部自动置 0 */
>     ```
>
>     实际内容：`1 2 0 / 4 0 0`。
>
>   - **平铺视角（验证“按行优先”）**：把二维数组看成一段连续内存，则 `a[i][j]` 就是第 $i \times 3 + j$ 个元素：

```
int *flat = &a[0][0];
/* flat[0..5] = 1 2 3 4 5 6，且 flat[4] 就是 a[1][1] */
```

    - **不要写成**：`int a[2][3] = {1,2,3,4,5,6,7};`——元素给多了是**编译错误**（不是警告）。

- **二维数组必须给出列数**：`int a[][4] = {{1,2,3,4}, {...}};` 合法；`int a[][]` 非法。
- **地址关系**：`a[i][j]` 的地址 = 首地址 $+ (i \times$ 列数 $+ j) \times$ sizeof(元素)。
- 数组名在表达式中**退化为指向首元素的指针**，但 `sizeof(数组名)` 得到的是整个数组的字节数。
- 遍历模板：

```
for (i = 0; i < 行数; i++)
    for (j = 0; j < 列数; j++)
        ... a[i][j] ...
```

2. 掌握字符数组和字符串的表示及输入输出；理解字符、字符串、数组容量和字符串长度的区别，以及字符串结束标志 `'\0'` 的作用。

**解析**：

- **字符串 = 字符数组 + 末尾的 `'\0'`**。`'\0'`（ASCII 0）是字符串结束标志，不是字符 `'0'`（ASCII 48）。
- **四个概念要分清**（以 `char s[10] = "abc";` 为例）：
    - 数组容量 = 10（能装 10 个字符）
    - 字符串长度 = 3（`strlen(s)`，不算 `'\0'`）
    - 实际占用 = 4 字节（3 个字符 + `'\0'`）
    - 剩余有效可用 = 6 字节
- **初始化方式对比**：
    - `char s[] = "abc";` → 长度 4，内容 `a b c \0`。
    - `char s[] = {'a','b','c'};` → 长度 3，**没有 `'\0'`，不是字符串**，`printf("%s")` 会越界。
    - `char s[3] = "abc";` → **错**，放不下 `'\0'`（是合法 C 但是未终止字符串）。
- **输入输出**：`scanf("%s", s)` 遇空白停止且自动补 `'\0'`，**不能读含空格的字符串**；读整行用 `fgets(s, sizeof(s), stdin)`（会保留 `\n`）。`puts(s)` 输出字符串并换行。
- **遍历字符串的惯用写法**：`for (i = 0; s[i] != '\0'; i++)` 或 `for (p = s; *p; p++)`。
- **重要习惯：遍历时不要把指向串头的那个指针移走**
    - **问题**：如果直接移动保存串头的指针（如 `while (*s) s++;`），循环结束后 `s` 已经指向 `'\0'`。此时 `strlen(s)` 得 **0**，`strcpy`/`strcat`/`puts` 都会得到**空串**——串头丢了，再也回不去。
    - **正确做法：另用一个游标指针**（或直接用下标），原指针保持不动：

```c
char buf[] = "hello";

/* ✓ 推荐：游标 p 移动，buf 始终指向串头 */
char *p = buf;
while (*p) { /* 处理 *p */ p++; }
printf("%s", buf);        /* 仍输出 hello */

/* ✗ 反例：直接把 buf 所在的指针一路推到底 */
char *t = buf;
while (*t) t++;              /* t 现在指向 '\0' */
printf("%d", (int)strlen(t));   /* 输出 0！原串"看不见"了 */
```

- **实测对比**（同一份 `"hello"`）：

| 写法 | 循环后长度 | 还能当原串用吗 |
|---|---|---|
| `p = buf` 用副本遍历 | `strlen(buf) = 5` | ✓ 能，`buf` 仍指向串头 |
| 直接 `while (*t) t++;` | `strlen(t) = 0` | ✗ 不能，`t` 已到底 |
| 下标 `buf[i]` 遍历 | `strlen(buf) = 5` | ✓ 能，根本不碰指针 |

- **三条实用建议**：

  1. **优先用下标写法** `for (i = 0; s[i]; i++)`——最安全，完全不会误移串头。
  2. 想用指针就**先做个副本**：`char *p = s;`，然后只动 `p`。
  3. 若确实移动了原指针，**记得恢复**（前提是备份过串头）：`char *head = p; ... p = head;`。

- **补充：数组名本身不能自增**（`char s[10]; s++;` 是编译错误 `lvalue required as increment operand`），所以"移动串头"的错误在数组上会直接报错，但在**指针变量**上会静默生效、更难发现。

3. 掌握 `strlen`、`strcmp`、`strcpy` 和 `strcat` 的基本用法；能进行字符统计、字符串比较、复制、连接和简单变换，并检查数组边界及存储空间是否足够。

**解析：**

| 函数 | 原型 | 功能 | 返回值 |
|---|---|---|---|
| `strlen` | `size_t strlen(const char *s)` | 求长度（不含 `'\0'`） | 长度值 |
| `strcmp` | `int strcmp(const char *a, const char *b)` | 字典序比较 | `<0` / `0` / `>0` |
| `strcpy` | `char *strcpy(char *dst, const char *src)` | 复制（含 `'\0'`） | `dst` |
| `strcat` | `char *strcat(char *dst, const char *src)` | 拼接（追加到 `dst` 末尾） | `dst` |

- **必须** `#include <string.h>`。
- `strcmp` **返回值的含义**：负数表示第一个串小，0 表示相等，正数表示第一个串大。**不要写成** `if (strcmp(a,b) == 1)`，可靠写法是 `if (strcmp(a,b) == 0)`（判等）或 `if (strcmp(a,b) < 0)`。
- 比较是**逐字符比较 ASCII 码**，不是比长度：`"abc" < "abd"`，`"Z" < "a"`（因为 90 < 97）。
- **字符串不能直接赋值、比较、拼接**：`s = "abc";`、`if (s1 == s2)`、`s1 + s2` 都是错的（它们退化为指针比较）。
- **越界是最常见的错**：`strcpy(dst, src)` 不管 `dst` 是否够大；`dst` 至少需要 `strlen(src)+1` 字节。`strcat(dst,src)` 需要 `strlen(dst)+strlen(src)+1` 字节。
- **安全版本**（了解即可）：`strncpy` / `strncat` / `snprintf`，但 `strncpy` 不保证补 `'\0'`，要手动补。

> **提示**：如果题目的考点为数组和字符串，那么是不会结合动态内存分配来考察的；但是如果题目的考点是动态内存分配，那么是有可能会结合数组与字符串来考察的。

## 五、函数

1. 掌握函数定义、声明、调用、形参与实参、返回值和 `void` 的用法；理解 C 语言按值传递参数的机制，能分析调用前后有关变量的变化。

> **解析**：
>
> - **定义 vs 声明**：定义给出完整的函数体（分配存储），一个函数只能定义一次；声明只给出返回类型、函数名、参数类型（**以 `;` 结尾**），可以多次。
> - 典型结构：先声明后使用；若定义在使用之前，可省掉声明。
> - **形参（形式参数）** 是函数内部的局部变量，在调用时才分配内存；**实参（实际参数）** 是调用时传入的表达式，其**类型需可转换为形参类型**。
> - **按值传递（最核心）**：函数内对形参的修改**不影响实参**。例：
>
> ```
> void f(int x) { x = 100; }   /* 交换失败 */
> void g(int *p) { *p = 100; }  /* 通过指针才能改到实参 */
> ```
>
> - `void` 的三种用法：① `void f(void)` 无参无形；② `void f()` 无返回值；③ `void *p` 通用指针。
> - `return`：返回值类型不写默认 `int`（C99 起不推荐）；非 `void` 函数若无 `return` 而使用返回值，行为未定义。
> - `return` 后可接表达式，会**先把结果转换为函数返回类型**再返回（如 `return 3.7;` 遇 `int` 返回类型得 3）。

2. 掌握函数的嵌套调用和数组作为函数参数的基本用法；理解递归的终止条件及调用过程，能编写简单递归函数。

> **解析**：
>
> - **嵌套调用**：C 允许函数互相调用，但**不允许函数定义嵌套**（不能在函数体内再定义函数）。
> - **数组作为参数**：数组名作为实参传入时退化为指针，形参写 `int a[]` 与 `int *a` **完全等价**，编译器不会知道数组长度，所以**必须另传长度**：

```
void f(int a[], int n);      /* 等价于 int *a */
```

- 后果：函数内用 `sizeof(a)` 得到的是**指针的大小**（8 或 4 字节），**不是数组总大小**。这是经典陷阱。
- 正因为传的是地址，函数内修改 `a[i]` 会**真正改变**调用者的数组——这是"数组能带出结果"的原因。
- **二维数组作为参数**必须指定列数：`void f(int a[][4], int rows)`。
- **递归三要素**：① 有**终止条件（基准情形）**；② 每次递归**问题规模变小**；③ 递归调用向终止条件靠近。
- 递归执行过程本质是**函数调用栈**：每次调用压入一个新的栈帧（保存参数、局部变量、返回地址），逐层返回时弹出。
- 典型递归：

```
long fact(int n) { return n <= 1 ? 1 : n * fact(n - 1); }
int fib(int n)  { return n <= 1 ? n : fib(n-1) + fib(n-2); }
int gcd(int a, int b) { return b == 0 ? a : gcd(b, a % b); }
```

- 优缺点：代码简洁但**时间/空间开销大**（如 `fib` 朴素递归是指数级），且层次太深会**栈溢出**。

3. 掌握局部变量与全局变量的作用域；理解变量的存储期、静态局部变量的特点，以及声明与定义的区别。

**解析：**

- **作用域（空间）vs 存储期（时间）** 是两个维度，要分开看。

| 变量类别 | 定义位置 | 作用域 | 存储期 | 初值 |
|---|---|---|---|---|
| 自动局部 | 函数内 | 本函数块内 | 自动（进块生、出块死） | 不确定 |
| 静态局部 | 函数内加 `static` | 本函数块内 | 整个程序运行期 | 自动为 0 |
| 全局 | 函数外 | 从定义处到文件末（可用 `extern` 跨文件） | 整个程序运行期 | 自动为 0 |
| 静态全局 | 函数外加 `static` | 仅本文件 | 整个程序运行期 | 自动为 0 |

- **静态局部变量的关键特点**：① 只在**第一次进入函数时初始化一次**；② 函数返回后值**依然保留**；③ 作用域仍限于本函数，外部不可见。
- **完整代码示例（对比静态局部 vs 自动局部 + 局部遮蔽全局）：**

```c
#include <stdio.h>

int g_init = 10;       /* .data：全局已初始化 */
int g_zero;            /* .bss ：全局未初始化，自动为 0 */
static int sg = 20;    /* .data：静态全局（仅本文件可见） */

void counter(void) {
    static int cnt = 0;    /* .data：只在第一次进入时初始化一次 */
    int local = 100;       /* 栈：每次调用都重新创建 */
    cnt++;                 /* 累加并保留 */
    local++;               /* 加完就丢 */
    printf("  cnt=%d local=%d\n", cnt, local);
}

int shadow(void) {
    int g_init = 999;      /* 局部变量遮蔽同名全局变量 */
    return g_init;
}

int main(void) {
    counter(); counter(); counter();
    printf("函数内=%d, 全局=%d\n", shadow(), g_init);
    return 0;
}
```

**实际输出**（可直接对照）：

```
  cnt=1 local=101
  cnt=2 local=101      ← cnt 保留了，local 每次都是 101
  cnt=3 local=101
函数内=999, 全局=10     ← 遮蔽只影响函数内，全局没变
```

- **各存储区域对照表（把上面的变量对号入座）：**

| 存储区 | 存放内容 | 生存期 | 初值 | 代码里的例子 |
|---|---|---|---|---|
| `.text` **代码段** | 机器指令、函数体 | 整个程序 | — | `counter` `main` |
| `.data` | 已初始化的全局/静态变量 | 整个程序 | **写好初值** | `g_init` `sg` `cnt` |
| `.bss` | **未**初始化的全局/静态变量 | 整个程序 | **自动为 0** | `g_zero` |
| **栈（stack）** | 自动局部变量、形参、返回地址 | 进块生、出块死 | **不确定（垃圾值）** | `local` `g_init`(shadow 内) |

| 存储区 | 存放内容 | 生存期 | 初值 | 代码里的例子 |
| --- | --- | --- | --- | --- |
| **堆（heap）** | `malloc` / `calloc` 申请的内存 | `malloc` 到 `free` | **不确定** | `int *p = malloc(...)` |
| **只读数据段** | 字符串常量、`const` 常量 | 整个程序 | 固定 | `"hello"` |

- **各段的地址高低顺序**（低 → 高）：`.text` → `.data` → `.bss` → **堆（向上增长）** → **.....** → **栈（向下增长）**。用 `printf("%p", (void*)&变量)` 打印地址就能直观看出：`.text` 最小，`.data`/`.bss` 居中，**栈地址大得最多**。这也是为什么栈上局部变量的值**不打初值就是垃圾**（它属于运行期才建立的内存）。
- **一句话判据：加了 `static`（或写在函数外）→ 生命周期贯穿整个程序，初值自动为 0；没加 `static` 且写在函数内 → 生命随代码块，初值是垃圾。**
- **局部变量与全局变量重名**：局部会**屏蔽（shadow）**全局，函数内用的是局部变量（如上例 `shadow` 返回 999，但 `g_init` 全局仍是 10）。
- **声明 vs 定义**：定义**分配存储**（如 `int a;`），声明**只告知存在**（如 `extern int a;`）。变量可多次声明但只能定义一次；同一变量可以有多个声明。
- `static` 用在全局与局部含义不同：修饰**全局**→ 限制可见性为**本文件**；修饰**局部**→ 改变**存储期为整个程序**。

**提示**：这部分以常规函数的定义与调用为考查对象，重点考查参数传递、返回值、递归过程及变量的作用域和存储期。不涉及利用宏定义程序段实现函数，也不涉及利用函数指针模拟其他函数高阶用法等知识，函数指针的基本概念和简单用法归入指针及其应用部分考查。

## 六、指针及其应用

1. 掌握地址、指针变量、取地址运算和间接访问；能正确声明、初始化和使用指针，理解空指针及有效访问的条件。

**解析**：

- **指针变量的本质**：存放的是**地址**，且带有“指向什么类型”的信息。`int *p;` 表示 `p` 指向 `int`。
- 两个核心运算符：`&x` 取变量 `x` 的地址；`*p` **解引用**（取得 `p` 所指对象）。二者互为逆运算：`*&x` 就是 `x`。
- **声明与初始化的陷阱**：`int *p, q;` 中 **`p` 是指针，`q` 是 int**（`*` 只修饰紧跟的一个变量）。
- **未初始化指针是危险的**：它指向随机地址，解引用会崩溃。好习惯：`int *p = NULL;` 或直接指向有效对象。
- **空指针**：`NULL`（定义于 `<stddef.h>`/`<stdio.h>`，通常是 `((void*)0)`）。解引用空指针是未定义行为，通常立即崩溃。
- 安全访问的守卫：`if (p != NULL) { ... *p ... }`。
- **指针的大小**：与所指类型无关，同一平台上所有指针大小相同（64 位下 8 字节，32 位下 4 字节）。
- **野指针与悬垂指针**：野指针 = 未初始化；悬垂指针 = 所指内存已释放。`free(p)` 后应 `p = NULL;`。

- 经典面试题——用指针交换两数：

```
void swap(int *a, int *b) { int t = *a; *a = *b; *b = t; }
/* 调用：swap(&x, &y); */
```

2. 掌握通过指针访问一维数组和字符串、用指针参数访问或修改实参所指对象的方法；理解指针运算与所指类型的关系，以及数组名在常见表达式中的转换。

**解析：**

- **数组名退化为指针**：在大多数表达式中，`a` 等价于 `&a[0]`，类型为 `int *`；但 `sizeof(a)` 和 `&a` 是例外。
- **下标与指针的等价关系**（必记）：

```
a[i]  <=>  *(a + i)  <=>  *(p + i)  <=>  p[i]
&a[i] <=>  a + i
```

- **指针运算的尺度**：`p + 1` 不是地址加 1，而是**加一个 `sizeof(*p)` 字节**。`int *p; p+1` 实际前进 4 字节；`char *p; p+1` 前进 1 字节。
- 两个指针相减得到**元素个数**（`ptrdiff_t`），不是字节数：`&a[5] - &a[2] == 3`。
- 只有指向**同一数组**的两个指针相减才有意义。
- **遍历数组的指针写法**：

```
for (p = a; p < a + n; p++) sum += *p;
```

- **字符串遍历**：`while (*s) { ...; s++; }`（等价于 `*s != '\0'`）。
  - **但请慎重**：这样写会把 `s` 推到串尾，之后 `strlen`/`strcpy`/`puts` 之类的库函数就**失效**（它们都要求参数指向串头）。
  - **推荐写法**：先复制一份游标再走，**保留串头**：

```
const char *p = s;          /* 游标，可自由移动 */
while (*p) { putchar(*p); p++; }
/* 此处 s 未变，仍可 strcpy(dst, s) / printf("%s", s) */
```

- **字符指针 vs 字符数组**：

```
char s[] = "abc";   /* 可写数组，内容可修改 */
char *p  = "abc";   /* 指向只读字符串常量，*p = 'x' 会崩 */
```

- `const` **与指针：三种写法，语义完全不同（高频考点）**
  - **一句话抓住本质**：`const` 在 `*` 左边 → 锁“值”；`const` 在 `*` 右边 → 锁“指针”。

| 写法 | 读法 | *p 可改吗 | p 可改吗（改指向） | 记忆 |
|---|---|---|---|---|
| `const int *p` | p 指向「const int」 | **✗ 不可改值** | ✓ 可以改指向 | **const 在 * 左 → 锁值** |
| `int * const p` | p 是「const 指针」，指向 int | ✓ 可以改值 | **✗ 不可改指向** | **const 在 * 右 → 锁指针** |
| `const int * const p` | 两者都是 const | ✗ 不可改值 | ✗ 不可改指向 | **两边都有 → 全锁** |

- **完整可运行示例：**

```c
#include <stdio.h>

int main(void) {
    int x = 1, y = 2;

    /* ① 指向常量的指针：值不可改，指针本身可改方向 */
    const int *p1 = &x;
    /* *p1 = 10;    ✗ 编译错误：assignment of read-only location '*p1' */
    p1 = &y;        /* ✓ 改指向没问题 */

    /* ② 常量指针：指针本身不可改，所指值可改 */
    int * const p2 = &x;
    *p2 = 10;       /* ✓ 改值没问题 */
    /* p2 = &y;     ✗ 编译错误：assignment of read-only variable 'p2' */

    /* ③ 两者都 const */
    const int * const p3 = &x;
    /* *p3 = 5;     ✗ 值不可改 */
    /* p3 = &y;     ✗ 指向不可改 */

    printf("x=%d y=%d\n", x, y);   /* x=10 y=2 */
    return 0;
}
```

- **实测的编译错误原文**（记住关键字，考试时能对上号）：
  - 改 `const int *` 所指内容 → `error: assignment of read-only location '*p'`
  - 改 `int * const` 的指向 → `error: assignment of read-only variable 'p'`
- **两种等价写法**（`const` 写在 `int` 前面还是后面，效果一样）：

```
const int *a;    /* 与下一行完全等价 */
int const *b;    /* const 修饰的都是 int（被指向的类型） */
```

- **赋值规则：只能“收紧”，不能“放宽”**（权限视角）

| 赋值方向 | 是否允许 | 原因 |
| --- | --- | --- |
| `int *` → `const int *` | ✓ **允许** | 权限**收紧**，更安全 |
| `const int *` → `int *` | ⚠ **警告** | 权限**放宽**，丢掉了 `const` 保护 |

```
int x = 1;
const int *cp = &x;      /* ✓ 普通指针赋给只读指针，允许 */
int *q = cp;             /* ⚠ warning: initialization
discards 'const'
                                qualifier from pointer target
type */
```

- **最容易误解的一点：`const` 保护的是“通过这个指针的写操作”，不是那块内存**

```
int z = 7;
const int *cp = &z;
*cp = 8;    /* ✗ 通过指针写，被禁止 */
z = 8;      /* ✓ 变量 z 本身不是 const，直接写完全合法 */
```

  也就是说 `const int *p = &z;` **并没有**把 `z` 变成只读变量，只是“**承诺不通过 `p` 去改它**”。

- **实用场景**：函数参数用 `const` 表明“只读不改”，这是 C 里的标准写法：

```
void print_all(const int *a, int n);            /* 承诺不修改
数组内容 */
void show(const struct Student *p);             /* 只读取记录
*/
size_t strlen(const char *s);                   /* 标准库原型
就是这么写的 */
```

- **记忆口诀**：“**`const` 靠左锁值，靠右锁指针，两边都有全锁死**”；再补一句 **“权限只能收紧不能放宽”**。

- **为什么指针能修改实参**：传的是地址，函数内 `*p = ...` 写的就是调用者的内存。这是“返回值只能带一个”时的常用绕行手段。

3. 理解字符数组与字符指针、指针数组与指向数组的指针的区别；能分析简单的二维数组地址表达式。了解二级指针、函数指针、返回指针的函数及命令行参数的基本用途。

**解析：**

- **易混声明表（重点记住法：从变量名出发，由内向外读）：**

| 声明 | 读法 | 含义 |
|---|---|---|
| `char *p` | p 是指向 char 的指针 | 字符串指针 |
| `char p[10]` | p 是 10 个 char 的数组 | 字符数组 |
| `char *a[10]` | a 是数组，元素是 `char *` | **指针数组**（10 个字符串） |
| `char (*a)[10]` | a 是指针，指向 10 个 char 的数组 | **数组指针**（指向一行） |
| `int **pp` | pp 是指针，指向 `int *` | 二级指针 |
| `int (*f)(int)` | f 是指针，指向函数 | 函数指针 |
| `int *f(int)` | f 是函数，返回 `int *` | 返回指针的函数 |

- **区分诀窍**：`[]` 与 `()` 的优先级高于 `*`，所以 `char *a[10]` 先看 `a[10]`（数组）；`char (*a)[10]` 用括号强制先看 `*a`（指针）。
- **二维数组的地址表达式**（设 `int a[3][4]`）：
  - `a` 类型 `int(*)[4]`，指向第 0 行；`a + i` 指向第 `i` 行，**跨过一个行**（16 字节）。
  - `a[i]` 类型 `int *`，等价于 `*(a + i)`，指向第 `i` 行首元素。
  - `a[i][j]` 等价于 `*(*(a + i) + j)`。
  - 注意：`a` 与 `a[0]` 数值相同，但**类型不同、`+1` 跳过的长度不同**。
- **二级指针** `int **pp`：指向"指针的指针"。经典用途：① 在函数内**修改一级指针本身**（如 `void f(char **p)`）；② 指针数组的数组名会退化为二级指针（如 `argv`）。
- **函数指针 + 返回指针的函数（完整代码示例）：**

```c
#include <stdio.h>

int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }

/* 返回指针的函数：返回 static 数组里的元素（安全） */
char *num2name(int i) {
    static char *name[] = {"zero", "one", "two"};
    if (i < 0 || i > 2) return NULL;   /* 越界返回空指针 */
    return name[i];
}

int main(void) {
    /* ① 函数指针：定义 + 两种等价的调用写法 */
    int (*f)(int, int) = add;       /* f 是指针，指向"两个 int 进、一个 int 出"的函数 */
    printf("%d %d\n", f(3, 4), (*f)(3, 4));   /* 7 7 —— 两种写法完全等价 */
    f = sub;                        /* 可以改指向别的同型函数 */
    printf("%d\n", f(9, 4));        /* 5 */
```

```
    /* ② 函数指针数组：分派表（按编号选函数） */
    int (*ops[2])(int, int) = {add, sub};
    printf("%d %d\n", ops[0](5, 2), ops[1](5, 2));   /* 7 3 */

    /* ③ 返回指针的函数 */
    char *s = num2name(2);          /* 拿到 "two" 的地址 */
    printf("%s\n", s);
    return 0;
}
```

**实际输出**：`7 7` / `5` / `7 3` / `two`

- **函数指针的读法**：`int (*f)(int, int)` —— 从 `f` 出发，**先看 `*f`（f 是指针）**，再看后面的 `(int,int)`（指向函数），最后看最左的 `int`（返回值类型）。
- **为什么 `f(3,4)` 和 `(*f)(3,4)` 都对**：函数名本身就是地址，`f` 已经是指针，`*f` 又还原成函数，C 允许省略这个 `*`。
- **`(*f)` 的括号不能省**：写成 `int *f(int,int)` 就变成「返回 `int*` 的**函数声明**」，含义完全变了。
- **`num2name` 为什么返回 `static char *name[]` 是安全的**：`static` 数据在**静态区**，函数返回后依然存在。若写成 `char name[3][6]`（普通局部数组），返回后栈帧失效，就成了**悬垂指针**。

- **数组指针 `int (*p)[3]` 详解（这是最容易卡住的一个）**
  - **先把两个词分清**（关键就在 `[]` 和 `*` 谁先结合）：

| 写法 | 读法（从变量名出发） | 本质 | sizeof |
|---|---|---|---|
| `int *a[3]` | `a` 先和 `[3]` 结合 → **a 是数组**，元素是 `int*` | **指针数组**（3 个指针） | `3 × 8 = 24` |
| `int (*p)[3]` | 括号强制 `p` 先和 `*` 结合 → **p 是指针**，指向含 3 个 `int` 的数组 | **数组指针**（1 个指针） | `8` |

  - **口诀**：`[]` 和 `()` 的**优先级高于** `*`。所以 `int *a[3]` 先看 `a[3]`（是数组）；加了括号 `int (*p)[3]` 才让 `*p` 先结合（是指针）。
  - **数组指针最自然的用途：遍历二维数组**（下面三种写法结果完全相同，都输出 `1 2 3 4 5 6`）：

```
#include <stdio.h>

int main(void) {
    int a[2][3] = {{1,2,3},{4,5,6}};
    int (*p)[3] = a;              /* p 指向第 0 行（一个含 3 个 int 的数组） */

    /* 写法①：直接用 p 做二维下标（最直观） */
    for (int i = 0; i < 2; i++)
```

```
        for (int j = 0; j < 3; j++)
            printf("%d ", p[i][j]);        /* p[i][j] 完全等价于 a[i][j] */

    /* 写法②：外层先取到第 i 行（还是数组指针），内层照旧用下标 */
    for (int i = 0; i < 2; i++) {
        int (*row)[3] = p + i;             /* row 指向第 i 行，步长仍是一整行 */
        for (int j = 0; j < 3; j++)
            printf("%d ", row[0][j]);      /* 纯数组下标，不移动指针 */
    }

    /* 写法③：外层把一行"降级"成 int*，内层同样用下标 */
    for (int i = 0; i < 2; i++) {
        int *row = *(p + i);               /* 取第 i 行首元素的地址 */
        for (int j = 0; j < 3; j++)
            printf("%d ", row[j]);         /* 纯数组下标，不移动指针 */
    }
    return 0;
}
```

**三种写法的对应关系**（这是理解数组指针的关键）：

| 写法 | 外层拿到什么 | 内层怎么取 | 等价于 |
|---|---|---|---|
| ① | `p` 本身（数组指针） | `p[i][j]` | `a[i][j]` |
| ② | `row = p + i`（数组指针，指向第 i 行） | `row[0][j]` | `a[i][j]` |
| ③ | `row = *(p + i)`（**降级为 `int *`**，行首元素地址） | `row[j]` | `a[i][j]` |

- **写法②要写 `row[0][j]` 而不是 `row[j]`**：因为 `row` 的类型是 `int (*)[3]`（**指向数组**），`row[0]` 才是一行里的首元素，所以要多一层下标。
- **写法③能写 `row[j]`**：因为 `*(p + i)` 已经把类型从 `int (*)[3]` **降级成了 `int *`**，此时 `row` 就是普通的一维指针。
- **两者都不要移动 `row` 本身**——`row[j]` 这种下标写法不改变 `row`，循环结束时 `row` 仍指向行首，后续还能继续用。这正是「不要动串头/行头指针」的好习惯。

- **核心理解：`p + i` 的"步长"是一整行，不是 4 字节**
  - 因为 `p` 指向的是「**含 3 个 int 的数组**」，所以 `p + 1` 直接跨过**一整个行**。
  - 实测：`sizeof(*p) = 12`，`(char*)(p+1) - (char*)p = 12` —— 正好一行 3 × 4 字节。

| 表达式 | 类型 | +1 跨过的字节数 |
| --- | --- | --- |
| int *q（指向单个 int） | int * | 4（一个 int） |
| int (*p)[3]（指向一行） | int (*)[3] | 12（一行 3 个 int） |

- **两层解引用的含义**（写法③的 *(p + i) 为什么能得到 int *）：
  - p + i —— 第 i **行**的地址（类型 int (*)[3]，还是个**数组指针**）
  - *(p + i) —— 取出行地址后**退化为 int ***，即第 i 行**首元素**的地址
  - *(p + i) + j —— 第 i 行**第 j 个元素**的地址
  - *(*(p + i) + j) —— 再解引用，拿到**元素的值**

- **对照：指针数组的用法**（注意它是"数组"，不是"指针"，+1 只跨 8 字节）：

```
int r0[3] = {1,2,3}, r1[3] = {4,5,6};
int *arr[2] = {r0, r1};   /* 2 个 int* 组成的数组，两行可以不相邻 */
for (int i = 0; i < 2; i++)
    for (int j = 0; j < 3; j++)
        printf("%d ", arr[i][j]);     /* 用法很像，但机制完全不同 */
```

  - arr 是**指针数组**：arr[0]、arr[1] 是**各自独立**的两个指针，可以指向**互不相邻**的两行（r0 和 r1 就是分开的数组）。
  - p 是**数组指针**：它只记录**一行的大小**，靠 p + i 的"整行步长"**推算**出下一行的位置——所以要求二维数组必须是**连续存放**的。

- **一句话区分：看名字先跟谁结合**——先跟 [] 就是**数组**（指针数组），先跟 * 就是**指针**（数组指针）。int *a[3] 是"**装指针的数组**"，int (*p)[3] 是"**指向数组的指针**"。

- **命令行参数**：int main(int argc, char *argv[])。argc 是参数个数（含程序名），argv[0] 是程序名，argv[i] 是第 i 个参数，argv[argc] == NULL。

**提示**：这部分重点考查指针所指对象、访问范围及指针操作对数据的影响，程序采用含义明确的声明和表达式，不以复杂声明、多层间接访问或运算符的刻意嵌套设置阅读障碍。二级指针、函数指针、返回指针的函数及命令行参数仅要求理解基本含义、辨认简单用法，不要求利用这些机制设计复杂程序。

## 七、结构体与其他自定义类型

1. 掌握结构体类型及变量的定义、初始化和成员访问，掌握结构体数组和结构体指针的基本应用；能区分并正确使用成员运算符 . 和 ->。

**解析**：

- **定义与声明**：

```
struct Student {          /* 定义类型，不占存储 */
    char name[20];
    int  age;
```

```
    float score;
};
struct Student s1 = {"Zhao", 18, 90.5f};  /* 定义变量并初始化 */
```

- **typedef 简化写法全览（五种写法对照，都能编译运行）：**

```
#include <stdio.h>

/* ① 传统写法：先定义 struct 类型，使用时必须带着 struct 关键字 */
struct PointA { int x, y; };
struct PointA pa = {1, 2};          /* ✓ 必须写 struct PointA */
/* PointA pb = {3, 4}; */           /* ✗ 错！PointA 这个名字还不存
在 */

/* ② typedef 起别名（分两步）：先定义 struct，再起别名 */
struct PointB { int x, y; };
typedef struct PointB PointB_t;
PointB_t pb = {3, 4};               /* ✓ 别名可直接用，省掉 struct
*/

/* ③ typedef + 匿名 struct（一步到位，最常用） */
typedef struct { int x, y; } PointC;
PointC pc = {5, 6};

/* ④ typedef + 带标签（既能用别名，也能用 struct 名，最灵活） */
typedef struct PointD { int x, y; } PointD_t;
struct PointD pd1 = {7, 8};         /* 老写法照样能用 */
PointD_t      pd2 = {9, 10};        /* 新别名也能用 */

/* ⑤ 只定义变量、不给类型名（匿名 struct） */
struct { int x, y; } aaa;           /* aaa 是一个变量，不是类型名 */

int main(void) {
    printf("%d %d\n", pa.x, pa.y);      /* 1 2 */
    printf("%d %d\n", pb.x, pb.y);      /* 3 4 */
    printf("%d %d\n", pc.x, pc.y);      /* 5 6 */
    printf("%d %d\n", pd1.x, pd1.y);    /* 7 8 */
    printf("%d %d\n", pd2.x, pd2.y);    /* 9 10 */
    aaa.x = 11; aaa.y = 12;
    printf("%d %d\n", aaa.x, aaa.y);    /* 11 12 */
    return 0;
}
```

- **typedef 的语义：它是"给类型起别名"，不是"定义新类型"**
  - **本质**：`typedef A B;` 的含义是「**从此 B 就是 A 的同义词**」。编译器在语法分析阶段把 B 替换理解为 A，**不产生任何新类型**、不分配存储、不在运行时存在。
  - **所以下面的写法都合法**（证明"只是别名"）：

```c
typedef unsigned int uint;
uint a = 1;                 /* 等价于 unsigned int a = 1; */
unsigned int b = a;         /* 两者可以互相赋值，不存在"类型不同" */

typedef int Arr[10];        /* Arr 是"含10个int的数组"的别名 */
Arr x;                      /* 等价于 int x[10]; */
```

- **关键区别 1：`typedef` 是别名，`#define` 是文本替换**

| 对比项 | `typedef` | `#define` |
|---|---|---|
| 处理阶段 | **编译阶段**（语法分析时就认得） | **预处理阶段**（纯文本替换） |
| 是否理解作用域 | ✓ 有作用域，可定义在函数内 | ✗ 没有作用域，从定义处到文件末 |
| 能否做指针别名 | ✓ `typedef int *Pint;` | ✗ `#define Pint int *` 会有坑 |
| 出错提示 | 明确是类型错误 | 展开后报错，难定位 |

```c
typedef int *Pint;
Pint a, b;          /* ✓ a、b 都是 int* */

#define Pint2 int *
Pint2 c, d;         /* ✗ 展开成 int *c, d;  → c 是指针，d 是 int!  */
```

- **关键区别 2：有标签 vs 无标签**（就是写法③和写法④的区别）
  - `typedef struct { ... } PointC;` —— **匿名**，`PointC` 是唯一名字，**无法自引用**（结构体内部不能写 `PointC *next`）。
  - `typedef struct PointD { ... } PointD_t;` —— **有标签** `PointD`，可以自引用，链表/树节点必须这样写：

```c
typedef struct Node {
    int data;
    struct Node *next;    /* ✓ 必须写 struct Node，不能写 Node（别名还没生效） */
} Node;
```

**注意**：`struct Node *next;` 里**只能写** `struct Node`。因为在结构体内部时，`Node` 这个 `typedef` 别名**还没定义完**，编译器还不认识它。

    - **一句话总结**：`typedef` 只是"**起个短名字**"，类型还是原来那个类型；面试/考试常考"`typedef` 和 `#define` 的区别"以及"结构体自引用为什么必须带 `struct`"。

- **`.` 与 `->` 的区分（必考）**：

| 运算对象 | 运算符 | 例 |
|---|---|---|
| 结构体**变量** | `.` | `s1.age` |
| 结构体**指针** | `->` | `p->age` 等价于 `(*p).age` |

  - 记法：**有指针用箭头，没指针用点**。`(*p).age` 的括号不能省，因为 `.` 优先级高于 `*`。
  - **结构体可整体赋值**：`s2 = s1;` 合法（逐字节复制）；但**不能用 `==` 比较两个结构体**，要逐成员比。
  - **结构体数组**：`struct Student st[30];`，用 `st[i].age` 访问。
  - **结构体指针遍历**：`for (p = st; p < st + n; p++) sum += p->score;`

2. 理解结构体嵌套，以及结构体变量、结构体指针作为函数参数的用法；能完成简单记录的查询、统计与排序。

**解析**：

- **嵌套**：成员本身也可以是结构体，多层访问用多个 `.`：`s.birth.year`。
- **传结构体的两种方式**（性能差别很大）：
  - 传值 `void f(struct Student s)`：**整体复制一份**，大结构体开销大；函数内修改不影响实参。
  - 传指针 `void f(struct Student *p)`：只传地址，**高效**；用 `p->age` 访问，修改会影响实参。
  - 如果只是读取不想改，加 `const`：`void f(const struct Student *p)`。
- **排序**：结构体数组的排序是**交换整个结构体**或交换指针。注意 `strcmp` 比名字、`>` 比分数：

```
/* 按分数降序的冒泡排序 */
for (i = 0; i < n-1; i++)
    for (j = 0; j < n-1-i; j++)
        if (st[j].score < st[j+1].score) {
            struct Student t = st[j]; st[j] = st[j+1]; st[j+1] =
t;
        }
```

- **查询/统计模板**：线性扫描，用标志变量或计数变量汇总。

3. 理解枚举类型和 `typedef` 的基本用法；了解共用体的含义及其与结构体的存储区别。结构体具体占用空间的计算按题目给定的类型大小和布局规则进行。

**解析**：

- **枚举**：

```
enum Week { MON, TUE, WED, THU, FRI, SAT, SUN };
```

    - 枚举常量**默认从 0 递增**（MON=0, TUE=1, ...）；可显式指定：`enum { A = 1, B = 5, C };` 则 C = 6。
    - **枚举变量本质上是整数类型**（通常为 `int`），底层可赋任意整数值，但**不建议**超出枚举范围。
    - 优点：用有意义的名字代替魔法数字，可提高可读性。
  - `typedef`：为已有类型起**别名**，不是新类型。

```
typedef unsigned int uint;
typedef int Arr[10];        /* Arr 是 int[10] 的别名，Arr a; 即 int a[10]; */
```

    - 与应用场景对比：`#define` 是**文本替换**（预处理阶段），`typedef` 是**类型别名**（编译阶段）。
  - **共用体（union）与结构体的根本区别：**

| **对比项** | `struct` | `union` |
| --- | --- | --- |
| 成员存储 | 各成员**各自占空间**，依次存放 | 所有成员**共用同一块**空间，相互覆盖 |
| 大小 | ≥ 各成员大小之和（考虑对齐/填充） | = **最大成员**的大小（考虑对齐） |
| 同时有效 | 所有成员可同时有效 | **同一时刻只能有一个成员有效** |
| 典型用途 | 组织多条相关数据 | 节省空间、类型双关（同一内存多种解释） |

  - **结构体大小的计算**（题目会给规则）：需考虑**内存对齐**。经典规则：① 每个成员相对首地址的偏移必须是其**自身大小**的整数倍；② 结构体总大小必须是**最大成员大小**的整数倍；③ 需要时在成员间和末尾**填充字节**。
  - 例：`struct { char c; int i; }` 通常是 **8** 字节（c 后填充 3 字节），`struct { char c; char d; int i; }` 也是 8 字节。

> **提示**：这部分重点考查利用结构体组织数据、访问成员及处理简单记录的能力，不涉及位域、复杂的嵌套声明或利用共用体进行跨类型的数据解释。枚举和 `typedef` 以基本用法为主。涉及结构体存储布局和占用空间计算时，题目将给出必要条件，不要求记忆特定编译环境下的布局规则。

## 八、编译预处理与程序组织

1. 掌握 `#include` 和不带参数的宏定义；理解宏替换、带参数的宏与函数调用的区别，能分析简单宏展开及括号对结果的影响。

> **解析：**
>
> - 预处理命令以 `#` 开头，**行末不加分号**，且在**编译之前**完成。
> - **`#include` 的两种写法：**
>   - `#include <stdio.h>`：从**系统目录**找（标准库头文件）。
>   - `#include "my.h"`：先从**当前目录**找，找不到再去系统目录（自定义头文件）。
> - **不带参数的宏：**`#define PI 3.14159`。预处理时**纯文本替换**，不做类型检查、不分配存储、不计算。
> - **带参数的宏：**`#define SQR(x) ((x)*(x))`。本质上仍是文本替换。
> - **括号陷阱（必考）：**

```
#define SQR(x)  x*x          /* 错 */
SQR(1+2)  →  1+2*1+2  =  5     /* 不是 9!  */
#define SQR(x)  ((x)*(x))    /* 对 */
SQR(1+2)  →  ((1+2)*(1+2))  =  9
```

同理 `#define ADD(a,b) ((a)+(b))`。**整个表达式要加最外层括号，每个参数也要加括号。**

- **宏与函数的区别（重点）：**

| 对比项 | 宏 | 函数 |
|---|---|---|
| 处理阶段 | 预处理（编译前） | 运行期 |
| 机制 | 文本替换，**无类型检查** | 参数传递，**有类型检查** |
| 开销 | 无调用开销，但代码膨胀 | 有调用/返回开销，代码不膨胀 |
| 副作用 | 参数可能被**求值多次** | 参数只求值一次 |
| 调试 | 难以调试 | 可单步跟踪 |

- **多次求值陷阱**：`#define MAX(a,b) ((a)>(b)?(a):(b))`，若有副作用如 `MAX(i++, j++)`，`i++` 可能执行两次。
- **取消除宏**：`#undef PI`。
- 宏名习惯**全大写**，以便与变量、函数区分。

2. 了解条件编译和多文件程序的基本组织方式，了解头文件、外部声明和文件作用域下 `static` 的基本用途。

**解析：**

- **条件编译**（选择性地编译某些代码）：

```
#if 常量表达式
#elif 常量表达式
#else
#endif
#ifdef 宏名  ... #ifndef 宏名  ... #endif
```

  - `#ifdef` 等价于 `#if defined(...)`；可用于"本环境 vs 跨平台"、调试开关。
  - 其条件是**预处理期**求值的常量，不能用变量。
- **头文件保护（必会）**，防重复包含：

```
#ifndef STUDENT_H
#define STUDENT_H
/* 头文件内容 */
#endif
```

- **多文件程序组织（三段式）：**

- `.h` 头文件：放**类型定义、宏、函数声明、`extern` 变量声明**（不产生代码）。
- `.c` 源文件：放**函数定义、全局变量定义**（产生代码）。
- `main.c` / `#include` 后使用；编译时把多个 `.c` 一起交给编译器（如 `gcc main.c util.c`），由链接器合并。

- **声明 vs 定义（多文件核心）**：
  - 定义：`int count = 0;`（只可一处，分配空间）。
  - 声明：`extern int count;`（可多处，不分配空间）。头文件中放 `extern`。
- `static` **的两个含义**：
  - 修饰**全局变量/函数**时，把可见性限制在**本文件**（其他文件即使 `extern` 也看不到）。优点：避免重名冲突。
  - 修饰**局部变量**时，改变的是**存储期**（见第六章）。
- `static` **与** `extern` **完整实例（两个文件，实测可编译运行）**
  - **文件 1：**`util.c`（提供定义）

```c
#include <stdio.h>

int g_count = 10;              /* 定义全局变量：外部链接，别的文件能 extern 到 */
static int s_secret = 99;      /* 定义静态全局：内部链接，只有本文件可见 */

void add(void) { g_count++; }

int counter(void) {
    static int cnt = 0;        /* 静态局部：只初始化一次，函数返回后值保留 */
    cnt++;
    return cnt;
}

static void hidden(void) {     /* 静态函数：仅本文件可见 */
    printf("hidden called, s_secret=%d\n", s_secret);
}

void call_hidden(void) { hidden(); }   /* 对外暴露一个“壳”，间接调用 */
```

  - **文件 2：**`main.c`（使用声明）

```c
#include <stdio.h>

extern int g_count;            /* 声明：告诉编译器它定义在别处，不分配空间 */
int counter(void);             /* 函数声明 */
void add(void);
void call_hidden(void);

int main(void) {
    printf("g_count=%d\n", g_count);        /* 直接用别的文件
```

```
的变量 */
    add();
    printf("after add: g_count=%d\n", g_count);
    printf("counter=%d %d %d\n", counter(), counter(),
counter());
    call_hidden();
    return 0;
}
```

- **编译运行：**

```
gcc main.c util.c -o app.exe
./app.exe
```

- **实际输出**（即上例的真实结果）：

```
g_count=10
after add: g_count=11
counter=3 2 1
hidden called, s_secret=99
```

  - `g_count` 在 `util.c` 里被 `add()` 改成了 11，`main.c` 看得到 → `extern` **让全局变量跨文件共享**。
  - `counter()` 三次分别返回 **3 2 1**（不是 1 1 1）→ 证明 `static` **局部变量确实"只死一次"**。（三次调用的**求值顺序**是未规定的，这里按从右往左求值，所以打印 `3 2 1`；但三次调用拿到 1、2、3 三个不同值这件事本身是确定的。）
- **反例：静态的东西跨文件就用不了**（实测报链接错误）

```
/* bad1.c —— 想访问 util.c 里的 static 变量 */
extern int s_secret;              /* ✗ 声明得到，链接找不到 */
int main(void) { return s_secret; }
/* 报错：undefined reference to `s_secret' */

/* bad2.c —— 想调用 util.c 里的 static 函数 */
void hidden(void);                /* ✗ 同样链接失败 */
int main(void) { hidden(); return 0; }
/* 报错：undefined reference to `hidden' */
```

**注意**：这类错误**编译期不报，链接期才报**，报的关键字是 `undefined reference to '名字'`。

- **一句话对比：**

| 关键字 | 作用 | 跨文件可见吗 | 典型用途 |
| --- | --- | --- | --- |
| `extern` | **声明**（不分配空间），引用别的文件里的定义 | ✓ **正是为了跨文件** | `.h` 里放 `extern int count;` |
| `static`（修饰全局/函数） | **定义**但限制为内部链接 | ✗ **仅本文件** | 隐藏内部实现、避免重名 |
| `static`（修饰局部） | 改存储期为**整个程序** | 不适用（本来就是局部的） | 计数器、缓存、记忆上次状态 |

- **常见坑**：
  - `extern` **加初始化就变成定义了**：`extern int x = 5;` 等价于 `int x = 5;`（也是定义），别在头文件里这么写，否则多个 `.c` 包含它就会**重复定义**。
  - **头文件里放定义会导致重复定义**：`.h` 只放 `extern` 声明和函数声明；真正的 `int count = 0;` 只能出现在**一个** `.c` 里。
  - `static` **全局变量在** `.bss`**/**`.data` **中照样占空间**，只是**不给外部链接**，别以为它"消失了"。
  - `static` **函数无法被单元测试直接调用**（外部看不见），所以若确实需要外部调用，就得像上例 `call_hidden()` 那样加一层"壳"。

**提示**：这部分重点考查宏替换、文件包含及多文件程序中声明与定义的关系。带参数的宏仅涉及简单展开及括号使用，不考查利用宏组织复杂程序，也不涉及字符串化、记号拼接或多层嵌套的宏展开。多文件程序以理解各文件之间的关系为主，不要求掌握复杂的工程配置和构建工具。

## 九、动态内存分配与指针应用

1. 掌握 `malloc` 和 `free` 的基本用法，能根据数据类型和数量确定所需空间，检查分配结果并在适当时机释放空间；理解动态申请的空间与指针变量的关系，能识别越界访问、内存泄漏、重复释放和释放后访问等常见问题。

**解析**：

- 动态内存位于**堆**，生存期由程序员控制；不同于栈上的局部变量（出作用域自动回收）。
- `malloc` **原型**：`void *malloc(size_t size);`，需 `#include <stdlib.h>`。以**字节**为单位申请；返回 `void*`（无需强转，但 C++ 中必须转）。
- **申请习惯写法**（推荐用 `sizeof` 自算，避免平台差异）：

```
int *a = (int *)malloc(n * sizeof(int));     /* 数组 */
if (a == NULL) { /* 分配失败，处理 */ }
/* ... 使用 a[0..n-1] ... */
free(a);
a = NULL;                                    /* 防悬垂 */
```

- `free` **规则**：
  - `free(p)` 释放的是**整块内存**，不需要也不能只释放一部分。
  - 释放的是 `malloc` **返回的原始地址**；若 `p++` 后再 `free(p)` 会崩溃。
- **四大常见错误（必考）**：

| 问题 | 含义 | 后果 |
| --- | --- | --- |
| 越界访问 | 读写 `a[n]` 等超出范围 | 静默破坏堆结构，难排查 |
| 内存泄漏 | `malloc` 后没 `free` | 长期运行内存耗尽 |
| 重复释放 | 对同一块 `free` 两次 | 未定义行为，常崩溃 |
| 释放后访问 | `free(p)` 后又用 `*p` | 未定义行为（悬垂指针） |

- **内存泄漏的隐蔽形式**：指针被重新赋值导致原地址丢失，如 `p = malloc(...); p = malloc(...);` 第一块就泄露了。
- 只用 `malloc` + `free`（不考 `calloc`/`realloc`）。二者区别：`calloc` 会自动置 0，`realloc` 用于调整已分配空间大小。
- **动态空间的字符串（重点：字符串也是"数组 + '\0'"，所以动态申请时那一位不能忘）**
  - **最核心的一条：容量必须按 `strlen(s) + 1` 算**——多出来的那个 1 是给结尾的 `'\0'` 的。

```c
size_t need = strlen(src) + 1;    /* 别忘了 +1 */
char *s = malloc(need);
if (s == NULL) return 1;
strcpy(s, src);                   /* 现在才安全 */
```

| 写法 | 对吗 | 后果 |
| --- | --- | --- |
| `malloc(strlen(s) + 1)` | ✓ **正确** | 刚好装下 `'\0'` |
| `malloc(strlen(s))` | ✗ **少 1 字节** | 写完 `'\0'` 就**堆越界**，破坏堆结构 |
| `malloc(sizeof(s))`（`s` 是指针） | ✗ **只开了 8 字节** | 复制长串必然溢出 |
| `malloc(sizeof("hello"))` | ✓ 也可以 | `sizeof` 对**字符串字面量**含 `'\0'`，得 6 |

  - **读取任意长度的一行（`getchar` + `realloc` 翻倍扩容，可背）：**

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    int cap = 4, n = 0;                  /* cap=容量, n=已存字符数 */
    char *s = malloc(cap);
    if (s == NULL) return 1;
    s[0] = '\0';                         /* 空串：任何情况下都合法 */
```

```
    int c;
    while ((c = getchar()) != '\n' && c != EOF) {
        if (n + 1 == cap) {             /* 留一位给 '\0' */
            cap *= 2;
            char *t = realloc(s, cap);
            if (t == NULL) { free(s); return 1; }
            s = t;
        }
        s[n++] = (char)c;
        s[n]   = '\0';                  /* 始终保持是合法字符
串 */
    }
    printf("[%s] strlen=%zu n=%d cap=%d\n", s, strlen(s), n,
cap);
    free(s);
    return 0;
}
```

**实测结果**（`printf "hello world\n"` 与 `printf "ab\n"` 两种输入）：

| 输入 | 输出 | 说明 |
| --- | --- | --- |
| `hello world`（11 字符） | `[hello world] strlen=11 n=11 cap=16` | 容量 `4 → 8 → 16`，共扩两次 |
| `ab`（2 字符） | `[ab] strlen=2 n=2 cap=4` | 没超容量，**一次都不扩** |

- `n` **与** `strlen(s)` **始终相等**——因为每存一个字符就立刻补 `'\0'`，串永远合法。
- **为什么每次要** `s[n] = '\0'`：这样即使中途 `realloc` 或退出循环，`s` 也一直是**合法字符串**，可以直接交给 `printf`/`strlen`/`strcpy`。

- **动态字符串的拼接：先算总量再申请**（不要一边拼一边超出边界）：

```
const char *a = "hello", *b = " world";
size_t need = strlen(a) + strlen(b) + 1;   /* 4+1+4+1+1 之外
还要 +1 */
char *s = malloc(need);
if (s == NULL) return 1;
strcpy(s, a);           /* 先用 strcpy 打底 */
strcat(s, b);           /* 再用 strcat 追加 */
printf("s=[%s] len=%zu need=%zu\n", s, strlen(s), need);
free(s);
```

**实测输出**：`s=[hello world] len=11 need=12`——`need` 是 12，`strlen` 是 11，差的正是那个 `'\0'`。

- **动态字符串数组（"指针数组 + 每项单独申请"）**：

```
char *v[3];                                  /* ① 3 个指针，还没分配内容 */
const char *src[3] = {"aa", "bbbb", "cccccc"};
for (int i = 0; i < 3; i++) {
    v[i] = malloc(strlen(src[i]) + 1);       /* ② 每个元素按自己的长度开 */
    if (v[i] == NULL) return 1;              /* ③ 每次都要检查 */
    strcpy(v[i], src[i]);
}
for (int i = 0; i < 3; i++) printf("%s ", v[i]);    /* aa bbbb cccccc */
for (int i = 0; i < 3; i++) free(v[i]);      /* ④ 逐个释放（不是 free(v)） */
```

  - **为什么不能只 `free(v)`**：`v` 是**栈上的数组**，本身不需要释放；真正在堆上的是 `v[0]`、`v[1]`、`v[2]` 各自指向的 3 块内存，**一块都不能漏**。
  - **变长字符串数组**（行数也不定）就再套一层：`char **v = malloc(rows * sizeof(char *));`，然后逐行 `malloc`；释放时**先逐行 `free(v[i])`，最后 `free(v)`**（顺序不能反）。

- **动态字符串的字符串长度，仍然用 `strlen` 而不是 `sizeof`**：

```
char *s = malloc(100);
strcpy(s, "abc");
sizeof s     /* = 8: s 是指针，不是数组！ */
strlen(s)    /* = 3: 真正要的长度 */
```

  这里和第四章「数组名不退化」正好形成对照：**栈上真数组**可以用 `sizeof a`，**动态申请得到的指针**只能 `strlen` 或自己记 `n`。

- **易错清单**：

| 错误 | 后果 | 正确做法 |
|---|---|---|
| `malloc(strlen(s))` 少写 `+1` | 写 `'\0'` 时**堆越界** | `malloc(strlen(s) + 1)` |
| `malloc(sizeof(p))`（`p` 是指针） | 只开 8 字节 | `malloc(strlen(p) + 1)` |
| `char *p = malloc(10); p++; free(p);` | **实测崩溃**（退出码 `0xC0000374` 堆损坏） | 备份原地址，`free` 原始指针 |
| `char *v[3]` 只 `free(v)` | 3 块堆内存全泄漏 | `for (i) free(v[i]);` |
| 动态串用 `sizeof` 取长度 | 拿到指针大小 8 | 用 `strlen(s)` |

| 错误 | 后果 | 正确做法 |
| --- | --- | --- |
| `strcat(dst, src)` 前不算 `dst` 容量 | 溢出冲掉堆 | 先算 `strlen(dst)+strlen(src)+1` |

**`p++` 后 `free` 的正确写法**（对应第四章「不要把串头指针移走」的另一个后果）：

```
char *p = malloc(10);
char *head = p;       /* ① 备份原始地址 */
while (*p) p++;       /* ② 可以自由移动（例如遍历） */
free(head);           /* ③ 释放的一定是最初那个地址 */
```

- **开动态数组大小的板子（可直接背）：**

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n;
    if (scanf("%d", &n) != 1 || n <= 0) return 1;   /* 先拿到大小
*/

    int *a = malloc(n * sizeof(int));   /* 按 n 开数组 */
    if (a == NULL) {                    /* 必检 */
        printf("分配失败\n");
        return 1;
    }

    for (int i = 0; i < n; i++) a[i] = i + 1;   /* 当普通数组用 */
    for (int i = 0; i < n; i++) printf("%d ", a[i]);

    free(a);          /* 用完必释放 */
    a = NULL;         /* 防悬垂 */
    return 0;
}
```

- **动态二维数组的板子**（"行指针数组"，每一行单独申请）：

```
int rows = 2, cols = 3;
int **m = malloc(rows * sizeof(int *));       /* ① 先开 rows 个行指
针 */
if (m == NULL) return 1;
for (int i = 0; i < rows; i++) {
    m[i] = malloc(cols * sizeof(int));        /* ② 每行再开 cols 个
int */
    if (m[i] == NULL) return 1;
}
/* 用法完全一样：m[i][j] */
```

```
m[1][2] = 6;

for (int i = 0; i < rows; i++) free(m[i]);     /* ③ 先逐行释放 */
free(m);                                       /* ④ 再释放行指针数组
*/
```

**释放顺序不能反**：先 `free(m)` 之后行指针就丢了，那 rows 块行内存再也找不到 → 泄漏。

2. 能按照题目给定的操作规则，利用指针、数组或结构体管理动态申请的存储空间，完成数据的保存、读取和删除，并正确更新相关指针和计数变量。所需的存取规则和操作要求由题目明确给出。

**解析**：

- **"计数变量 + 指针"配对管理**是这类题的核心套路：用一个 `count` 记录当前有效元素个数，用指针访问存储区。
- **常用操作模板**：
  - 增加：`a[count] = x; count++;`
  - 遍历：`for (i = 0; i < count; i++) ... a[i] ...`
  - 查找：线扫找目标下标；找不到返回 −1。
  - 删除（**数组式**，保序）：找到后把后面的元素依次前移，再把 `count--`：

```
for (; i < count - 1; i++) a[i] = a[i + 1];
count--;
```

  - 删除（**链表式**）——**分「删头」和「删中间/尾部」两种情况，删头是特例，最容易写错**：
    - **情况 A：删的是头节点**——必须**改 `head` 本身**，所以函数要**返回新头**，调用处得**接住返回值**：

```
/* 删除头节点：旧头 free 掉，head 指向第二个节点 */
Node *del_head(Node *head) {
    if (head == NULL) return NULL;     /* 空链表，什么也不
做 */
    Node *victim = head;               /* ① 先记下旧头（待
释放） */
    head = head->next;                 /* ② head 后移到第
二个节点 */
    free(victim);                      /* ③ 再释放旧头 */
    return head;                       /* ④ 返回新头！ */
}
/* 调用处必须接住：head = del_head(head);  ← 少了这个赋
值，head 就成了悬垂指针 */
```

**三处易错**：① 顺序必须是「先记 → 再改 → 最后 free」，先 `free(head)` 就再也读不到 `head->next` 了；② 一定要 `return head`，`head` 是**局部副本**，函数内改了外面不知道；③ 调用处必须写 `head = del_head(head);`。

- **情况 B：删中间/尾部的节点**（也包括删头，用二级指针统一处理，**推荐背这个**）：

```c
/* 按值删除第一个等于 v 的节点；删头/删中/删尾都能处理 */
Node *del(Node *head, int v) {
    Node **pp = &head;      /* pp 指向"指向当前节点的那个指针" */
    while (*pp && (*pp)->val != v)
        pp = &(*pp)->next;  /* 让 pp 始终指向"前驱的 next 字段" */
    if (*pp) {              /* 找到了 */
        Node *victim = *pp;     /* ① 先记下要释放的那个节点 */
        *pp = victim->next;     /* ② 让前驱直接跳过它（删中部时等价 (前驱)->next = 目标->next） */
        free(victim);           /* ③ 最后释放 */
    }
    return head;            /* 删的是头时，head 已被 *pp 改掉了 */
}
```

  - **为什么用 `Node **pp` 而不是 `Node *prev`**：因为删**头**节点时没有"前驱节点"，用 `prev` 就得单独写一个 `if` 分支；用 `pp = &head` 后，**"头"和"前驱的 next"被统一成同一种东西**，一个循环就搞定。
  - **那句 `pp = &(*pp)->next` 的含义**：让 `pp` 从"指向当前节点"变成"指向当前节点的 `next` 字段"，也就是**向前挪一格**。

- **实测结果**（`1 2 3 4 5` 依次删头、删中、删尾、删不存在的值）：

| 操作 | 结果 | 说明 |
|---|---|---|
| `head = del(head, 1)` | `2 3 4 5` | **删头**，`head` 后移 |
| `head = del(head, 3)` | `2 4 5` | 删中间 |
| `head = del(head, 5)` | `2 4` | 删尾，前驱的 `next` 变 `NULL` |
| `head = del(head, 99)` | `2 4` | 没找到，**链表原样不动** |

```c
/* 完整可运行示例（含建表、打印、销毁） */
typedef struct Node { int val; struct Node *next; } Node;

Node *push(Node *head, int v) {          /* 头插法，返回新头 */
    Node *n = malloc(sizeof(Node));
    n->val = v; n->next = head;
    return n;
}
void print(Node *p) { for (; p; p = p->next) printf("%d
```

```
", p->val); putchar('\n'); }

int main(void) {
    Node *head = NULL;
    for (int i = 5; i >= 1; i--) head = push(head, i);
/* 建出 1 2 3 4 5 */
    print(head);                    /* 1 2 3 4 5 */
    head = del(head, 1);  print(head);   /* 2 3 4 5  ←
删头 */
    head = del(head, 3);  print(head);   /* 2 4 5    ←
删中间 */
    head = del(head, 5);  print(head);   /* 2 4      ←
删尾 */
    head = del(head, 99); print(head);   /* 2 4      ←
没找到 */
    while (head) { Node *t = head; head = head->next;
free(t); }  /* 整表销毁 */
    return 0;
}
```

- **链表删除的顺序铁律：先记（`victim` / `next`）→ 再断（改链接）→ 最后 `free`。**反过来先 `free` 就再也拿不到 `->next` 了：

```
free(p); p = p->next;   /* ✗ 用了已释放的内存（悬垂指针） */
```

- **删除类题目的三个坑：**① 删除前先保存后续信息（**顺序不能反**）；② **释放后把指针置空**（`p = NULL;`）；③ 别忘了更新**计数变量**；④ **删头是特例**——必须改 `head` 并把新头**返回/接住**。
- **扩容思路**（题目会给规则时）：申请更大空间 → 复制旧数据 → 释放旧空间 → 更新指针与容量。
- **数组扩容标准代码（`realloc`，容量翻倍）：**

```
int cap = 2, n = 0;                 /* cap=容量, n=已用个数 */
int *a = malloc(cap * sizeof(int));
if (a == NULL) return 1;
for (int v = 1; v <= 5; v++) {
    if (n == cap) {                 /* 满了才扩容 */
        cap *= 2;                   /* ① 先算新容量 */
        int *t = realloc(a, cap * sizeof(int));   /* ② 申请更大空间
*/
        if (t == NULL) { free(a); return 1; }     /* ③ 失败则先释放
旧的 */
        a = t;                      /* ④ 成功了才覆写指针 */
    }
    a[n++] = v * 10;                /* 存入新元素 */
}
free(a);
```

**输出**：存 5 个元素（10 20 30 40 50）时，容量变化 `2 → 4 → 8`，最终 `n=5, cap=8`。

- **扩容必记的三条**：
  - **绝不要写 `p = realloc(p, ...)`**：若 `realloc` 失败返回 `NULL`，旧内存地址就丢了，既没释放也无法访问 → **内存泄漏**。必须用临时变量 `t` 接。
  - **`realloc` 会自动搬运旧数据**，不需要自己 `memcpy`；它可能原地扩大，也可能换到新地址并释放旧块。
  - **扩容后只有 `[0, n)` 有效**，新扩出来的 `[n, cap)` 是未初始化的。
- **另一种写法——手动"搬家"**（不要求掌握，理解即可）：`malloc` 新块 → `memcpy` 旧数据 → `free` 旧块 → 指针改指新块，效果与 `realloc` 相同。
- **末尾空位幂等技巧**（处理输入时很实用）：哪怕用户第一句就按 Ctrl+Z（EOF），也要保证 `s[0]` 是合法的空串——在申请后先补一个 `'\0'`：

```c
char *s = malloc(cap * sizeof(char));
if (s == NULL) return 1;
s[0] = '\0';                          /* 空串，任何情况下都合法 */
int n = 0;
int c;
while ((c = getchar()) != EOF) {      /* 读一个字符就存一个 */
    if (n + 1 == cap) {               /* 留一位给 '\0' */
        cap *= 2;
        char *t = realloc(s, cap);
        if (t == NULL) { free(s); return 1; }
        s = t;
    }
    s[n++] = (char)c;
    s[n]   = '\0';                    /* 始终保持是字符串 */
}
printf("[%s] n=%d cap=%d\n", s, n, cap);
free(s);
```

**提示**：这部分仅涉及 `malloc` 和 `free`，不考查 `calloc`、`realloc` 及内存分配器的内部实现。题目将明确给出数据的存取规则和操作要求，重点考查根据规则申请和释放空间、通过指针访问数据，以及在操作过程中维护指向关系的能力，不要求预先掌握特定的数据组织方式或背诵成套的实现代码。

## 十、文件操作

1. 理解文本文件、二进制文件和文件类型指针的基本概念；掌握 `fopen`、`fclose` 及常见打开模式，能检查文件是否成功打开。

**解析**：

- **文本文件 vs 二进制文件（根本差别只有一句：文本模式会"翻译"换行）**
  - **文本文件**：内容被当作**一行行字符**看待。Windows 下读写时会自动做 **`\n` ⇄ `\r\n` 转换**——写文件时把你写的 `'\n'` 变成两个字节 `\r\n`，读文件时再把 `\r\n` 变回一个 `'\n'`。所以**文件里的字节数 ≠ 你写的字符数**。
  - **二进制文件**：内容的字节**原样进出**，不做任何转换。结构体、数组可以整块 `fwrite` 出去、整块 `fread` 回来。

- **一句话判据：要“给人看”用文本模式；要“给程序存数据”用二进制模式。**带 `b` 就是二进制（`"rb"`、`"wb"`）。
- **实测对比**（同样写入字符串 `"A\nB\n"`，共 4 个可见字符 + 2 个 `\n`）：

| 写入内容 | 模式 | 文件实际字节数 | 说明 |
|---|---|---|---|
| `"A\nB\n"` | `"w"`（文本） | **6** | 每个 `\n` 变成 `\r\n`，多出 2 字节 |
| `"A\nB\n"` | `"wb"`（二进制） | **4** | `\n` 原样，一个字节就是一个 `\n` |

- **文件类型指针：**`FILE *fp;`（定义在 `<stdio.h>`）。它**不指向文件里的数据**，而是指向一个描述**缓冲区、当前读写位置、状态标志**等信息的结构体。你只管拿它当“文件的把手”传给各个函数。
- **`fopen` 原型：**`FILE *fopen(const char *filename, const char *mode);`——成功返回 `FILE *`，**失败返回** `NULL`。
- **打开模式全表（必背）**

| 模式 | 含义 | 文件不存在时 | 已有内容 | 可读 | 可写 |
|---|---|---|---|---|---|
| `"r"` | 只读（文本） | **打开失败返回 `NULL`** | 保留 | ✓ | ✗ |
| `"w"` | 只写（文本） | 新建 | **清空（危险！）** | ✗ | ✓ |
| `"a"` | 追加（文本） | 新建 | 在末尾追加 | ✗ | ✓（只能末尾） |
| `"r+"` | 读 + 写 | 失败返回 `NULL` | 保留 | ✓ | ✓ |
| `"w+"` | 读 + 写 | 新建 | **清空** | ✓ | ✓ |
| `"a+"` | 读 + 写（追加） | 新建 | 末尾追加 | ✓ | ✓（只能末尾） |
| `"rb"` / `"wb"` / `"ab"` | 二进制模式 | 同上 | 同上 | — | — |
| `"r+b"` / `"wb+"` / `"a+b"` | 二进制读写 | 同上 | 同上 | — | — |

- **三个最容易混的点：**
  1. `"w"` **会先把文件清空。**文件里原来的内容**全部没了**，即使你一个字都没写。所以**读文件绝不能用** `"w"`。
  2. **只有** `"r"` **系列在文件不存在时会失败，**`"w"` / `"a"` 系列会**自动新建。**
  3. `"a"` **的写入位置永远在末尾，**即使你 `fseek` 到中间，下一次写还是回到末尾。想在中间改写要用 `"r+"` 或 `"w+"`。
- **打开失败必检**（高频考点，实测验证）：

```
FILE *fp = fopen("build/data.txt", "r");
if (fp == NULL) { printf("打开失败\n"); return 1; }   /* 必须判 NULL */
```

> **实测行为**：对**不存在**的文件用 "r" 打开 → 返回 NULL；用 "w" 打开同名新文件 → 返回正常指针（自动新建）。所以：

| 情况 | 返回值 | 说明 |
| --- | --- | --- |
| fopen("不存在的文件", "r") | NULL | 只读模式不会新建，必须判错 |
| fopen("不存在的文件", "w") | 正常 FILE * | 自动新建空文件 |
| fopen("不存在的目录/x.txt", "w") | NULL | 目录不存在，无法新建 |

- **fclose(fp) 的作用（三个都要知道）**：① 把**缓冲区里还没写出去的数据刷盘**；② 释放 FILE 结构占用的资源；③ 断开与操作系统的关联。**成功返回 0，失败返回 EOF**。
  - **为什么忘记 fclose 会丢数据**：fprintf / fputc 只是把数据写进了**内存缓冲区**，并没有立刻落盘。缓冲区满了或 fclose/程序正常退出时才真正写出去。**程序非正常中止（崩溃、exit 前未关）就可能丢掉最后一批数据**。
  - **配套的 fflush(fp)**：手动强制刷盘（不关闭文件）。调试时打印到 stdout 看不到输出，就可以 fflush(stdout);。
- 三个预定义流 stdin / stdout / stderr **不需 fopen，也不应 fclose**（由系统在程序启动/结束时自动管理）。
- **一个最小的"写→读"闭环（实测可编译运行）**：

```c
#include <stdio.h>

int main(void) {
    /* ① 写：fputc / fputs / fprintf 三种都上 */
    FILE *fp = fopen("build/demo.txt", "w");
    if (fp == NULL) { printf("打开失败\n"); return 1; }
    fputc('A', fp);                                  /* 写一个字符
*/
    fputc('\n', fp);
    fputs("hello", fp);                              /* 写一个字符
串 */
    fputc('\n', fp);
    fprintf(fp, "%d %s %.2f\n", 42, "world", 3.14);  /* 格式化写
*/
    fclose(fp);                                      /* 别忘了关!
*/

    /* ② 读：逐字符读回，验证字节数 */
    fp = fopen("build/demo.txt", "r");
    if (fp == NULL) { printf("打开失败\n"); return 1; }
    int ch, n = 0;
    while ((ch = fgetc(fp)) != EOF) {                /* 用返回值判
结束 */
        n++;
        printf("ch=%d [%c]\n", ch, (ch == '\n') ? 'N' : ch);
    }
    fclose(fp);
    printf("总字节数=%d\n", n);
    return 0;
}
```

**实测输出**（逐字节列出，N 代表换行符）：

```
ch=65 [A]      ch=10 [N]      ← 'A'=65, '\n'=10
ch=104 [h]     ch=101 [e]     ch=108 [l]   ch=108 [l]   ch=111 [o]
ch=10 [N]      ← "hello" 后面那个换行
ch=52 [4]      ch=50 [2]      ch=32 [ ]    ← 数字 42 和空格
ch=119 [w]     ch=111 [o]     ch=114 [r]   ch=108 [l]   ch=100 [d]
ch=32 [ ]      ch=51 [3]      ch=46 [.]    ch=49 [1]    ch=52 [4]
ch=10 [N]
总字节数=22
```

- **数一下**：A+\n(2) + hello+\n(6) + 42 world 3.14+\n(14) = **22** ✓ 与程序输出一致。**这说明文本模式下 \n 在"逐字符读"时确实还原成了一个 \n**（写时有 \r\n 转换，读时又转回来了）。

2. 掌握字符、字符串和格式化方式的基本文件读写；理解 fread、fwrite 的用途及返回值，能根据读写结果控制处理过程。

**解析：**

- **三组读写函数对照：**

| 类别 | 读 | 写 | 说明 |
|---|---|---|---|
| 字符 | fgetc(fp) / getc(fp) | fputc(c, fp) / putc | 返回 int（能区分 EOF） |
| 字符串 | fgets(buf, n, fp) | fputs(s, fp) | fgets 最多读 n-1 个，保留 \n |
| 格式化 | fscanf(fp, fmt, ...) | fprintf(fp, fmt, ...) | 用法同 scanf/printf |
| 二进制块 | fread(ptr, size, n, fp) | fwrite(ptr, size, n, fp) | 适合结构体数组 |

- **fread / fwrite 参数含义**：fread(缓冲区地址, 单个元素字节数, 元素个数, 文件指针)。

```
fwrite(a, sizeof(struct Student), n, fp);   /* 把 n 个记录一次性写入 */
fread(a,  sizeof(struct Student), n, fp);   /* 同理读回 */
```

- **返回值**：fread/fwrite 返回**实际成功读写的元素个数**（不是字节数）。若返回值小于请求值，说明遇到文件末尾或出错。
- **判 EOF**：fgetc 读到时返回 EOF（值为 −1，必须用 int 接收）；也可用 feof(fp)（只有尝试读过之后才可靠）。
- **常用读文件模板**：

```
while ((ch = fgetc(fp)) != EOF) { /* 处理 ch */ }
```

**(1) 字符读写：`fputc` / `fgetc`（最底层，最通用）**

| 函数 | 原型 | 返回值 |
| --- | --- | --- |
| `fputc(c, fp)` | `int fputc(int c, FILE *fp)` | 成功返回写入的字符，失败返回 `EOF` |
| `fgetc(fp)` | `int fgetc(FILE *fp)` | 成功返回字符（提升为 `int`），读到末尾或出错返回 `EOF` |

- **为什么用 `int` 接收**：要能同时表示 **0~255 的所有字符 + `EOF`（−1）**，`char` 装不下这个区分。
- **标准读文件模板（背下来）**：

```
int ch;                                 /* 必须是 int，不能是 char */
while ((ch = fgetc(fp)) != EOF) {
    putchar(ch);                        /* 或做任何处理 */
}
```

- **`getc` 与 `fgetc`**：功能完全相同，`getc` 通常实现为宏，略快但**不能传带副作用的表达式**（可能被求值多次）。
- **统计行数的惯用法**（`wc -l` 的简化版，实测 `lines.txt` 得 **2**）：

```
int ch, lines = 0;
while ((ch = fgetc(fp)) != EOF)
    if (ch == '\n') lines++;            /* 数换行符个数 */
```

**(2) 字符串读写：`fputs` / `fgets`（按行读写的主力）**

| 函数 | 原型 | 特点 |
| --- | --- | --- |
| `fputs(s, fp)` | `int fputs(const char *s, FILE *fp)` | **不自动加 `\n`**，需自己写 |
| `fgets(buf, n, fp)` | `char *fgets(char *buf, int n, FILE *fp)` | 最多读 `n-1` 个字符，**保留末尾的 `\n`**，自动补 `'\0'` |

- **`fgets` 的三个要点（重点）**：
  1. **最多读 `n-1` 个字符**（留一位给 `'\0'`），所以 `fgets(buf, 64, fp)` 最多装 63 个字符。
  2. **`\n` 会被读进 `buf` 并保留**——这是 `fgets` 和 `scanf("%s")` 最大的差别。
  3. 若一行比 `n-1` 还长，会**分成多次读**（下次调用接着读剩下的）。
- **`fgets` 的返回值**：成功返回 `buf`，**读到文件末尾或出错返回 `NULL`**。所以循环条件写 `while (fgets(...) != NULL)`，**不要写 `while (!feof(fp))`**。
- **为什么要手动去掉末尾 `\n`**（否则 `strcmp` 判等永远失败）：

```
size_t len = strlen(buf);
if (len > 0 && buf[len - 1] == '\n') buf[len - 1] = '\0';
```

- **完整实测示例**（写三行，其中最后一行**没有**换行符，再逐行读回）：

```
#include <stdio.h>
#include <string.h>

int main(void) {
    /* ① 用 fputs 逐行写（注意前面两个自己带了 '\n'，第三个故意不带）
*/
    FILE *fp = fopen("build/lines.txt", "w");
    if (!fp) return 1;
    fputs("line one\n", fp);
    fputs("line two\n", fp);
    fputs("third line no newline", fp);    /* 没有 '\n' */
    fclose(fp);

    /* ② 用 fgets 逐行读回，检查是否带 '\n' */
    fp = fopen("build/lines.txt", "r");
    if (!fp) return 1;
    char buf[64];
    int i = 0;
    while (fgets(buf, sizeof(buf), fp) != NULL) {
        size_t len = strlen(buf);
        int has_nl = (len > 0 && buf[len - 1] == '\n');
        printf("L%d len=%zu newline=%d text=[%s]\n", i++, len,
has_nl, buf);
    }
    fclose(fp);
    return 0;
}
```

**实测输出：**

```
L0 len=9 newline=1 text=[line one
]
L1 len=9 newline=1 text=[line two
]
L2 len=21 newline=0 text=[third line no newline]
```

- `len=9` 而 `"line one"` 只有 8 个字符 → **多出来的 1 个正是 `\n`**（`newline=1`）✓
- **最后一行 `newline=0`**、`len=21` 正好等于 `"third line no newline"` 的长度 → **文件末尾没有换行符时，`fgets` 就不会带 `\n`** ✓
- **这正是必须写那句"去掉末尾 `\n`"判断的原因**：末行可能根本没有 `\n`，无脑砍最后一个字符会砍掉真实数据。

**(3) 格式化读写：`fprintf` / `fscanf`（用法与 `printf`/`scanf` 完全一样，只多一个文件指针）**

| 函数 | 对应关系 |
| --- | --- |
| `fprintf(fp, fmt, ...)` | 就是 `printf(fmt, ...)` → `fprintf(stdout, fmt, ...)` |
| `fscanf(fp, fmt, ...)` | 就是 `scanf(fmt, ...)` → `fscanf(stdin, fmt, ...)` |

- `fscanf` **返回成功赋值的项数**，正好可以**当循环结束条件**（这是它最漂亮的地方）：

```
while (fscanf(fp, "%d %s %f", &id, name, &score) == 3) {   /* 必须读满 3 项 */
    /* 处理一条记录 */
}
```

读满一条就返回 3；读到文件末尾返回 `EOF`（−1）；遇到格式不符返回已成功赋值的项数（`< 3`）。**三种情况都能安全退出循环。**

- **完整实测示例：**

```
#include <stdio.h>

int main(void) {
    /* ① 写两条记录 */
    FILE *fp = fopen("build/rec.txt", "w");
    if (!fp) return 1;
    fprintf(fp, "%d %s %f\n", 1, "Tom", 88.5);
    fprintf(fp, "%d %s %f\n", 2, "Amy", 92.0);
    fclose(fp);

    /* ② 逐条读回 */
    fp = fopen("build/rec.txt", "r");
    if (!fp) return 1;
    int id; char name[32]; float score;
    int cnt = 0;
    while (fscanf(fp, "%d %s %f", &id, name, &score) == 3) {
        printf("read: id=%d name=%s score=%.1f\n", id, name, score);
        cnt++;
    }
    printf("fscanf read %d records\n", cnt);
    fclose(fp);
    return 0;
}
```

**实测输出：**

```
read: id=1 name=Tom score=88.5
read: id=2 name=Amy score=92.0
```

```
fscanf read 2 records
```

- **注意**：`fscanf` 用 `%s` 读字符串时**遇空白就停**，所以它**只适合读"不含空格"的记录**（如 `"Tom"`）。要读含空格的整行，还是得用 `fgets`。
- **文本文件的"记录"格式必须事先约定**：上面靠 `%d %s %f` 三个字段固定顺序；字段一多就容易错位，所以真实项目更常用 `fgets` 读一行再用 `sscanf` 拆字段。

**(4) 二进制块读写：`fread` / `fwrite`（结构体数组的存取利器）**

- **参数含义再强调一次**：`fwrite(起始地址, 单个元素字节数, 元素个数, fp)`，**中间两个参数的乘积才是总字节数**。
- **返回值是"元素个数"，不是"字节数"**（常考！）。实测：写 3 个 `sizeof(Stu)=24` 的结构体，返回 **3**，而文件实际大小是 **72** 字节。
- **完整实测示例**（写 3 条记录 → 量文件大小 → 定位读第 2 条 → 整块读回 → 故意多读）：

```c
#include <stdio.h>

typedef struct { int id; char name[16]; float score; } Stu;

int main(void) {
    Stu a[3] = { {1,"Tom",88.5f}, {2,"Amy",92.0f}, {3,"Bob",75.5f}
};

    /* ① 二进制整块写入 */
    FILE *fp = fopen("build/stu.dat", "wb");          /* 注意 "wb"
*/
    if (!fp) return 1;
    size_t w = fwrite(a, sizeof(Stu), 3, fp);
    printf("fwrite wrote %zu elements, sizeof(Stu)=%zu\n", w,
sizeof(Stu));
    fclose(fp);

    /* ② 求文件字节数 */
    fp = fopen("build/stu.dat", "rb");
    if (!fp) return 1;
    fseek(fp, 0L, SEEK_END);                          /* 移到末尾
*/
    long size = ftell(fp);                            /* 当前位置=
总字节数 */
    printf("file size=%ld\n", size);
    rewind(fp);                                       /* 回到开头
*/

    /* ③ 只读第 2 条记录（跳过第 1 条） */
    fseek(fp, 1L * sizeof(Stu), SEEK_SET);
    Stu one;
    size_t r = fread(&one, sizeof(Stu), 1, fp);
    printf("fread got %zu: id=%d name=%s score=%.1f\n", r, one.id,
one.name, one.score);
```

```
    /* ④ 整块读回 */
    rewind(fp);
    Stu b[3];
    size_t rn = fread(b, sizeof(Stu), 3, fp);
    printf("fread %zu records:", rn);
    for (int i = 0; i < 3; i++)
        printf(" %d/%s/%.1f", b[i].id, b[i].name, b[i].score);
    putchar('\n');

    /* ⑤ 故意要 5 条，看返回值 */
    Stu c[5];
    rewind(fp);
    size_t rp = fread(c, sizeof(Stu), 5, fp);
    printf("asked 5 got %zu, feof=%d\n", rp, feof(fp));

    fclose(fp);
    return 0;
}
```

**实测输出：**

```
fwrite wrote 3 elements, sizeof(Stu)=24
file size=72
fread got 1: id=2 name=Amy score=92.0
fread 3 records: 1/Tom/88.5 2/Amy/92.0 3/Bob/75.5
asked 5 got 3, feof=16
```

- `sizeof(Stu)=24`：`int`(4) + `char[16]`(16) + `float`(4) = 24，**没有填充字节**（正好对齐）。
  - **对比警告**：下面第 (6) 小节的综合示例把 `float` 换成了 `double`，`sizeof` 立刻变成 **32**（多了 4 字节填充）。**同一个结构体成员只改一个类型，大小就变了**——所以代码里写 `sizeof(Stu)`、别写死数字。
- `file size=72` = 3 × 24 → **二进制模式下一字不差，1 个元素就是 24 字节** ✓
- **要 5 条只得到 3 条** → `fread` **返回值小于请求值就说明到末尾了**，这是判断"读了多少条"的标准做法 ✓
- `feof=16`：`feof` 返回的是**非零标志值**（实现相关，不一定正好是 1），所以判空要写 `if (feof(fp))` 或 `!= 0`，**不要写** `== 1`。

- **二进制模式的必要条件**：必须用 `"wb"` / `"rb"`。**若用文本模式 `"w"`/`"r"` 存结构体，`\n`（0x0A）会被悄悄加上 `\r`（0x0D），整个文件的字节偏移就乱了**，`fseek` 定位和 `fread` 读回都会错位。

- `fwrite` **的常见误用：**

| 写法 | 问题 |
| --- | --- |
| `fwrite(a, 3, sizeof(Stu), fp)` | **参数顺序反了**，写成"3 字节 × 24 个元素" |

| 写法 | 问题 |
| --- | --- |
| `fwrite(a, sizeof(a), 1, fp)` | 也行（整块当 1 个元素），但返回值含义变成 1 |
| 用 `"w"` 而不是 `"wb"` | 换行转换导致文件损坏 |
| 把含**指针**的结构体 `fwrite` 出去 | 存的是地址，换个进程/重开程序读回来就是**野指针** |

3. 理解文件结束与读写错误的区别；了解 `fseek`、`ftell` 和 `rewind` 的作用，能分析给定的简单文件定位示例。

**解析：**

- **文件结束（EOF）与读写错误是两回事：**
  - EOF 是"正常读完了"，属于预期状态；
  - 错误是"读写失败"（如磁盘损坏、权限问题）。
  - 用 `feof(fp)` 判前者，`ferror(fp)` 判后者，`clearerr(fp)` 清除错误标志。
  - **`feof` 的坑：**即使到达文件末尾，`feof` 在**读操作失败前**仍返回 0。所以应用**函数返回值**判结束，而不是提前调 `feof`。
- **三个定位函数：**

| 函数 | 作用 | 说明 |
| --- | --- | --- |
| `fseek(fp, offset, origin)` | 定位到指定位置 | 成功返回 0 |
| `ftell(fp)` | 返回当前位置 | 相对文件开头的字节偏移 |
| `rewind(fp)` | 回到文件开头 | 等价于 `fseek(fp, 0, SEEK_SET)`，并清除错误标志 |

- **`fseek` 的 `origin` 三个宏：**`SEEK_SET`（文件开头）、`SEEK_CUR`（当前位置）、`SEEK_END`（文件末尾）。`offset` 可为负。
- 典型用法：

```
fseek(fp, 0L, SEEK_END);
long size = ftell(fp);      /* 文件字节数 */
rewind(fp);
```

- 注意：在文本模式下（Windows）`ftell` 返回的值未必等于字节数（因为 `\r\n` 转换），算文件大小宜用二进制模式 `"rb"`。
- **三个定位函数详解（实测数值见后）**

| 函数 | 原型 | 作用 | 返回值 |
| --- | --- | --- | --- |
| `fseek(fp, offset, origin)` | `int fseek(FILE *fp, long offset, int origin)` | 把读写位置移到指定处 | 成功 0，失败非 0 |
| `ftell(fp)` | `long ftell(FILE *fp)` | 返回**当前读写位置**（相对文件开头的字 | 成功返回偏移，失败返回 |

| 函数 | 原型 | 作用 | 返回值 |
| --- | --- | --- | --- |
| | | 节偏移） | `-1L` |
| `rewind(fp)` | `void rewind(FILE *fp)` | 回到文件开头，**并清除错误标志** | 无 |

- `origin` **三个宏的确切含义**（`offset` 相对谁）：

| 宏 | 基准点 | 典型写法 | 含义 |
| --- | --- | --- | --- |
| `SEEK_SET` | **文件开头** | `fseek(fp, 0, SEEK_SET)` | 跳到第 0 字节（= 回到开头） |
| `SEEK_CUR` | **当前位置** | `fseek(fp, -1, SEEK_CUR)` | 往回退 1 个字节 |
| `SEEK_END` | **文件末尾** | `fseek(fp, 0, SEEK_END)` | 跳到文件末尾 |

- `fseek(fp, -1, SEEK_END)`：定位到**最后一个字节**（常用于去掉末尾换行）。
- `fseek` **允许越界到末尾之后**，此时读会返回 `EOF`；写则会**在中间填 0**（形成"空洞文件"）。
- **万能公式：算文件大小**

```
fseek(fp, 0L, SEEK_END);      /* ① 位置移到末尾 */
long size = ftell(fp);        /* ② 此时 ftell 就是总字节数 */
rewind(fp);                   /* ③ 位置复位，否则后面读不到东西 */
```

  **必须用** `"rb"`：文本模式下 Windows 会把 `\n` 存成 `\r\n`，`ftell` 得到的偏移**与真实字节数不一致**。
- **定位到第 i 条记录（二进制文件"随机访问"）**：这正是二进制文件最大的优势——可以**不读前面**直接跳到第 i 条：

```
fseek(fp, (long)i * sizeof(Stu), SEEK_SET);   /* 跳过前 i 条 */
fread(&one, sizeof(Stu), 1, fp);
```

  **原理**：每条记录等长（`sizeof(Stu)` 字节），所以第 i 条的起始偏移就是 $i \times \text{sizeof(Stu)}$。**文本文件做不到这一点**（行长不等，无法按公式算偏移）。
- **实测数值（对上一小节的** `stu.dat`**，3 条 × 24 字节 = 72 字节）**：

| 操作 | 实测结果 | 说明 |
| --- | --- | --- |
| `fseek(fp,0,SEEK_END); ftell(fp)` | `72` | 文件总字节数 |
| `rewind(fp); ftell(fp)` | `0` | 复位成功 |
| `fseek(fp, 1*sizeof(Stu), SEEK_SET); ftell(fp)` | `24` | **正好跳过 1 条记录** |
| 在偏移 24 处 `fread` 一条 | `id=2`<br>`name=Amy` | **直接定位读第 2 条，无需遍历第 1 条 ✓** |

- `feof` **的正确用法与经典陷阱（必考，实测验证）**
  - `feof(fp)` **的含义**：返回**"上一次读操作是否因为撞到文件末尾而失败"的标志。它是一个**事后标志**，不是"是否还有数据"的预测。**

- **所以 `while (!feof(fp))` 是错的**——因为即使只剩最后 0 个字节，`feof` 在你**真正尝试读**之前仍然是 0。**这个循环会多跑一轮。**
- **错误写法的实测过程**（文件 `abc.txt` 内容为 3 字节 `"abc"`，**无换行**）：

```
while (!feof(fp)) {           /* ✗ 错误写法 */
    ch = fgetc(fp);
    printf("read n=%d ch=%d\n", n++, ch);
}
```

**实测输出（多读了第 4 次！）：**

```
read n=0 ch=97      ← 'a'
read n=1 ch=98      ← 'b'
read n=2 ch=99      ← 'c'
read n=3 ch=-1      ← ✗ 多出来的这一轮，EOF 被当成数据"处理"了
```

  - **原因**：第 3 次读拿到 `c` 后，`feof` 仍是 0（还没尝试读第 4 次）；第 4 次读**失败了**、`feof` 才变 1。所以循环体多执行了一次，`ch = -1` 被当成有效数据处理——**典型 bug：文件末尾多输出一个乱码字**。
- **正确写法的实测过程：**

```
while ((ch = fgetc(fp)) != EOF) {     /* ✓ 正确：直接看返回值 */
    printf("read n=%d ch=%d\n", n++, ch);
}
printf("feof=%d ferror=%d count=%d\n", feof(fp), ferror(fp), n);
```

**实测输出：**

```
read n=0 ch=97
read n=1 ch=98
read n=2 ch=99
feof=16 ferror=0 count=3      ← 正好 3 次，不多不少 ✓
```

  - `ferror=0` 说明这次结束是**正常读到末尾**，不是出错。
  - `feof=16` 说明读完后 `feof` 标志已置位（值实现相关，判空写 `if (feof(fp))` 即可）。
- **一句话铁律：永远先读、再看返回值判断；`feof` 只能用于“读完之后的归因”，不能用于“读之前的预判”。**

- **EOF 与读写错误的区分（三个函数的配合）**

| 函数 | 作用 | 备注 |
| --- | --- | --- |
| `feof(fp)` | 是否因**文件结束**而停止 | 事后标志 |
| `ferror(fp)` | 是否发生了**读写错误** | 事后标志 |
| `clearerr(fp)` | **清除**上述两个标志 | `rewind` 也会顺带清除 |

- **标准的"读完归因"写法：**

```
while ((ch = fgetc(fp)) != EOF) { /* 处理 */ }
if (ferror(fp))       printf("读取出错\n");     /* 先查错误 */
else if (feof(fp))    printf("正常读到文件末尾\n");  /* 再查末尾 */
```

**顺序很重要**：先判 `ferror`，因为出错和到末尾都可能让函数返回 `EOF`，需要靠标志区分到底是哪种。

- `EOF` **等价于** `while ((ch = fgetc(fp)) != EOF)` 中的 `EOF` 是宏，值为 **-1**。文件里**可能真的存有 0xFF 字节**，但因为是 `int` 返回，`0xFF` 返回的是 `255` 而不是 `-1`，**不会混淆**——这也是必须用 `int` 接收的原因。

**(5) 选哪个函数？一张表定结论**

| 你要做的事 | 用什么 | 为什么 |
| --- | --- | --- |
| 一个字符一个字符处理 | `fgetc` / `fputc` | 最底层，能拿到每个字节 |
| 按行处理文本 | `fgets` / `fputs` | 能读含空格的行 |
| 按"字段"读数字/单词 | `fscanf` / `fprintf` | 自动跳过空白、自动转换类型 |
| 整块存结构体/数组 | `fread` / `fwrite` | 一次搞定，且能随机定位 |
| 从键盘/屏幕 | `scanf` / `printf` | 等价于对 `stdin`/`stdout` 的上述操作 |

- **一句话选型：给人看的文本用 `fgets`/`fprintf`；给程序存的数据用 `fread`/`fwrite`。**

**(6) 综合完整示例：学生成绩文件的"写入 → 统计 → 追加 → 定位读"**

这是文件操作最典型的考法——把上面所有函数串起来用。**实测可编译运行：**

```
#include <stdio.h>
#include <stdlib.h>

typedef struct { int id; char name[16]; double score; } Stu;

int main(void) {
    const char *path = "build/students.dat";
    Stu a[4] = {
        {1, "Tom",  88.5},
        {2, "Amy",  92.0},
        {3, "Bob",  75.5},
        {4, "Cathy", 95.0}
    };
```

```
    /* ① 全部写入（二进制，可整块存取） */
    FILE *fp = fopen(path, "wb");
    if (fp == NULL) { fprintf(stderr, "创建失败\n"); return 1; }
    size_t w = fwrite(a, sizeof(Stu), 4, fp);
    printf("写入 %zu 条记录，每条 %zu 字节\n", w, sizeof(Stu));
    fclose(fp);

    /* ② 读回全部并统计（平均分 / 最高分） */
    fp = fopen(path, "rb");
    if (fp == NULL) { fprintf(stderr, "打开失败\n"); return 1; }

    fseek(fp, 0L, SEEK_END);
    long bytes = ftell(fp);                         /* 总字节数 */
    rewind(fp);
    int n = (int)(bytes / (long)sizeof(Stu));       /* 记录条数 = 总字节 /
每条字节 */
    printf("文件 %ld 字节，共 %d 条记录\n", bytes, n);

    Stu *all = malloc((size_t)n * sizeof(Stu));     /* 先问出条数，再开数组
*/
    if (all == NULL) { fclose(fp); return 1; }
    size_t rn = fread(all, sizeof(Stu), (size_t)n, fp);
    fclose(fp);

    double sum = 0, max = all[0].score;
    const char *best = all[0].name;
    for (int i = 0; i < (int)rn; i++) {
        printf("  %d %-6s %.1f\n", all[i].id, all[i].name,
all[i].score);
        sum += all[i].score;
        if (all[i].score > max) { max = all[i].score; best =
all[i].name; }
    }
    printf("平均分=%.2f 最高分=%.1f(%s)\n", sum / (double)rn, max,
best);

    /* ③ 随机定位读第 3 条（不遍历前两条） */
    fp = fopen(path, "rb");
    if (fp == NULL) { free(all); return 1; }
    fseek(fp, 2L * (long)sizeof(Stu), SEEK_SET);    /* 跳过前 2 条 */
    Stu one;
    if (fread(&one, sizeof(Stu), 1, fp) == 1)
        printf("第 3 条: id=%d name=%s score=%.1f\n", one.id, one.name,
one.score);
    fclose(fp);
    free(all);

    /* ④ 追加一条（注意用 "ab"，不能用 "wb" 否则清空） */
    fp = fopen(path, "ab");
    if (fp == NULL) return 1;
    Stu extra = {5, "David", 81.0};
    fwrite(&extra, sizeof(Stu), 1, fp);
    fclose(fp);
```

```
    printf("追加 1 条完成\n");
    return 0;
}
```

**实测输出：**

```
写入 4 条记录，每条 32 字节
文件 128 字节，共 4 条记录
  1 Tom    88.5
  2 Amy    92.0
  3 Bob    75.5
  4 Cathy  95.0
平均分=87.75 最高分=95.0(Cathy)
第 3 条：id=3 name=Bob score=75.5
追加 1 条完成
```

- `sizeof(Stu)=32`**（含填充，实测）**。别自己心算，看实测数据：

| 成员 | 类型 | 大小 | 起始偏移 | 说明 |
|---|---|---|---|---|
| `id` | `int` | 4 | **0** | 从 0 开始 |
| `name` | `char[16]` | 16 | **4** | `int` 后接续，偏移 4 |
| （填充） | — | 4 | 20 | `double` 要 8 字节对齐，20 → 24 需补 4 字节 |
| `score` | `double` | 8 | **24** | 对齐到 8 的倍数 |
| 合计 | — | 28 | — | **但总大小必须是最大成员(8)的倍数 → 补齐到 32** |

- **对齐两条规则**：① 每个成员偏移必须是**自身大小**的整数倍；② 结构体总大小必须是**最大成员大小**的整数倍。
- **验证方法**：用 `offsetof` 宏（需 `<stddef.h>`）打印偏移：

```
printf("sizeof=%zu id=%zu name=%zu score=%zu\n",
       sizeof(Stu), offsetof(Stu,id), offsetof(Stu,name),
offsetof(Stu,score));
/* 实测：  sizeof=32 id=0 name=4 score=24 */
```

- **对比：把 `double` 换成 `float` 就不一样了**

| 结构体 | 成员合计 | 填充 | `sizeof` |
|---|---|---|---|
| `{ int; char[16]; double; }` | 28 | 4 | **32** |
| `{ int; char[16]; float; }` | 24 | 0 | **24** |

—— 这就是为什么**写文件时必须用 `sizeof(Stu)` 而不是手算数字**。

- `128 = 4 × 32` ✓ 文件字节数正好是"条数 × 每条字节数"，这正是二进制文件能**随机定位**的基础。
- `平均分=87.75` = (88.5+92.0+75.5+95.0)/4 = 351/4 ✓
- `最高分=95.0(Cathy)` ✓
- **第 3 条是 `id=3 Bob`** —— 用 `fseek(fp, 2*sizeof(Stu), SEEK_SET)` 直接跳过前两条，**一次都没读前两条** ✓
- **追加必须用 `"ab"`**：若用 `"wb"` 会把原有 4 条**全部清空**，只剩新增的 1 条。追加后文件变成 **5 × 32 = 160 字节**。
- `n = bytes / sizeof(Stu)` **这个套路要记住**：二进制文件"先问大小 → 再算条数 → 再 `malloc` → 再整块读回"是最标准的批量处理写法。
- `fprintf(stderr, ...)` **用于报错**：`stderr` 无缓冲，报错信息不会被重定向到结果文件，也不会因为程序崩溃而丢失。**这是好习惯，建议用上**。

**(7) 定位读取数据板子（"查"的标准流程：先 `fseek` 移光标，再 `fgetc`/`fread` 读）**

**先直接回答那个问题：是的，网页/文件里的"查"就是「先把光标移到目标位置，再读」。** 但要分清两种"查"，做法完全不同：

| 场景 | 能不能直接跳过去 | 做法 |
|---|---|---|
| **二进制文件**（记录等长） | ✓ **能**，算偏移直接跳 | `fseek` 算偏移 → `fread` 读整条 |
| **文本文件**（行长不等） | ✗ **不能算**，只能顺序扫 | 循环 `fgets`/`fgetc` 逐行逐字找，找到才停 |

---

**① 文件"光标"到底是什么（理解这一句就全通了）**

- 每个打开的文件都有一个**读写位置**（俗称光标），记录"下一次读/写从第几个字节开始"。
- **打开时**：光标在 **0**（文件开头）。
- **每次读/写后**：光标**自动前进**刚刚读/写的字节数——**不需要你手动移**。
- `ftell(fp)` 读光标位置；`fseek(fp,...)` 手动移动光标。
- **不是"必须 fseek"**：顺序读时 `fgetc`/`fgets`/`fread` 自己会走光标，只有**要跳过一段、或回头重读**时才需要 `fseek`。

**实测验证「读一次光标就自己走一格」**（文件内容 `0123456789`，共 10 字节）：

```
pos=2 -> '2'      ← fseek 到 2
pos=3 -> '3'      ← 没再 fseek，光标自动到 3
pos=4 -> '4'
pos=5 -> '5'
pos=6 -> '6'
```

—— **只 `fseek` 了一次，后面 5 个字符全是自动走的**。这就是「不用每次读都 fseek」的证据。

---

**② 你问的那个写法：`fseek` 移光标 → `fgetc` 读（正确，实测）**

```
FILE *fp = fopen("build/seek.txt", "r");   /* 内容 "0123456789" */
if (fp == NULL) return 1;

int r = fseek(fp, 5L, SEEK_SET);   /* ① 光标移到第 5 字节（从头数，0 起）
*/
printf("fseek 返回值=%d\n", r);

long p = ftell(fp);                /* ② 确认光标位置 */
printf("ftell=%ld\n", p);

int ch = fgetc(fp);                /* ③ 读"当前光标处"的字符 */
printf("fgetc=%c\n", ch);

p = ftell(fp);                     /* ④ 再问一次：光标已前进 1 */
printf("ftell=%ld\n", p);
fclose(fp);
```

**实测输出：**

```
fseek 返回值=0
ftell=5
fgetc=5
ftell=6
```

- `fseek(5, SEEK_SET)` **后** `ftell=5` → 光标确实停在**第 5 个字节**（`'0'` 是第 0 个，所以第 5 个是 `'5'`）✓
- `fgetc` **拿到** `'5'` → **读的就是光标所在的那个字节**，不是从头找 ✓
- **读完** `ftell=6` → 光标**自动前进了一格**，下次读就轮到 `'6'` ✓

⚠️ **一个必须避开的坑（我自己就踩了）：**

```
printf("ftell=%ld fgetc=%c\n", ftell(fp), fgetc(fp));   /* ✗ 结果不确
定！ */
```

- `printf` **参数求值顺序是未规定的**（和第三章 `i++` 那题同源）。上面这行，编译器可能**先执行** `fgetc`、再执行 `ftell`，于是你看到的是 `ftell=6` 而不是期望的 `5`——**看着像 bug，其实是求值顺序的锅**。
- **正确做法：把** `fseek` **/** `ftell` **/** `fgetc` **拆成独立语句**（像我上面实测那样），或在 `printf` 之前先把值存进变量。这既是考试易错点，也是实际写代码的好习惯。

---

**③ 定位读取的三个参数（`origin`）怎么选**

| 想干什么 | 写法 | 实测结果（文件 0123456789） |
|---|---|---|
| 从头数第 5 个 | `fseek(fp, 5, SEEK_SET)` | 读 '5' |
| 从当前位置往后跳 4 个 | `fseek(fp, 4, SEEK_CUR)` | 读 '0' 后跳，读 '5' |
| 读最后 3 个字节 | `fseek(fp, -3, SEEK_END)` | 依次读得 789 |
| 回退一格重读（peek） | `fseek(fp, -1, SEEK_CUR)` | 又读到同一个字符 |

```
/* SEEK_CUR：相对当前位置跳。实测：读 '0'（pos 变 1）→ 跳 +4（到 5）→ 读
'5' */
fgetc(fp);                      /* 读到 '0'，pos=1 */
fseek(fp, 4L, SEEK_CUR);        /* pos = 1 + 4 = 5 */
ftell(fp);                      /* 5 */
fgetc(fp);                      /* '5' */

/* SEEK_END：读末尾 3 个字节。实测输出 789 */
fseek(fp, -3L, SEEK_END);
int c;
while ((c = fgetc(fp)) != EOF) putchar(c);   /* 789 */

/* 回退一格做 peek（看一个字符但不想消费掉）。实测两次都得到 '0' */
int first = fgetc(fp);          /* '0'，pos=1 */
fseek(fp, -1L, SEEK_CUR);       /* 退回 pos=0 */
int again = fgetc(fp);          /* 又是 '0' ✓ */
```

---

**④ 板子一：在文本文件里"查"某一行（只能顺序扫，实测）**

文本文件**行长不等**，没法用乘法算偏移，所以只能**逐行读、边读边比**：

```
#include <stdio.h>
#include <string.h>

int main(void) {
    FILE *fp = fopen("build/lines.txt", "r");
    if (fp == NULL) return 1;

    char buf[256];
    int line_no = 0, found = -1;
    long found_pos = -1;

    while (fgets(buf, sizeof(buf), fp) != NULL) {          /* 逐行读 */
        size_t len = strlen(buf);
        if (len > 0 && buf[len - 1] == '\n') buf[--len] = '\0';   /* 去
掉 '\n' */

        if (strcmp(buf, "line two") == 0) {                /* 找到目标
*/
            found = line_no;
```

```
            found_pos = ftell(fp) - (long)len - 1;          /* 反推行首偏
移 */
            break;                                          /* 找到就停，
不再往下读 */
        }
        line_no++;
    }
    if (found >= 0) printf("找到第 %d 行，行首偏移约 %ld\n", found,
found_pos);
    else            printf("没找到\n");
    fclose(fp);
    return 0;
}
```

- **这就是"查"的本质：没有索引可算，只能从头一行行扫**，找到了用 break 提前收工。
- **为什么不用 fseek 加速**：因为"line two" 在第几个字节取决于前面每行的长度，**事先不知道**。这正是文本文件与二进制文件的分水岭。
- ftell(fp) **在 fgets 之后**得到的是**下一行的行首**，所以要**减掉刚读的长度**才能得到本行行首。

---

**⑤ 板子二：按"第几条记录"直接定位（二进制专用，实测）**

二进制**每条等长**，所以第 i 条的偏移 = $i \times \text{sizeof}(\text{记录})$，**一步跳过去，不用读前 i−1 条**：

```
#include <stdio.h>

int main(void) {
    /* 准备：写入 5 个 int（每条 4 字节） */
    int a[5] = {10, 20, 30, 40, 50};
    FILE *fp = fopen("build/arr.bin", "wb");
    if (fp == NULL) return 1;
    fwrite(a, sizeof(int), 5, fp);
    fclose(fp);

    /* 查：指定要读哪几条（无序也照样能跳） */
    int want[3] = {4, 0, 2};
    fp = fopen("build/arr.bin", "rb");
    if (fp == NULL) return 1;

    for (int k = 0; k < 3; k++) {
        int i = want[k];
        fseek(fp, (long)i * sizeof(int), SEEK_SET);    /* ① 算偏移，跳到
第 i 条 */
        int v;
        if (fread(&v, sizeof(int), 1, fp) == 1)        /* ② 读一条（返回
值=条数） */
            printf("  a[%d]=%d\n", i, v);              /* ③ 用 */
    }
    fclose(fp);
```

```
    return 0;
}
```

**实测输出**（注意顺序是 4 → 0 → 2，**任意跳**）：

```
a[4]=50
a[0]=10
a[2]=30
```

- **每次都从文件开头重新算偏移**（因为 `SEEK_SET` 是绝对定位），所以**可以倒着读、跳着读**。
- `fread` **返回值判成 1** 才说明真读到了；读到越界位置会返回 0，**必须判**。

---

**⑥ 定位读的四个边界行为（实测，容易考）**

| 操作 | 实测结果 | 说明 |
|---|---|---|
| `fseek(fp, 5, SEEK_SET)` | 返回值 **0** | **成功返回 0**，不是返回新位置 |
| `fseek(fp, 20, SEEK_SET)`（文件只 10 字节） | 返回值 0，`ftell=20` | **越界到末尾之后也允许**，不报错 |
| 在越界位置 `fgetc` | 返回 **-1**（`EOF`），`feof=16` | 读越界 → 返回 `EOF` 并置 `feof` 标志 |
| `fseek(fp, -5, SEEK_SET)` | 返回值 **-1** | **移到文件开头之前会失败**（负偏移非法） |

- **两条要记住**：① `fseek` **成功返回 0、失败返回非 0**（不是返回位置！想知道位置要另调 `ftell`）；② **往后越界不报错、往前越界（负位置）才失败**。
- **越界写的后果**：用 `"r+"`/`"wb+"` 越界写会在中间**填 0**，形成"空洞文件"，文件大小会变成你写的那个偏移。

---

**⑦ 一句话总结这个板子**

- **顺序读不用 `fseek`**——读函数自己会推光标（实测 `pos=2→3→4→5→6`）。
- **要跳读就先 `fseek` 再读**——`fseek` 只移光标不读数据，`fgetc`/`fread` 读的是**光标当前所在位置**。
- **能不能直接跳，取决于记录是否等长**：二进制（等长）能算偏移直接跳；文本（不等长）只能顺序扫。
- `fseek` **返回 0 表示成功**，位置要用 `ftell` 问。

**(8) 实战板子：读 YAML 风格的键值对配置文件（`key: value`）**

这类文件（`config.yaml`、`.ini`、`.env`、`.properties`）**每行一条** `键: 值`，是考试和工作里最常见的文本格式。核心就三步：**逐行读 → 找冒号切开 → 左边当键、右边当值**。

---

**① 目标文件长什么样（下面按这个文件实测）**

```
# 服务器配置                    ← 注释行，要跳过
host: 192.168.1.10            ← 普通键值
port: 8080
name: my-server
                              ← 空行，要跳过
  debug: true                 ← 前置空格，要 trim 掉
timeout: 30                   ← 值后面可能有空格，也要 trim
#comment-only: nope           ← 整行被注释，不能当键值
bad_line_no_colon             ← 没有冒号，非法行，要跳过
desc: hello world with spaces  ← 值里可以含空格！
```

---

**② 完整可运行解析代码（实测）**

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAXLINE 256

/* 原地去掉首尾空白，返回 trim 后的起始指针 */
static char *trim(char *s) {
    while (*s && isspace((unsigned char)*s)) s++;          /* 跳过前导空白 */
    if (*s == '\0') return s;
    char *end = s + strlen(s) - 1;
    while (end >= s && isspace((unsigned char)*end)) end--; /* 从尾部回退 */
    end[1] = '\0';
    return s;
}

int main(void) {
    FILE *fp = fopen("build/config.yaml", "r");
    if (fp == NULL) { fprintf(stderr, "打开失败\n"); return 1; }

    char line[MAXLINE];
    int lineno = 0, kv = 0, skip = 0;

    while (fgets(line, sizeof(line), fp) != NULL) {        /* ① 逐行读 */
        lineno++;

        size_t len = strlen(line);                          /* ② 去掉尾部 '\n' */
        if (len > 0 && line[len - 1] == '\n') line[len - 1] = '\0';

        char *s = trim(line);                               /* ③ 去首尾空白 */
        if (*s == '\0') { skip++; continue; }               /* 空行：跳
```

```
过 */
        if (*s == '#')  { skip++; continue; }                /* 注释：跳
过 */

        char *colon = strchr(s, ':');                        /* ④ 找第一
个冒号 */
        if (colon == NULL) {                                 /* 没冒号=非
法行 */
            printf("L%d 跳过（无冒号）: [%s]\n", lineno, s);
            skip++;
            continue;
        }

        *colon = '\0';                                       /* ⑤ 把冒号
改成 '\0' */
        char *key = trim(s);                                 /*    左半
= 键 */
        char *val = trim(colon + 1);                         /*    右半
= 值 */

        printf("L%d key=[%s] value=[%s]\n", lineno, key, val);
        kv++;
    }
    fclose(fp);
    printf("---\n共 %d 行，解析出 %d 对键值，跳过 %d 行\n", lineno, kv,
skip);
    return 0;
}
```

**实测输出：**

```
L2  key=[host]    value=[192.168.1.10]
L3  key=[port]    value=[8080]
L4  key=[name]    value=[my-server]
L6  key=[debug]   value=[true]            ← 前置空格被 trim 掉了
L7  key=[timeout] value=[30]              ← 值后的空格被 trim 掉了
L9  跳过（无冒号）: [bad_line_no_colon]    ← 非法行被识别出来
L10 key=[desc]    value=[hello world with spaces]  ← 值里的空格完整保留
---
共 10 行，解析出 6 对键值，跳过 4 行
```

- **跳过 4 行** = 注释(1) + 空行(1) + 注释(1) + 非法行(1) ✓ 与上面文件的构造完全对上。
- `L6 key=[debug]`（**不是** `[ debug]`） → `trim` **生效了**，前置空格没进键名 ✓
- `desc` **的值** `hello world with spaces` **完整保留** → **只切第一个冒号**，不在空格处断 ✓

---

**③ 四个关键点（都是必须处理的坑）**

| 要处理的 | 不处理会怎样 | 怎么处理 |
| --- | --- | --- |
| **尾部 `\n`** | 值末尾多个 `\n`，`strcmp` 判等永远失败 | `if (line[len-1]=='\n') line[--len]='\0';` |
| **空行 / 注释** | 被当成键值，解析出垃圾数据 | `if (*s=='\0' \|\| *s=='#') continue;` |
| **冒号前后空格** | 键变成 `"port "`，查不到 | **切完冒号后再各 `trim` 一次** |
| **行里没有冒号** | `strchr` 返回 `NULL`，**直接解引用就崩溃** | 先判 `colon == NULL` 再往下走 |

- **最危险的是最后一条**：`char *colon = strchr(s, ':'); *colon = '\0';` 如果这行没有冒号，`colon` 是 `NULL`，`*colon = '\0'` 就是**向空指针写——必然崩溃。任何返回指针的查找函数都必须判 `NULL`**。

---

**④ `trim` 函数的写法要点（这个函数值得背）**

```
static char *trim(char *s) {
    while (*s && isspace((unsigned char)*s)) s++;   /* ① 前导：直接后移指针 */
    if (*s == '\0') return s;                        /* ② 全是空白的边界情况 */
    char *end = s + strlen(s) - 1;                   /* ③ 指向最后一个字符 */
    while (end >= s && isspace((unsigned char)*end)) end--;  /* ④ 从后往前退 */
    end[1] = '\0';                                   /* ⑤ 把后面的空白截断 */
    return s;                                        /* ⑥ 返回新起点！
*/
}
```

- **① 前导空白用「移动指针」**（`s++`），**尾部空白用「写 `'\0'`」**（`end[1] = '\0'`）。两种手法不同，别混。
- **② 必须单独判空串**：若整行都是空白，`strlen(s)-1` 会算出 **-1**，`s + (-1)` **越界读**。
- **⑥ 必须返回新指针**：因为前导空白是靠 `s++` 跳过的，**原来的调用者不知道新起点在哪**。所以用法是 `s = trim(s);` 或 `char *key = trim(s);`，**不能写 `trim(s);` 就完事**。
- **`(unsigned char)` 强转是标准做法**：`isspace` 接收 `int`，若传负的 `char`（如中文字节）是未定义行为，转成 `unsigned char` 才安全。

---

**⑤ 真正实用的用法：封装成「按 key 查 value」的函数（这就是配置文件读取的成品）**

实际项目里不会反复手写解析循环，而是封一个 `lookup`：**打开文件 → 逐行找 → 命中就返回**。

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
```

```c
#include <ctype.h>

#define MAXLINE 256

static char *trim(char *s) {
    while (*s && isspace((unsigned char)*s)) s++;
    if (*s == '\0') return s;
    char *end = s + strlen(s) - 1;
    while (end >= s && isspace((unsigned char)*end)) end--;
    end[1] = '\0';
    return s;
}

/* 查：按 key 找 value。返回 1=找到，0=没这个键，-1=文件打不开 */
static int lookup(const char *path, const char *want, char *out, size_t
outsz) {
    FILE *fp = fopen(path, "r");
    if (fp == NULL) return -1;                      /* 文件打不开 */

    char line[MAXLINE];
    int found = 0;
    while (fgets(line, sizeof(line), fp) != NULL) {
        size_t len = strlen(line);
        if (len > 0 && line[len - 1] == '\n') line[len - 1] = '\0';

        char *s = trim(line);
        if (*s == '\0' || *s == '#') continue;      /* 空行/注释跳过 */

        char *colon = strchr(s, ':');
        if (colon == NULL) continue;                /* 无冒号跳过 */

        *colon = '\0';
        char *key = trim(s);
        char *val = trim(colon + 1);

        if (strcmp(key, want) == 0) {               /* 命中 */
            snprintf(out, outsz, "%s", val);        /* 安全拷贝，自动截
断 */
            found = 1;
            break;                                  /* 找到就停，不再往
下读 */
        }
    }
    fclose(fp);
    return found;
}

int main(void) {
    const char *path = "build/config.yaml";
    char buf[128];

    /* ① 逐项查询（含一个不存在的键） */
    const char *keys[4] = {"host", "port", "name", "missing_key"};
    for (int i = 0; i < 4; i++) {
```

```
        int r = lookup(path, keys[i], buf, sizeof(buf));
        if (r == 1)      printf("查 %-12s -> [%s]\n", keys[i], buf);
        else if (r == 0) printf("查 %-12s -> 没有这个键\n", keys[i]);
        else             printf("文件打不开\n");
    }

    /* ② 取出来当数字用 */
    if (lookup(path, "port", buf, sizeof(buf)) == 1) {
        int port = atoi(buf);                          /* 字符串 -> int
*/
        printf("port=%d, port+1=%d\n", port, port + 1);
    }

    /* ③ 取出来当布尔用 */
    if (lookup(path, "debug", buf, sizeof(buf)) == 1) {
        int debug = (strcmp(buf, "true") == 0);
        printf("debug=%d\n", debug);
    }

    /* ④ 带缺省值：查不到就用默认 */
    const char *timeout = (lookup(path, "timeout", buf, sizeof(buf)) ==
1) ? buf : "60";
    printf("timeout=%s\n", timeout);
    const char *workers = (lookup(path, "workers", buf, sizeof(buf)) ==
1) ? buf : "60";
    printf("workers=%s\n", workers);
    return 0;
}
```

**实测输出：**

```
查 host         -> [192.168.1.10]
查 port         -> [8080]
查 name         -> [my-server]
查 missing_key  -> 没有这个键
port=8080, port+1=8081
debug=1
timeout=30
workers=60
```

- **三种返回值分得清**：`1` 找到、`0` 没这个键、`-1` 文件打不开。**把「查不到」和「出错」分开**，调用者才能分别处理。
- `atoi(buf)` **把字符串转成数字**：实测 `port=8080`，`port+1=8081`。**注意 `atoi` 不做错误检查**（`"abc"` 会静默返回 0），要严格检查可用 `strtol`。
- **布尔值靠自己判**：YAML 里的 `true` 只是**字符串**，`strcmp(buf,"true")==0` 才得到 1。**C 没有内置布尔解析**。
- **缺省值三目写法很实用**：`lookup(...)==1 ? buf : "60"` ——`timeout` 从文件读到 **30**，`workers` 文件里没有所以取缺省 **60** ✓

- `snprintf(out, outsz, "%s", val)` **而不是** `strcpy`：`snprintf` 会**限制长度、不会溢出**，是往调用者缓冲区写数据的正确做法（对应第四章 `strcpy` 越界的坑）。

---

**⑥ 这个板子和前面「查某一行」是同一个套路**

| 对比 | 查某一行 | 查某个键 |
| --- | --- | --- |
| 读法 | `fgets` 逐行 | `fgets` 逐行 |
| 比较对象 | 整行 `strcmp(buf, "line two")` | 冒号左半 `strcmp(key, "port")` |
| 切分 | 不需要 | `strchr(s,':')` + 写 `'\0'` |
| 命中后 | `break` | `break` |

- **两者都不能 `fseek` 直接跳**——因为文本文件**行长不等**，`"port"` 在第几个字节事先算不出来，只能顺序扫（见上面 ⑦ 的结论）。
- **唯一能算偏移直接跳的情况**：把所有键值**预先加载到内存数组**，或者把文件存成**等长的二进制记录**。前者更常见：

```
char keys[64][32];      /* 最多 64 个键，每个键最长 31 字符 */
char vals[64][128];
int  n = 0;
/* 解析时：snprintf(keys[n], 32, "%s", key); snprintf(vals[n],
128, "%s", val); n++; */
/* 之后在内存里查，就不必再碰文件了 */
```

**加载进内存**是配置文件的标准做法：文件只读一次，之后所有查询都在数组里做，又快又不用反复开文件。

**提示**：这部分重点考查根据任务选择文件打开模式、完成基本读写和定位操作，并正确使用函数返回值。涉及读写错误、流状态、部分读写、阻塞、原子性与持久化等问题时，题目将提供必要材料，要求结合材料和已学知识分析简单程序，不要求预先掌握相关底层机制。

## 十一、综合程序设计与材料阅读

1. 掌握累加、累乘、计数、求最值、整数数位处理、素数判断、简单递推和枚举等基本方法；了解顺序查找、二分查找、冒泡排序和简单选择排序的基本概念，了解各方法的适用条件。

**解析**：

- **基本算法模板（必会）**：

```
/* 累加 */ sum = 0; for (i = 1; i <= n; i++) sum += i;
/* 累乘 */ prod = 1; for (i = 1; i <= n; i++) prod *= i;
/* 计数 */ cnt = 0; for (...) if (条件) cnt++;
/* 最值 */ max = a[0]; for (i = 1; i < n; i++) if (a[i] > max) max
= a[i];
/* 求平均 */ sum = 0; for (...) sum += a[i]; avg = (double)sum /
n;
```

- **整数数位处理**（非常高频）：从低位到高位用 `% 10` + `/ 10`；从高位到低位需先算位数或递归。

```
/* 求位数 */ int d(int n) { int k = 0; do { k++; n /= 10; } while
(n); return k; }
/* 逐位拆分 */ while (n > 0) { int dig = n % 10; n /= 10; /* 处理
dig */ }
```

- **「对 mask 取余」数位处理板子（核心：把 `10` 当 mask，`%` 取低位、`/` 去低位）**
  - **一句话原理**：`n % 10` 取出最低位，`n / 10` 把它丢掉。反复做就逐位取完。这就是「对 mask 取余」——**mask 就是 10**（或 $10^k$）。
  - **① 低位 → 高位（最常用）**：

```
while (t > 0) {
    int dig = t % 10;      /* 取最低位 */
    t /= 10;               /* 丢掉最低位 */
}
/* 1234567 → 7 6 5 4 3 2 1（低位在前） */
```

  - **② 高位 → 低位（先造 mask，`/` 取、`%` 去）**：

```
long long p = 1;
while (p * 10 <= n) p *= 10;     /* mask = 10^(位数-1) */
while (p > 0) {
    int dig = t / p;           /* 用除法取最高位 */
    t %= p;                    /* 用取余丢掉最高位 */
    p /= 10;                   /* mask 退一位 */
}
/* 1234567 → 1 2 3 4 5 6 7（高位在前） */
```

  - **③ 取 / 去掉「低 k 位」：mask 换成 $10^k$**

| 想要 | 写法 |
| --- | --- |
| 取低 k 位 | `y % 10^k` |

| 想要 | 写法 |
| --- | --- |
| 去掉低 k 位 | `y / 10^k` |
| 取第 m~n 位 | `(y / 10^m) % 10^{n-m+1}` |

**实测** `y = 987654321`：k=1 → `1`／k=2 → `21`／k=3 → `321`／k=4 → `4321`。

- **④ 位运算版（同样思想，换 mask）**：`& (2^n-1)` 等价 `% 2^n`，`>> n` 等价 `/ 2^n`。

```
x & ((1 << (4*k)) - 1);    /* 取低 k 个十六进制位 */
x >> (4 * k);              /* 去掉低 k 个十六进制位 */
```

**实测** `x = 0xABCD`：`& 0xF` = `0xD` → `& 0xFF` = `0xCD` → `& 0xFFF` = `0xBCD`。

- **⑤ 数位和**：`while (t) { sum += t % 10; t /= 10; }`（**实测 1234567 → 28**）。
- **⑥ 回文判断（掐头去尾）**：

```
while (w > 0) {
    if (w / div != w % 10) { ok = 0; break; }    /* 比较最高位与最低位 */
    w = (w % div) / 10;                          /* 一步掐掉头尾 */
    div /= 100;                                   /* 一次去两位 */
}
```

- **⑦ 负数陷阱（必考）**：**实测** `-1234 % 10 = -4`（不是 4），因为余数符号跟被除数。后果是 `while (n > 0)` 对负数一次都不循环。

```
if (n < 0) n = -n;       /* 必须先取绝对值 */
```

- **⑧ 速查表**

| 要干什么 | 写法 |
| --- | --- |
| 取最低位 | `n % 10` |
| 去掉最低位 | `n / 10` |
| 取最低 k 位 | `n % 10^k`（十六进制 `n & (2^{4k}-1)`） |
| 去掉最低 k 位 | `n / 10^k`（十六进制 `n >> 4k`） |
| 取最高位 | 先算 $p = 10^{\text{位数}-1}$，再 `n / p` |
| 去掉最高位 | `n % p` |

| 要干什么 | 写法 |
| --- | --- |
| 求位数 | `while (p * 10 <= n) p *= 10;` 后数 `p` 的位数 |
| 数位和 | `while (t) { sum += t % 10; t /= 10; }` |

  - **整套只有一个思想：用 $10^k$ 当 mask，`%` 拿低位，`/` 丢低位。**位运算版把 $10^k$ 换成 $2^k$、`%` 换成 `&`、`/` 换成 `>>`，其余完全一样。
- **素数判断**：只需试除到 $\sqrt{n}$。**循环条件写 `i <= n / i`，别写 `i * i <= n`**——`i * i` 在 $n$ 接近 `INT_MAX` 时会溢出成负数（实测 `46341 * 46341 = -2147479015`），循环提前退出，把素数误判成合数。

```
int is_prime(int n) {
    if (n < 2) return 0;                    /* 0、1 不是素数，2 是唯一的偶素数 */
    for (int i = 2; i <= n / i; i++)        /* 两边都 ≤ n，不会溢出 */
        if (n % i == 0) return 0;
    return 1;
}
```

  - **两种写法数学上等价**：$i \le \frac{n}{i} \iff i^2 \le n$（`i`、`n` 为正时成立），但 `n / i` 不会有溢出。
  - **也可用**：`for (int i = 2; (long long)i * i <= n; i++)`（强转 `long long`）。
- **枚举（穷举）**：逐个试所有可能。适合范围小、规则复杂而不易用公式的题（“百鸡问题”“水仙花数”）。
- **递推**：从已知初值出发，用前项推导后项。如阶乘、斐波那契、杨辉三角。

```
int fib[100]; fib[0] = 0; fib[1] = 1;
for (i = 2; i < n; i++) fib[i] = fib[i-1] + fib[i-2];
```

- **四种查找/排序对比（记适用条件）：**

| 方法 | 适用条件 | 平均时间复杂度 | 特点 |
| --- | --- | --- | --- |
| 顺序查找 | 任意 | $O(n)$ | 简单，无需有序 |
| 二分查找 | **必须已有序** | $O(\log n)$ | 快，但需有序 |
| 冒泡排序 | 任意 | $O(n^2)$ | 相邻两两交换，每轮冒出最值 |
| 简单选择排序 | 任意 | $O(n^2)$ | 每轮选中最小（大）值放到已排序末尾 |

- **二分查找模板（必记）：**

```
int lo = 0, hi = n - 1;
while (lo <= hi) {
    int mid = (lo + hi) / 2;
```

```
        if (a[mid] == key) return mid;
        else if (a[mid] < key) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
```

- **冒泡排序关键点**：内层是 `j < n-1-i`，**减** `i` 是为了避免重复扫描已排好的部分。
- **简单选择排序**：$i$ 从 0 到 $n-2$，找到 $[i, n-1]$ 中最小的下标，再与 $a[i]$ 交换。
- **整数溢出提醒**：求和大、阶乘大时用 `long long`；`int` 最大只有约 21 亿。

2. 能根据题意选择数据类型和组织方式，将处理过程分解为函数，编写完成基本任务的程序；能追踪变量及数据变化，补全关键语句，指出并修改常见错误。

**解析**：

- **从题意到代码的四步**：① 找**输入**（几个量、什么类型）→ ② 找**输出**（算什么、什么格式）→ ③ 找**关系/公式**（题目给的规则）→ ④ 找**流程**（顺序/选择/循环）。
- **数据类型选择**：计数用 `int`；大数求和/阶乘用 `long long`；平均值/除法用 `double`（并强转）；字符串用 `char` 数组。
- **模块化**：每个子任务一个函数，用**函数名体现意图**；参数传递采用"**传入数据 + 返回结果**"模式。
- **手跟代码（追踪变量）**：画一张表，列出循环变量 + 关键变量的值，逐行填写。这是检查逻辑错误最快的方法。
- **找出常见错误的口袋清单**：
  - 边界：循环是 `<` 还是 `<=`；数组下标是否超界；字符串是否有 `'\0'`。
  - 类型：`/` 是否写成了整除；是否忘了强转；`scanf` 读 `double` 是否误用了 `%f`（应用 `%lf`）。
  - 输入输出：`scanf` 是否忘了 `&`；格式串是否匹配。
  - 指针：是否可能为 `NULL`；是否越界；`malloc` 后是否检查、是否 `free`。
  - 逻辑：`=` 与 `==`；`&&` 与 `||`；初值是否写对（最值、乘积）。

3. 能从文字、表格和示意图中提取条件，将给定规则转化为计算步骤或程序。材料题涉及的新增概念、规则及必要的计算机组成背景由题目给出，不要求提前记忆本大纲所列知识点之外的专门知识。

**解析**：

- **材料题解题三步**：
  1. **划出量**：把材料中出现的每个数据项、每个状态都列出来，并定好变量名与类型。
  2. **写出规则表**：把文字/表格/图中的规则逐条转成"条件 → 动作"的表格，尤其注意**边界与特殊情况**。
  3. **先走个例子**：用手算跑一遍小例子，确认规则理解对，再写代码。
- **常见的规则形式**：
  - "如果……则……"→ `if-else`
  - "按……分类/分段"→ `if-else if` 梯或 `switch`
  - "重复直到……"→ `while` / `do-while`
  - "第 n 项等于前几项的……"→ 递推/递归
  - "从表中查……"→ 数组存储 + 遍历查找

> - **注意“必须用给定规则，不要自作主张”**：材料题考的是**按规则执行**的能力，不能凭生活经验改写规则。
> - **优先保证可读性**：变量名取有意义的名字，关键步骤写注释，把处理过程拆成函数。

# 附录：常用记忆清单

## A. 常用头文件与函数速查

> - **考试时最常出现的 5 个头文件**（前四个几乎每套题都会出现）：

| 头文件 | 主要函数 | 要记住的要点 |
|---|---|---|
| `<stdio.h>` | `printf scanf fopen fclose fgetc fputc fgets fputs fscanf fprintf fread fwrite fseek ftell rewind feof ferror getchar putchar` | **不用包含也能用**（`printf`/`scanf` 是隐式声明）但必须包含才规范；`stdin`/`stdout`/`stderr` 在这里定义 |
| `<string.h>` | `strlen strcmp strncmp strcpy strncpy strcat strncat strchr strrchr strstr strtok memset memcpy memcmp` | 字符串函数**必须包含**；`strlen` 不含 `'\0'`，`sizeof` 含 |
| `<stdlib.h>` | `malloc calloc realloc free atoi atof abs qsort exit` | **`malloc` 返回 `void*`**，取下标前要先转类型；`atoi` 不做错误检查 |
| `<ctype.h>` | `isdigit isalpha isalnum isspace isupper islower toupper tolower` | 参数要传 `unsigned char`（或确认非负）；返回**非 0 即真**，不一定是 1 |
| `<math.h>` | `sqrt pow fabs floor ceil round fmod sin cos log exp` | **参数和返回值都是 `double`**；`fmod` 用于浮点求余（整型用 `%`）；**加 `-lm`**（gcc 需要） |

> - **另外 3 个"常量表"头文件**（写边界判断时用，比硬编码数字好）：

| 头文件 | 常用宏 | 实测值 |
|---|---|---|
| `<limits.h>` | `INT_MAX INT_MIN UINT_MAX CHAR_BIT CHAR_MIN CHAR_MAX SHRT_MAX LLONG_MAX` | `2147483647` / `-2147483648` / `4294967295` / `8` / `-128` / `127` / `32767` / `9223372036854775807` |
| `<float.h>` | `FLT_EPSILON DBL_EPSILON` | `1.1920929e-007` / `2.2204460492503131e-016` |
| `<stddef.h>` | `NULL size_t offsetof` | `offsetof(struct Pair, b)` 实测 `8`，`sizeof(Pair)` = `16` |

> - **实测输出（可直接对照）**：

| 表达式 | 实测结果 | 说明 |
|---|---|---|
| `strlen("hello")` | `5` | 不含 `'\0'` |
| `strcmp("a","b")` | `-1` | **只保证符号**，别写 `== -1` |
| `strncmp("abc","abd",2)` | `0` | 只比前 2 个 → 相等 |

| 表达式 | 实测结果 | 说明 |
| --- | --- | --- |
| `strcat("ab","XY")` | `abXY` | 追加到 `dst` 末尾 |
| `strncat("cd","ZZZ",2)` | `cdZZ` | 只追加 2 个字符 |
| `strchr("a/b/c", '/')` | `/b/c` | **第一个**匹配处起 |
| `strrchr("a/b/c", '/')` | `/c` | **最后一个**匹配处起 |
| `strstr("hello world","wor")` | `world` | 返回**子串起点** |
| `strtok("a,b,c", ",")` + `NULL` | `[a] [b] [c]` | **第一次传串，之后传 `NULL`** |
| `atoi("123abc")` | `123` | 遇到非数字就停 |
| `atof("3.14xyz")` | `3.14` | 同上 |
| `strtol("0x1F", NULL, 16)` | `31` | 可指定进制，比 `atoi` 稳 |
| `qsort(arr, 5, sizeof(int), cmp)` | `1 2 3 4 5` | 需自己写比较函数 |
| `isdigit('7')` / `isupper('A')` | `1` / `1` | 真值非 0 |
| `isspace(' ')` | `8` | **非 0 即真**，不一定是 1 |
| `sqrt(2)` / `pow(2,10)` | `1.4142` / `1024.0` | 都是 `double` |
| `floor(2.7)` / `ceil(2.1)` / `round(2.5)` | `2.0` / `3.0` / `3.0` | `round(2.5)` 得 3（**四舍五入，不是银行家舍入**） |
| `fmod(7,3)` | `1.0` | **浮点**求余 |

> - **三个最容易踩的坑：**
>   1. **`<string.h>` 一定要包含**——`strlen`/`strcpy` 不包含就是隐式声明，64 位下返回值被当 `int`，**可能得到错误结果**。
>   2. **`<math.h>` 链接时要加 `-lm`**（`gcc a.c -o a -lm`），否则报 `undefined reference to 'sqrt'`。**注意 `-lm` 要放在源文件之后**。
>   3. **`<ctype.h>` 的函数不能直接传 `char`**：若 `char` 是负数（如中文字节），传进去是未定义行为。标准写法：`isdigit((unsigned char)ch)`。
> - **一句话记忆：输入输出找 `stdio`，字符串找 `string`，内存/转换找 `stdlib`，字符判断找 `ctype`，数学找 `math`，边界常量找 `limits`。**

B. 高频考点与易错点速查

| 分类 | 必记内容 |
| --- | --- |
| **整数** | `int` 4 字节、−2147483648~2147483647；除法**向零截断**；`%` 符号跟**被除数** |
| **补码** | 正数照抄，负数**取反加一**；范围 $-2^{n-1} \sim 2^{n-1}-1$；`~x = -x-1` |
| **浮点** | `float` 4 字节、`double` 8 字节；**不能** `%`；比较用 `fabs(a-b) < 1e-6` |
| **转换** | 窄 → 宽自动提升；**有符号遇无符号，有符号转无符号**（`-1 > 0u` 为真） |

| 分类 | 必记内容 |
| --- | --- |
| `sizeof` | 运算符不是函数；**编译期求值、不求值操作数**；`sizeof('A') == 4` |
| **字符串** | 末尾必须有 `'\0'`；`strlen` 不含 `'\0'`；`strcpy` 需求 `strlen+1` 字节 |
| **数组** | 下标 `0 ~ n-1`；二维**按行优先**；传参后退化为指针（`sizeof` 变 8） |
| **指针** | `a[i]` ⇔ `*(a+i)`；`p+1` 加 `sizeof(*p)` 字节；`const` 靠左锁值、靠右锁指针 |
| **函数** | 按值传递；数组/结构体想改实参必须传指针；`int *f(int)` 与 `int (*f)(int)` 不同 |
| **存储期** | `static` 局部：只初始化一次、值保留；全局/静态未初始化自动为 0 |
| **内存** | `malloc` 后必检 `NULL`、必 `free`、`free` 后置 `NULL`；**不要 `p = realloc(p,...)`** |
| **文件** | 模式 `r/w/a` + `b`；`"w"` **会清空**；`feof` 只能事后判，**不能 `while(!feof)`** |
| **宏** | 纯文本替换；**整体和每个参数都要加括号**；参数可能求值多次 |

> **提示**：附录内容是前文的索引式浓缩，用于考前快速过一遍，具体细节仍以各章正文的解析为准。